

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG GIÁM SÁT VÀ PHÁT HIỆN
BẤT THƯỜNG MẠNG ZERO-TRUST
TRÊN NỀN TẢNG KUBERNETES**

Sinh viên thực hiện: Mạch Tiến Duy

MSSV: CT07N0111

Lớp: CT07

Giảng viên hướng dẫn: Nguyễn Minh Đế

Hồ Chí Minh, tháng 4 năm 2026

HỌC VIỆN KỸ THUẬT MẬT MÃ
KHOA CÔNG NGHỆ THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG GIÁM SÁT VÀ PHÁT HIỆN
BẤT THƯỜNG MẠNG ZERO-TRUST
TRÊN NỀN TẢNG KUBERNETES**

Sinh viên thực hiện: Mạch Tiến Duy

MSSV: CT07N0111

Lớp: CT07

Giảng viên hướng dẫn: Nguyễn Minh Đế

Hồ Chí Minh, tháng 4 năm 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp, em đã nhận được sự hướng dẫn, góp ý và hỗ trợ tận tình từ nhiều thầy cô, bạn bè và gia đình. Em xin gửi lời cảm ơn chân thành tới:

ThS. Nguyễn Minh Đế, giảng viên hướng dẫn đồ án tốt nghiệp, đã tận tình hướng dẫn, đưa ra định hướng và đóng góp nhiều ý kiến quý báu giúp em hoàn thành đồ án.

Các thầy cô trong Khoa Công nghệ thông tin, Học viện Kỹ thuật Mật mã đã truyền đạt kiến thức chuyên ngành về cấu trúc dữ liệu, giải thuật, hệ thống phân tán và an toàn thông tin, tạo tiền đề vững chắc cho việc nghiên cứu và triển khai đồ án.

Gia đình, bạn bè và đồng nghiệp đã luôn động viên, chia sẻ kinh nghiệm và hỗ trợ em trong suốt quá trình học tập và thực hiện đồ án.

Mặc dù đã cố gắng hoàn thiện đồ án ở mức tốt nhất có thể, song không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý từ quý thầy cô và hội đồng để đồ án được hoàn thiện hơn.

Hồ Chí Minh, tháng 6 năm 2026

Sinh viên thực hiện

Mạch Tiến Duy

LỜI CAM ĐOAN

Em xin cam đoan đồ án tốt nghiệp “Hệ thống giám sát và phát hiện bất thường mạng Zero-Trust trên nền tảng Kubernetes” là công trình nghiên cứu của riêng em, được thực hiện dưới sự hướng dẫn của **ThS. Nguyễn Minh Đế**.

Các nội dung nghiên cứu, kết quả thực nghiệm và số liệu trình bày trong đồ án là trung thực và do chính em thực hiện. Những phần kế thừa từ Báo cáo học phần tốt nghiệp (CNPM 1) của bản thân đã được nêu rõ và trích dẫn đầy đủ. Mọi định nghĩa, kết quả và thông tin tham khảo từ các tài liệu khác đều được trích dẫn và liệt kê trong mục Tài liệu tham khảo theo đúng quy định.

Nếu phát hiện có bất kỳ sự gian lận nào, em xin hoàn toàn chịu trách nhiệm về nội dung đồ án của mình.

Hồ Chí Minh, tháng 6 năm 2026

Sinh viên thực hiện

Mạch Tiến Duy

MỞ ĐẦU

Trong khoảng một thập kỷ trở lại đây, kiến trúc phần mềm doanh nghiệp đã dịch chuyển mạnh mẽ từ ứng dụng *một khối* (*monolithic*) sang *vi dịch vụ* (*microservices*) đóng gói trong container và điều phối bằng Kubernetes. Sự dịch chuyển này mang lại tốc độ phát triển và khả năng mở rộng vượt trội, nhưng đồng thời làm thay đổi căn bản bề mặt tấn công: thay vì các lời gọi nội bộ gọn trong một tiến trình, hệ thống nay có hàng trăm điểm cuối nội bộ liên lạc với nhau qua mạng. Phần lưu lượng nội cụm (hướng Đông–Tây) này gần như vô hình trước các biện pháp an ninh chu vi truyền thống, trở thành mảnh đất cho hai hành vi tấn công điển hình: *quét cổng* để trinh sát và *dịch chuyển ngang* để mở rộng quyền kiểm soát giữa các pod.

Đặt vấn đề. Làm thế nào để *quan sát* được trọn vẹn lưu lượng nội cụm và *phát hiện sớm* các hành vi quét cổng, dịch chuyển ngang ngay trong một cụm Kubernetes, với chi phí đủ thấp để chạy thường trực, rồi *khép kín* bằng một cơ chế phản hồi có kiểm soát? Đây là bài toán mà đề án hướng tới giải quyết.

Nội dung đề án. Đề án xây dựng và đánh giá một hệ thống giám sát — phản hồi an ninh đầu cuối cho cụm Kubernetes, đặt tên K8sSecDaP. Hệ thống thu thập sự kiện kết nối ngay tại *tầng nhân* bằng công nghệ eBPF, phát hiện bất thường bằng các *cấu trúc dữ liệu luồng* (Count-Min Sketch) và *giải thuật đồ thị* (Tarjan, BFS), quản lý sự cố qua một cụm dịch vụ SOC, và khép kín vòng phản hồi bằng cách sinh chính sách mạng NetworkPolicy có người duyệt để cách ly nguồn tấn công. Toàn bộ được triển khai và đo đạc trên một cụm Kubernetes thật.

Lý do chọn đề tài. Trong bối cảnh nêu trên, mô hình *Zero-Trust* — nơi mọi kết nối đều bị coi là không đáng tin cho đến khi được xác thực — là một nguyên tắc được giới học thuật và công nghiệp đồng thuận [15], nhưng việc *hiện thực hoá* nguyên tắc này trên một cụm Kubernetes nội bộ vẫn còn nhiều vướng mắc kỹ thuật. Điều khiến Zero-Trust khó hiện thực hóa nội cụm không nằm ở nguyên lý mà ở *khả năng quan sát*: muốn “không tin tưởng mặc nhiên mọi kết nối” thì trước hết phải *thấy* được mọi kết nối. Đây cũng là lý do đề án chọn quan sát ở tầng nhân bằng eBPF làm điểm tựa kỹ thuật: nó cung cấp tầm nhìn Đông–Tây mà các lớp giám sát hướng host không có, từ đó mới có thể đặt nền cho phát hiện và phản hồi có kiểm soát. Đề án cũng tiếp nối và mở rộng kết quả của Báo cáo học phần tốt nghiệp (CNPM 1): từ một bài toán trừu tượng về ước lượng tần suất trên luồng dữ liệu — giải bằng Count-Min Sketch — phát triển thành một hệ thống hoàn chỉnh từ thu thập ở tầng nhân tới quản lý sự cố và phản hồi trên cụm thật.

Bố cục đề án. Đề án gồm năm chương:

- **Chương 1 — Tổng quan:** bối cảnh, khảo sát nhu cầu thực tế, mô hình mối đe dọa, phân tích yêu cầu và khảo sát công cụ liên quan.

- **Chương 2 — Cơ sở lý thuyết và công nghệ:** eBPF/CO-RE, Count-Min Sketch, Tarjan, BFS, LPM-Trie và nền tảng công nghệ sử dụng.
- **Chương 3 — Phân tích và thiết kế hệ thống:** mô hình mối đe dọa chi tiết, kiến trúc phần mềm, thiết kế cơ sở dữ liệu và thiết kế giao diện người dùng.
- **Chương 4 — Cài đặt, triển khai và thử nghiệm hệ thống:** hạ tầng nền tảng, các thành phần lõi (collector/pipeline/SOC), ứng dụng và giao diện vận hành, cùng việc chạy thử nghiệm end-to-end trên cluster thật.
- **Chương 5 — Đánh giá và kết luận:** phương pháp và kết quả đánh giá định lượng, so sánh, giới hạn và hướng phát triển.

MỤC LỤC

LỜI CẢM ƠN	i
LỜI CAM ĐOAN	ii
MỞ ĐẦU	iii
BẢNG DIỄN GIẢI TỪ VIẾT TẮT VÀ TIẾNG ANH	ix
BẢNG DIỄN GIẢI KÝ HIỆU	xii
Chương 1. TỔNG QUAN	1
1.1 Bối cảnh kiến trúc: từ một khối đến vi dịch vụ trên Kubernetes	1
1.1.1 Kiến trúc một khối và mở rộng theo chiều dọc	1
1.1.2 Kiến trúc vi dịch vụ trên Kubernetes và mở rộng theo chiều ngang	2
1.1.3 Quét cổng và dịch chuyển ngang: định nghĩa và mô hình phát hiện	3
1.1.4 Hệ quả an ninh và lý do quan sát ở tầng nhân	4
1.2 Khảo sát nhu cầu thực tế tại doanh nghiệp	5
1.2.1 Môi trường và quy mô khảo sát	5
1.2.2 Phương pháp khảo sát và thu thập yêu cầu	6
1.2.3 Hiện trạng giám sát và các điểm đau	7
1.2.4 Tổng hợp nhu cầu	8
1.3 Mô hình mối đe dọa: chuỗi tấn công nội cụm	8
1.3.1 Kịch bản minh họa trong bối cảnh doanh nghiệp	9
1.4 Khảo sát hiện trạng và khoảng trống nghiên cứu	10
1.4.1 Bằng chứng từ ba bài tổng quan có hệ thống gần đây	10
1.4.2 Đối chiếu với báo cáo công nghiệp gần đây	11
1.4.3 Khoảng trống cụ thể mà đồ án nhắm tới	11
1.5 Mục tiêu, đối tượng và phạm vi	12
1.5.1 Mục tiêu nghiên cứu	12
1.5.2 Đối tượng và phạm vi	12
1.6 Khảo sát giải pháp và công cụ liên quan	13
1.6.1 An ninh truyền thống so với an ninh cloud-native	13
1.6.2 Các công cụ an ninh cloud-native tiêu biểu	14
1.7 Phân tích yêu cầu, đánh giá khả thi và đóng góp	14

1.7.1	Yêu cầu chức năng	15
1.7.2	Yêu cầu phi chức năng	15
1.7.3	Đánh giá tính khả thi	16
1.7.4	Đóng góp của đề án	16
1.8	Phương pháp nghiên cứu	17
1.9	Kết luận chương	17
Chương 2.	CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ	19
2.1	Thu thập sự kiện ở tầng nhân bằng eBPF	19
2.1.1	eBPF và mô hình thực thi an toàn trong nhân Linux	19
2.1.2	Lựa chọn điểm móc và tính khả chuyển nhờ CO-RE	21
2.2	Ước lượng tần suất trên luồng: Count-Min Sketch	23
2.3	Phân tích đồ thị và phân loại địa chỉ trong mạng nội cụm	24
2.3.1	Tarjan — phát hiện thành phần liên thông mạnh	25
2.3.2	BFS — bán kính ảnh hưởng theo k bước	26
2.3.3	LPM Trie — phân loại địa chỉ theo tiền tố dài nhất	27
2.4	Nền tảng công nghệ sử dụng	29
2.4.1	Kubernetes — nền điều phối container	30
2.4.2	CNI Calico — mạng pod và thực thi NetworkPolicy	31
2.4.3	containerd — runtime chạy container	32
2.4.4	NATS JetStream — trực thông điệp xuất bản/đăng ký	32
2.4.5	PostgreSQL — cơ sở dữ liệu quan hệ	33
2.4.6	Prometheus — thu thập số liệu	33
2.4.7	Grafana — trực quan hóa và bảng điều khiển	34
2.4.8	Docker — đóng gói ảnh container	34
2.4.9	Tekton — tích hợp liên tục gốc Kubernetes	35
2.4.10	Harbor — kho ảnh container riêng	35
2.4.11	ArgoCD — triển khai theo mô hình GitOps	36
2.4.12	Kiến trúc nền tảng tổng thể và khác biệt với cách tiếp cận truyền thống	36
2.5	Kết luận chương	37
Chương 3.	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	39
3.1	Mô hình mối đe dọa và yêu cầu hệ thống	39
3.1.1	Đặc tả use-case	41
3.2	Kiến trúc phần mềm và luồng dữ liệu	44
3.2.1	Luồng gọi giữa các dịch vụ	47
3.2.2	Thiết kế chi tiết từng thành phần	49
3.2.3	Đặc tả phần mềm theo mô-đun	50

3.3	Thiết kế cơ sở dữ liệu	51
3.4	Thiết kế quy trình phản hồi và giao diện vận hành	52
3.5	Kết luận chương	55
Chương 4.	CÀI ĐẶT, TRIỂN KHAI VÀ THỬ NGHIỆM HỆ THỐNG	56
4.1	Tổng quan hiện thực	56
4.2	Môi trường và công cụ triển khai	57
4.3	Hạ tầng nền tảng và triển khai (giới thiệu)	58
4.4	Bộ thu thập eBPF	60
4.4.1	Chương trình tầng nhân	60
4.4.2	Chương trình tầng người dùng	62
4.4.3	Đóng gói và triển khai	63
4.4.4	Kiểm chứng eBPF hoạt động ở tầng nhân	64
4.5	Bộ máy pipeline	66
4.5.1	Cấu hình theo dữ liệu	66
4.5.2	Vòng lặp xử lý sự kiện	67
4.5.3	Phát cảnh báo	68
4.6	Trung tâm vận hành an ninh (SOC)	69
4.6.1	Bộ tổng hợp (aggregator)	69
4.6.2	Dịch vụ sự cố (incident-service)	70
4.6.3	Console web	73
4.7	Ứng dụng và giao diện vận hành (chạy thật)	73
4.8	Chạy thử nghiệm hệ thống end-to-end	78
4.9	Kết luận chương	80
Chương 5.	ĐÁNH GIÁ VÀ KẾT LUẬN	81
5.1	Phương pháp đánh giá	81
5.1.1	Môi trường thực nghiệm	81
5.1.2	Bộ công cụ đo đạc	81
5.1.3	Đặc tả bộ phát hiện đo được	81
5.1.4	Nguyên tắc tránh nhiễu thí nghiệm	82
5.2	Kết quả phát hiện và tỷ lệ báo động giả	82
5.3	Đánh giá hiệu năng	83
5.3.1	Chi phí CPU nhân của tầng eBPF	83
5.3.2	Độ trễ vòng phát hiện → phản hồi	85
5.4	So sánh với các công cụ liên quan	86
5.5	Mô hình tin cậy và giới hạn an ninh của bộ thu thập	86
5.6	Giới hạn và hướng phát triển	87
5.6.1	Khó khăn triển khai và bài học rút ra	88

5.7	Kết luận	89
TÀI LIỆU THAM KHẢO		90
PHỤ LỤC		92
Chương A.	Mã nguồn lỗi: Count-Min Sketch	PL-1
Chương B.	Mã nguồn lỗi: Tarjan SCC và BFS bán kính ảnh hưởng	PL-3
B.1	Tarjan SCC (hiện thực lập)	PL-3
B.2	BFS bán kính ảnh hưởng (k-hop)	PL-3
Chương C.	Mã nguồn lỗi: chương trình eBPF nhân và bộ đọc tầng người dùng	PL-5
C.1	Khai báo BPF map và kprobe lúc vào (tcp_connect.bpf.c)	PL-5
C.2	Trình tự nạp và gắn ở tầng người dùng (collector.cpp)	PL-6
Chương D.	Mã nguồn lỗi: Console web (Flask + HTMX)	PL-8
Chương E.	DDL và truy vấn mẫu của PostgreSQL	PL-10
Chương F.	Kịch bản demo và minh chứng vận hành	PL-12
F.1	Chuẩn bị và xác nhận hiện trạng cụm	PL-12
F.2	Minh chứng eBPF can thiệp ở tầng nhân	PL-12
F.3	Mô phỏng tấn công: quét cổng và lan truyền ngang	PL-13
F.4	Vòng phát hiện – phản hồi – cách ly – xác minh	PL-13
F.5	Truy cập kho đối tượng MinIO	PL-14
F.6	Làm cho dashboard quan trắc hiển thị dữ liệu	15

BẢNG DIỄN GIẢI TỪ VIẾT TẮT VÀ TIẾNG ANH

Tên viết tắt	Tên (English)	Diễn giải (Tiếng Việt)
BFS	Breadth-First Search	Tìm kiếm theo chiều rộng
BPMN	Business Process Model and Notation	Ký pháp mô hình hóa quy trình nghiệp vụ
BFD	Business Function Diagram	Sơ đồ phân rã chức năng nghiệp vụ
BTF	BPF Type Format	Định dạng thông tin kiểu dữ liệu của kernel, nền tảng cho CO-RE
C4	C4 model	Mô hình kiến trúc phần mềm bốn mức (Context/Container/-Component/Code)
CI	Confidence Interval	Khoảng tin cậy (thống kê), ở đây dùng mức 95%
CMS	Count-Min Sketch	Cấu trúc dữ liệu "sketch" ước lượng tần suất phần tử trong luồng với sai số kiểm soát được
CNI	Container Network Interface	Chuẩn cắm mạng cho container/Kubernetes (vd Calico, Cilium)
CO-RE	Compile Once – Run Everywhere	Kỹ thuật biên dịch eBPF một lần chạy trên nhiều phiên bản kernel nhờ BTF
CSDL	Database	Cơ sở dữ liệu
DFD	Data Flow Diagram	Sơ đồ luồng dữ liệu
eBPF	extended Berkeley Packet Filter	Kỹ thuật chạy mã trong nhân Linux để quan sát/xử lý sự kiện một cách an toàn
ERD	Entity–Relationship Diagram	Sơ đồ thực thể – quan hệ (thiết kế CSDL)
FPR	False Positive Rate	Tỷ lệ báo động giả (lưu lượng hợp lệ bị gán cờ tấn công)
FR	Functional Requirement	Yêu cầu chức năng

GitOps	GitOps	Mô hình triển khai khai báo, đồng bộ trạng thái cụm từ Git
IP	Internet Protocol	Giao thức Internet dùng để định danh và chuyển gói tin
K8s	Kubernetes	Hệ thống điều phối container (deployment, scaling, scheduling)
kprobe	Kernel probe	Điểm chèn (hook) động vào hàm nhân để chạy chương trình eBPF
LPM	Longest Prefix Match	Khớp tiền tố dài nhất (phân loại địa chỉ theo CIDR)
MTTR	Mean Time To Respond	Thời gian trung bình để phản hồi/xử lý sự cố
NetworkPolicy	Kubernetes NetworkPolicy	Đối tượng K8s khai báo luật cho/chặn lưu lượng pod (do CNI thực thi)
NFR	Non-Functional Requirement	Yêu cầu phi chức năng
NIDS	Network Intrusion Detection System	Hệ phát hiện xâm nhập ở mức mạng
nmap	Network Mapper	Công cụ quét cổng/dịch vụ mạng phổ biến
OCI	Open Container Initiative	Chuẩn ảnh và runtime container mở
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò
RingBuf	Ring Buffer	Bộ đệm vòng của eBPF để chuyển sự kiện lên không gian người dùng
SCC	Strongly Connected Component	Thành phần liên thông mạnh (trong đồ thị có hướng)
SIEM	Security Information and Event Management	Hệ quản lý thông tin và sự kiện an ninh (phân tích nhật ký)
SLR	Systematic Literature Review	Tổng quan tài liệu có hệ thống
SOC	Security Operations Center	Trung tâm vận hành an ninh (giám sát và phản ứng sự cố)

SRE	Site Reliability Engineering	Kỹ nghệ độ tin cậy hệ thống (vận hành)
TCP	Transmission Control Protocol	Giao thức truyền tải hướng kết nối, tin cậy
UC	Use Case	Cả sử dụng (trong phân tích yêu cầu)
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa thống nhất
YAML	YAML Ain't Markup Language	Ngôn ngữ dữ liệu cấu hình dạng phân cấp

BẢNG DIỄN GIẢI KÝ HIỆU

S	Luồng sự kiện kết nối đầu vào của bộ phát hiện
τ	Ngưỡng phát hiện quét cổng (số kết nối/cửa sổ; trong đề án $\tau = 40$)
d, w	Chiều sâu (số hàm băm) và chiều rộng của Count-Min Sketch ($d = 5, w = 2048$)
M	Ma trận đếm của Count-Min Sketch, kích thước $d \times w$
h_i	Hàm băm thứ i của Count-Min Sketch, $i = 1, \dots, d$
$\hat{f}_x$	Tần suất (số kết nối) ước lượng của nguồn x trong cửa sổ
ε, δ	Tham số sai số và xác suất thất bại của ước lượng Count-Min Sketch
F_0	Số nguồn phân biệt xuất hiện trong cửa sổ (lực lượng tập khóa)
$G = (V, E)$	Đồ thị truyền thông có hướng: đỉnh V là địa chỉ IP, cung E là kết nối
s	Đỉnh nguồn (pod nghi vấn) trong phân tích đồ thị
k	Số bước (hop) tối đa của BFS bán kính ảnh hưởng (trong đề án $k = 3$)
$R_k(s)$	Bán kính ảnh hưởng: tập pod tối được từ nguồn s trong không quá k bước
p_{50}, p_{95}	Trung vị và phân vị 95 của độ trễ phản hồi
$O(\cdot)$	Ký hiệu Big-O: cận trên về độ phức tạp thời gian/không gian

DANH SÁCH BẢNG

1.1	Các bên liên quan được phỏng vấn và mối quan tâm chính.	7
1.2	Các điểm đau ghi nhận từ khảo sát hiện trạng.	8
1.3	Tổng hợp nhu cầu chứng cất từ các điểm đau.	8
1.4	Một số số liệu tiêu biểu về an ninh Kubernetes từ các khảo sát gần đây. .	11
1.5	Ranh giới phạm vi của đề án.	13
1.6	So sánh an ninh truyền thống và an ninh cloud-native (tầng nhân). . . .	13
1.7	Đối chiếu một số công cụ liên quan với đề án.	14
1.8	Yêu cầu chức năng của hệ thống.	15
1.9	Yêu cầu phi chức năng của hệ thống.	15
1.10	Giả định, rủi ro và biện pháp giảm thiểu.	16
2.1	So sánh các điểm móc eBPF cho bài toán quan sát kết nối.	21
2.2	Trạng thái ma trận CMS ($d = 2, w = 4$) sau luồng $S = (a, b, a, c, a, b)$, với $h_1: a \rightarrow 1, b \rightarrow 3, c \rightarrow 1$ và $h_2: a \rightarrow 2, b \rightarrow 0, c \rightarrow 3$	24
2.3	Các công cụ/công nghệ nền tảng và công dụng trong hệ thống.	30
3.1	Đặc tả các chức năng lá của sơ đồ phân rã chức năng (BFD).	41
3.2	Danh sách use-case và tác nhân.	42
3.3	Đặc tả use-case UC-02 — Điều tra chi tiết sự cố.	42
3.4	Đặc tả use-case UC-04 — Duyệt và áp hành động phản hồi.	43
3.5	Đặc tả use-case UC-05 — Cấu hình ngưỡng phát hiện và người dùng. . .	43
3.6	Đặc tả use-case UC-01 — Xem dashboard và cảnh báo.	44
3.7	Đặc tả use-case UC-03 — Ghi nhận / đóng sự cố.	44
3.8	Đặc tả phần mềm theo mô-đun (đầu vào / xử lý / đầu ra).	51
4.1	Bản đồ hiện thực: thành phần $\rightarrow$ kho mã, ngôn ngữ, đóng gói và tệp cốt lõi.	56
5.1	Kết quả phát hiện trên cụm thật (node worker-1, nhân 6.8). Hai dòng cuối là minh hoạ giới hạn, không tính vào chỉ số chính.	82
5.2	Chi phí CPU nhân cho mỗi connect() (trung bình $\pm$ CI 95%, $n = 10$ chu kỳ, ≈ 3000 kết nối/chu kỳ).	84
5.3	So sánh định tính với các công cụ an ninh runtime cho Kubernetes. . . .	86

DANH SÁCH HÌNH VẼ

1.1	Kiến trúc một khối (monolith) truyền thống.	2
1.2	Kiến trúc vi dịch vụ trên Kubernetes.	3
1.3	Hai mẫu hành vi tấn công và tín hiệu tương ứng.	4
1.4	Hiện trạng giám sát và vùng mù Đông-Tây tại doanh nghiệp.	7
1.5	Chuỗi tấn công (kill-chain) nội cụm và vùng phát hiện của đồ án.	9
1.6	Quy trình kỹ nghệ yêu cầu và mạch truy vết từ nhu cầu tới các chương sau.	15
2.1	Logo eBPF.	19
2.2	Mô hình thực thi của một chương trình eBPF.	20
2.3	Ba BPF map của bộ thu thập và vai trò tương ứng.	20
2.4	Vị trí gắn kprobe/kretprobe trong ngăn xếp TCP.	21
2.5	Đường đi của một sự kiện kết nối qua eBPF.	22
2.6	Cấu trúc Count-Min Sketch.	23
2.7	Ví dụ thành phần liên thông mạnh (SCC) trong đồ thị truyền thông.	26
2.8	Bán kính ảnh hưởng theo BFS k bước từ nguồn s	27
2.9	Ví dụ phân loại CIDR bằng LPM-Trie.	28
2.10	Lưu đồ xử lý một sự kiện kết nối qua bốn giải thuật cốt lõi.	29
2.11	Logo Kubernetes.	30
2.12	Logo Project Calico.	31
2.13	Logo containerd.	32
2.14	Logo NATS.	32
2.15	Logo PostgreSQL.	33
2.16	Logo Prometheus.	33
2.17	Logo Grafana.	34
2.18	Logo Docker.	34
2.19	Logo Tekton.	35
2.20	Logo Harbor.	35
2.21	Logo Argo (ArgoCD).	36
2.22	Sơ đồ nền tảng tổng thể: các tầng từ nhân Linux tới ứng dụng đặc thù.	37
3.1	Sơ đồ use-case của hệ thống.	40
3.2	Sơ đồ phân rã chức năng (BFD) của nền tảng giám sát an ninh.	40

3.3	Kiến trúc mức container (mô hình C4).	45
3.4	Sơ đồ luồng dữ liệu mức ngữ cảnh (DFD level 0).	45
3.5	Sơ đồ luồng dữ liệu mức 1.	46
3.6	Sơ đồ luồng dữ liệu mức 2 (tiến trình phát hiện).	46
3.7	Luồng xử lý phân tán trên mỗi nút (DaemonSet) và phần xử lý tập trung.	47
3.8	Sơ đồ tuần tự: thu thập sự kiện và phát hiện quét cổng.	48
3.9	Sơ đồ tuần tự: tạo và làm giàu sự cố.	48
3.10	Sơ đồ tuần tự: phê duyệt và áp hành động phản hồi.	49
3.11	Thiết kế thành phần: collector (trái) và pipeline (phải).	49
3.12	Thiết kế thành phần: aggregator (trái) và alert-bridge (phải).	50
3.13	Thiết kế thành phần: incident-service (trái) và web-console (phải).	50
3.14	Sơ đồ thực thể – quan hệ (ERD) của cơ sở dữ liệu SOC.	52
3.15	Quy trình nghiệp vụ phát hiện → phê duyệt → cách ly (BPMN).	53
3.16	Wireframe màn hình Incidents (bảng hành động chờ duyệt + bảng sự cố).	54
3.17	Wireframe màn hình chi tiết sự cố.	54
3.18	Wireframe màn hình Reports (báo cáo tuần + xuất nhật ký kiểm toán).	55
4.1	Sơ đồ tổng quát kiến trúc hệ thống.	57
4.2	Sơ đồ triển khai trên cụm Kubernetes ba node.	60
4.3	Đường đi của một sự kiện kết nối qua eBPF.	62
4.4	Minh chứng eBPF ở tầng nhân qua bpftool trên node.	66
4.5	Lưu đồ xử lý một sự kiện trong pipeline.	68
4.6	Lưu đồ sinh và áp NetworkPolicy cách ly.	72
4.7	Hàng đợi phê duyệt hành động trên console.	74
4.8	Chi tiết một sự cố với ngữ cảnh đã làm giàu và nhật ký kiểm toán.	75
4.9	Hành động cách ly đã được áp (executed) sau phê duyệt.	76
4.10	Trang báo cáo tuần kết xuất từ cơ sở dữ liệu.	77
4.11	Dashboard <i>ZT-SOC Overview</i> trên Grafana.	78
4.12	Dashboard <i>ZT-SOC Incidents</i> trên Grafana.	78
4.13	Lưu đồ kịch bản thử nghiệm end-to-end.	79
5.1	Hiệu chỉnh ngưỡng phát hiện (số liệu đo thật).	83
5.2	Chi phí CPU nhân cho mỗi connect().	84
5.3	Chi phí CPU nhân theo mức tải kết nối (suy diễn).	85
5.4	Độ trễ máy của vòng phản hồi.	85

CHƯƠNG 1

TỔNG QUAN

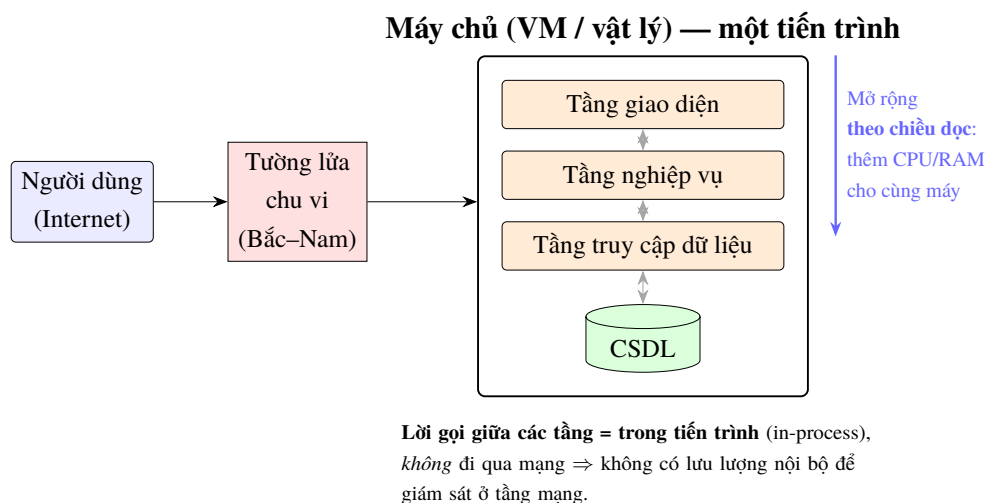
1.1. Bối cảnh kiến trúc: từ một khối đến vi dịch vụ trên Kubernetes

Phần này làm rõ *vì sao* bài toán giám sát an ninh trong đồ án lại đặt ra trên môi trường Kubernetes chứ không phải trên hạ tầng truyền thống, và *vì sao* cách quan sát phải đặt ở tầng nhân. Lý do nằm ở sự dịch chuyển kiến trúc phần mềm trong một thập kỷ qua: từ kiến trúc *một khối* (*monolithic*) chạy trên máy chủ vật lý, sang kiến trúc *vi dịch vụ* (*microservices*) đóng gói trong container và điều phối bởi Kubernetes. Sự dịch chuyển này không chỉ thay đổi cách phần mềm được xây dựng và mở rộng mà còn làm thay đổi căn bản *bề mặt tấn công*, khiến các biện pháp an ninh truyền thống trở nên kém hiệu quả — đó chính là khoảng trống mà đồ án hướng tới.

1.1.1. Kiến trúc một khối và mở rộng theo chiều dọc

Trong kiến trúc một khối, toàn bộ chức năng của ứng dụng — từ tầng giao diện, tầng nghiệp vụ đến tầng truy cập dữ liệu — được biên dịch và triển khai như *một tiến trình duy nhất* chạy trên một máy chủ (vật lý hoặc máy ảo) như minh họa ở Hình 1.1. Đặc điểm quan trọng nhất xét từ góc độ an ninh là: các lời gọi giữa các tầng diễn ra *trong cùng một tiến trình* (in-process), tức không đi qua mạng. Hệ quả là gần như toàn bộ luồng dữ liệu nội bộ của ứng dụng nằm gọn bên trong một máy chủ và không hề xuất hiện trên dây mạng để có thể bị quan sát. Khi cần phục vụ nhiều người dùng hơn, mô hình này mở rộng *theo chiều dọc* (*vertical scaling*): nâng cấp phần cứng (thêm CPU, RAM) cho chính máy chủ đó. Cách mở rộng này đơn giản nhưng có trần vật lý và tạo ra một điểm hỏng tập trung.

Trong mô hình này, an ninh được tổ chức theo nguyên lý *phòng thủ chu vi* (*perimeter security*): một tường lửa hoặc hệ thống phát hiện xâm nhập đặt ở *biên mạng*, kiểm soát lưu lượng ra vào giữa Internet và máy chủ — thường gọi là lưu lượng *Bắc–Nam*. Vì luồng nội bộ là in-process, mô hình phòng thủ chu vi này là *đủ*: kẻ tấn công muốn tới được dữ liệu phải vượt qua biên, và biên là nơi duy nhất cần canh gác.

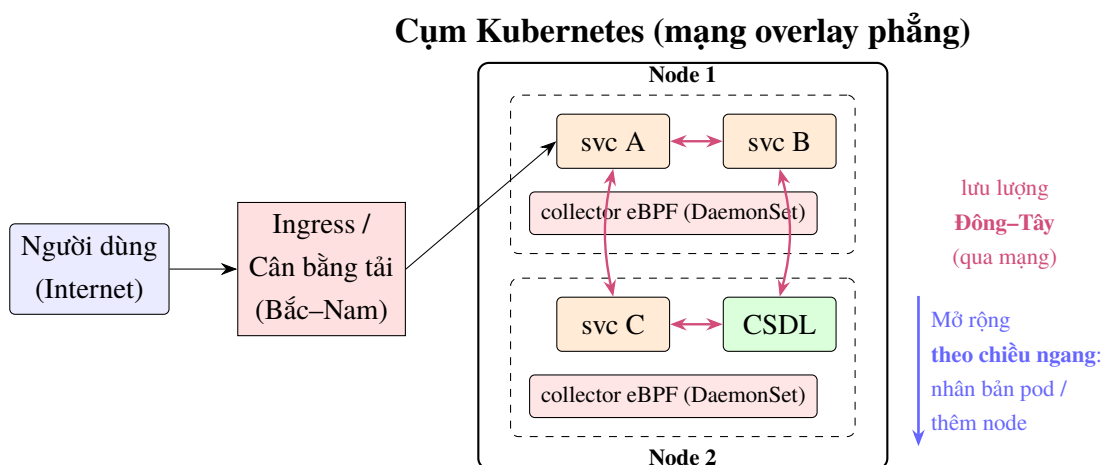


Hình 1.1. Kiến trúc một khối (monolith) truyền thống.

1.1.2. Kiến trúc vi dịch vụ trên Kubernetes và mở rộng theo chiều ngang

Kiến trúc vi dịch vụ phân rã ứng dụng thành nhiều dịch vụ nhỏ, độc lập, mỗi dịch vụ đóng gói trong một container và được điều phối bởi Kubernetes [3]. Như Hình 1.2 thể hiện, các dịch vụ này chạy phân tán trên nhiều nút (*node*) của cụm và giao tiếp với nhau *qua mạng* thông qua một mạng phủ (*overlay network*) phẳng. Mô hình mở rộng *theo chiều ngang* (*horizontal scaling*): thay vì nâng cấp một máy, hệ thống nhân bản thêm các bản sao pod hoặc thêm nút vào cụm. Điều này mang lại độ co giãn và khả năng chịu lỗi cao hơn hẳn, là lý do kiến trúc này trở thành tiêu chuẩn thực tế cho các hệ thống quy mô lớn hiện đại.

Tuy nhiên, chính sự phân tán này sinh ra một loại lưu lượng mới mà mô hình một khối không có: lưu lượng *Đông-Tây* — các lời gọi dịch vụ-tới-dịch vụ (pod-tới-pod) đi *qua mạng* bên trong cụm. Trong một hệ thống vi dịch vụ thực tế, lưu lượng Đông-Tây thường lớn hơn nhiều lần lưu lượng Bắc-Nam. Mỗi pod giờ đây vừa là một điểm có thể bị khai thác, vừa là một bàn đạp để kẻ tấn công *dịch chuyển ngang* (*lateral movement*) sang các pod khác. Vòng đời pod ngắn và địa chỉ IP thay đổi liên tục càng làm cho việc theo dõi trở nên khó khăn.



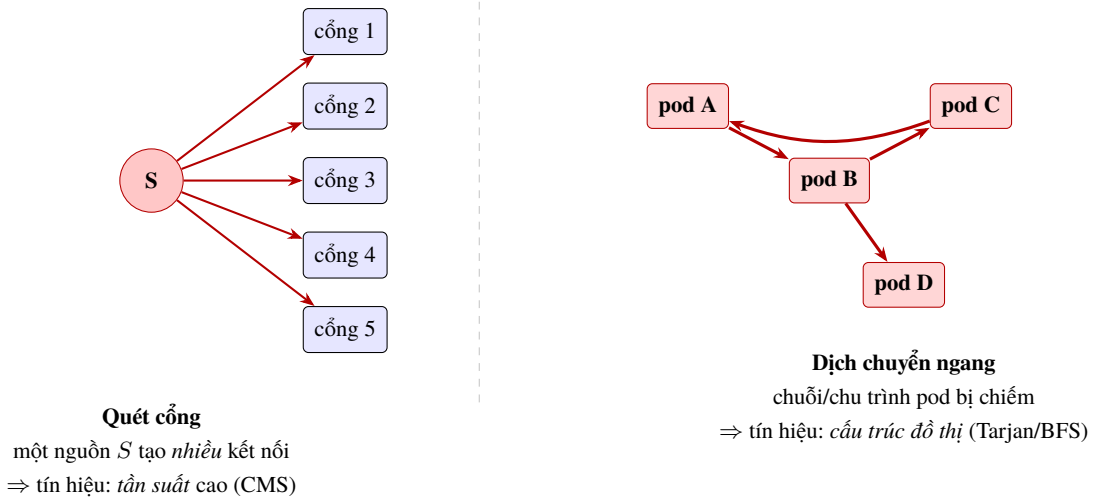
Hình 1.2. Kiến trúc vi dịch vụ trên Kubernetes.

1.1.3. Quét cổng và dịch chuyển ngang: định nghĩa và mô hình phát hiện

Trên bề mặt Đông-Tây vừa mô tả, hai hành vi tấn công là trọng tâm của đề án. Mục này định nghĩa chúng một cách chính xác và — quan trọng hơn — chỉ ra *tín hiệu* mà mỗi hành vi để lại, vì chính các tín hiệu này quyết định giải thuật phát hiện được dùng ở các mục sau.

Quét cổng (*port scanning*). Là hành vi một nguồn lần lượt thử kết nối tới nhiều cổng (quét *dọc* trên một đích) hoặc nhiều đích (quét *ngang* trên một cổng) nhằm lập bản đồ các dịch vụ đang mở. Trong cụm Kubernetes, đây thường là bước *trình sát* đầu tiên sau khi kẻ tấn công kiểm soát được một pod: từ pod đó, chúng dò quét các pod và dịch vụ lân cận để tìm điểm khai thác tiếp theo. Dấu hiệu đặc trưng của quét cổng, nhìn từ tầng nhân, là *một nguồn tạo ra số lượng kết nối tăng vọt bất thường* trong một khoảng thời gian ngắn (Hình 1.3, trái). Tín hiệu này có bản chất *tần suất*, và đó chính là lý do hệ thống dùng cấu trúc ước lượng tần suất Count-Min Sketch (Mục 2.2) để bắt nó với chi phí bộ nhớ thấp.

Dịch chuyển ngang (*lateral movement*). Là giai đoạn kẻ tấn công, sau khi chiếm một pod, lần lượt mở rộng quyền kiểm soát sang các pod khác để tiến gần tới mục tiêu giá trị cao (ví dụ cơ sở dữ liệu). Khác với quét cổng vốn lộ ra ở *tần suất*, dịch chuyển ngang lộ ra ở *cấu trúc* của lưu lượng: xuất hiện những chuỗi hoặc chu trình liên lạc pod-tới-pod bất thường (Hình 1.3, phải). Một nhóm pod liên lạc vòng tròn với nhau là một dấu hiệu đáng ngờ, và phạm vi mà một pod bị chiếm có thể với tới trong vài bước cho biết mức độ nghiêm trọng của sự cố.



Hình 1.3. Hai mẫu hành vi tấn công và tín hiệu tương ứng.

Mô hình phát hiện thống nhất. Hai tín hiệu trên được nắm bắt trên cùng một biểu diễn: mô hình hóa lưu lượng nội cụm thành một *đồ thị truyền thông* có hướng $G = (V, E)$, trong đó mỗi đỉnh là một địa chỉ (pod, dịch vụ, nút) và mỗi cung là một kết nối đã quan sát được. Trên biểu diễn này, bài toán phát hiện tách thành hai nhánh bổ trợ: (i) *nhánh tần suất* — ước lượng số kết nối phát ra từ mỗi đỉnh nguồn để phát hiện quét cổng, giải bằng Count-Min Sketch; và (ii) *nhánh cấu trúc* — phân tích quan hệ giữa các đỉnh để phát hiện dịch chuyển ngang, giải bằng giải thuật Tarjan (tìm thành phần liên thông mạnh) và duyệt BFS theo k bước (ước lượng bán kính ảnh hưởng), trình bày ở Mục 2.3. Việc tách bài toán theo hai loại tín hiệu như vậy là kim chỉ nam cho phần cơ sở giải thuật ở Chương 2.

1.1.4. Hệ quả an ninh và lý do quan sát ở tầng nhân

Đặt hai kiến trúc cạnh nhau, có thể thấy ngay vì sao phòng thủ chu vi truyền thống không còn đủ. Tường lửa biên mạng chỉ thấy lưu lượng Bắc–Nam và gần như mù trước toàn bộ lưu lượng Đông–Tây nội cụm — đúng nơi mà hành vi quét cổng và dịch chuyển ngang diễn ra. Khi kẻ tấn công đã chiếm được một pod, chúng có thể tự do dò quét và lan sang các pod khác mà không một lần nào phải đi qua biên. Đây là lý do mô hình an ninh hiện đại chuyển sang nguyên lý *Zero-Trust* [15]: không mặc nhiên tin tưởng lưu lượng nội bộ, phải quan sát và kiểm soát *mọi* kết nối kể cả bên trong vành đai. Các báo cáo công nghiệp gần đây xác nhận rủi ro này: cấu hình sai và thiếu khả năng quan sát nội cụm là nhóm nguyên nhân hàng đầu của sự cố an ninh Kubernetes [13, 18, 12].

Vấn đề kế tiếp là *quan sát ở đâu*. Có ba lựa chọn quen thuộc, mỗi lựa chọn một nhược điểm trong môi trường vi dịch vụ: (i) cài tác nhân giám sát vào *từng ứng dụng* — xâm lấn, phụ thuộc ngôn ngữ lập trình và phải sửa từng dịch vụ; (ii) chèn một *sidecar proxy* cạnh mỗi pod — tốn tài nguyên và chỉ thấy lưu lượng tầng ứng dụng đã được định tuyến qua nó; (iii) phản chiếu (*mirror*) toàn bộ gói tin trên mạng — chi phí lớn và khó

truy ngữ cảnh tiến trình. Cả ba đều không lý tưởng. Trong khi đó, có một điểm quan sát *tự nhiên* mà mọi kết nối đều phải đi qua: *nhân của hệ điều hành* trên mỗi nút. Vì mọi kết nối TCP của mọi pod đều được xử lý bởi nhân của nút chứa pod đó, một chương trình đặt tại tầng nhân sẽ quan sát được trọn vẹn lưu lượng Đông–Tây mà không cần sửa ứng dụng, không phụ thuộc ngôn ngữ và chỉ cần hiện diện một lần trên mỗi nút. Công nghệ cho phép làm điều đó một cách an toàn và hiệu năng cao chính là **eBPF**, được trình bày ở Chương 2. Như vậy, lựa chọn quan sát tầng nhân không phải ngẫu nhiên mà là hệ quả trực tiếp của đặc thù kiến trúc vi dịch vụ trên Kubernetes.

1.2. Khảo sát nhu cầu thực tế tại doanh nghiệp

Đề tài không xuất phát từ một giả định lý thuyết thuần túy mà bắt nguồn từ một nhu cầu vận hành có thật, được ghi nhận qua quá trình khảo sát tại một doanh nghiệp đang vận hành nền tảng Kubernetes nội bộ. Mục này trình bày môi trường khảo sát, phương pháp thu thập yêu cầu, hiện trạng giám sát an ninh và các điểm đau, từ đó tổng hợp thành tập nhu cầu làm đầu vào cho phần phân tích yêu cầu (Mục 1.7). Việc hiểu rõ bài toán thực tế là cơ sở để bảo đảm hệ thống được thiết kế đúng nhu cầu thay vì xuất phát từ giả định.

Về phương pháp luận, đề án tuân theo quy trình *kỹ nghệ yêu cầu* (*requirements engineering*) gồm bốn bước nối tiếp: (i) *khơi gợi* (*elicitation*) — thu thập thông tin từ các bên liên quan và hiện trạng; (ii) *phân tích* (*analysis*) — chưng cất thông tin thô thành nhu cầu và xác định mâu thuẫn, ưu tiên; (iii) *đặc tả* (*specification*) — phát biểu thành yêu cầu chức năng và phi chức năng có thể truy vết; và (iv) *thẩm định* (*validation*) — đối chiếu yêu cầu với khảo sát học thuật, công nghiệp và đánh giá khả thi. Chương này thực hiện trọn bốn bước đó: hai bước đầu nằm ở Mục 1.2 và Mục 1.3, bước đặc tả ở Mục 1.7, và bước thẩm định trải trên các mục khảo sát và đánh giá khả thi. Kết quả — tập yêu cầu — sau đó được *phân tích chi tiết thành thiết kế* ở Chương 3, đúng theo sự phân vai giữa “xác định cần gì” (chương này) và “thiết kế làm thế nào” (Chương 3).

1.2.1. Môi trường và quy mô khảo sát

Doanh nghiệp được khảo sát vận hành một cụm Kubernetes triển khai trên hạ tầng nội bộ (*on-premise*), không dùng dịch vụ Kubernetes do nhà cung cấp đám mây quản lý (như EKS, GKE hay AKS). Quy mô cụm ở mức lớn: hơn **một nghìn** đối tượng Deployment thuộc nhiều nhóm sản phẩm, tương ứng hàng nghìn pod hoạt động đồng thời, phân bố trên nhiều không gian tên (*namespace*) theo từng đội phát triển. Mạng pod do CNI *Calico* đảm nhiệm — một lựa chọn cho phép thực thi NetworkPolicy ở tầng mạng — song trên thực tế phần lớn không gian tên vẫn áp dụng chính sách mặc định cho-phép-rộng, bởi đội vận hành *thiếu thông tin về thực tế giao tiếp giữa các dịch vụ* để có thể siết chặt chính sách một cách an toàn.

Về tổ chức, cụm được chia theo nhiều không gian tên ứng với từng đội sản phẩm

và từng môi trường (phát triển, kiểm thử, sản xuất). Các dịch vụ rất đa dạng về công nghệ — nhiều ngôn ngữ và khung lập trình cùng tồn tại — và giao tiếp với nhau dày đặc theo mô hình đông–đông: một yêu cầu của người dùng có thể kích hoạt một chuỗi nhiều lời gọi nội bộ giữa các dịch vụ. Vòng đời pod ngắn (pod được tạo và hủy liên tục theo nhịp triển khai và co giãn), kéo theo địa chỉ IP của pod thay đổi thường xuyên, khiến mọi cơ chế giám sát gắn cứng theo địa chỉ đều nhanh chóng lỗi thời.

Về an ninh và quan trắc, doanh nghiệp đang dùng *Wazuh* làm nền tảng giám sát chính. *Wazuh* thu thập nhật ký, kiểm tra toàn vẹn tệp và giám sát sự kiện ở mức *máy chủ* (host) trên các nút, kết hợp với một số công cụ quản lý nhật ký tập trung và giám sát số liệu (Prometheus/Grafana). Ở biên hệ thống có tường lửa và thiết bị kiểm soát lưu lượng Bắc–Nam. Cách bố trí này hiệu quả với các mối đe dọa đi qua biên, nhưng có một giả định ngầm không còn đúng trong môi trường vi dịch vụ: rằng quan sát ở tầng máy chủ và biên mạng là đủ. Điểm mấu chốt là toàn bộ lớp giám sát hiện có dừng ở *ranh giới máy chủ và biên mạng* — nó hầu như không nhìn thấy lưu lượng mạng *giữa các pod* bên trong cụm, vốn là nơi diễn ra phần lớn hoạt động của một hệ vi dịch vụ quy mô lớn. Với hơn một nghìn deployment, số lượng cặp dịch vụ có thể giao tiếp là rất lớn, và việc không có bản đồ giao tiếp khiến cả đội an ninh lẫn đội nền tảng phải làm việc trong điều kiện thiếu thông tin căn bản.

1.2.2. Phương pháp khảo sát và thu thập yêu cầu

Quá trình thu thập yêu cầu được thực hiện theo cách tiếp cận quen thuộc của kỹ nghệ phần mềm, kết hợp ba nguồn dữ liệu để giảm thiên lệch. Thứ nhất là *phỏng vấn bán cấu trúc* (semi-structured interview) các bên liên quan; danh sách bên liên quan và mối quan tâm chính được tóm tắt ở Bảng 1.1. Thứ hai là *quan sát hiện trạng*: rà soát cấu hình các công cụ giám sát đang dùng, các NetworkPolicy hiện hành và bản đồ namespace của cụm. Thứ ba là *phân tích sự cố quá khứ* để hiểu những tình huống mà khả năng phát hiện hiện tại đã bỏ sót. Các câu hỏi phỏng vấn xoay quanh bốn trục: khả năng nhìn thấy lưu lượng nội cụm, khả năng phát hiện hành vi bất thường, quy trình ứng phó sự cố, và các ràng buộc về tài nguyên và vận hành. Cụ thể, một số câu hỏi tiêu biểu được đặt ra gồm: “Khi một dịch vụ bị nghi ngờ tổn hại, đội có thể biết nó đã kết nối tới những dịch vụ nào không?”; “Hiện có cảnh báo nào khi một pod đột ngột tạo nhiều kết nối tới nhiều đích khác nhau không?”; “Quy trình cách ly một pod khi xảy ra sự cố hiện diễn ra thế nào và mất bao lâu?”; và “Ràng buộc nào khiến việc cài thêm tác nhân giám sát vào ứng dụng trở nên khó khăn?”.

Tổng hợp câu trả lời cho thấy một bức tranh nhất quán giữa các vai trò. Đội an ninh xác nhận họ *không* trả lời được câu hỏi về quan hệ kết nối nội cụm và *không* có cảnh báo cho hành vi dò quét bên trong; đội nền tảng nhấn mạnh rằng cài tác nhân vào hơn một nghìn deployment đa ngôn ngữ là không khả thi về công sức bảo trì; đội vận hành lo

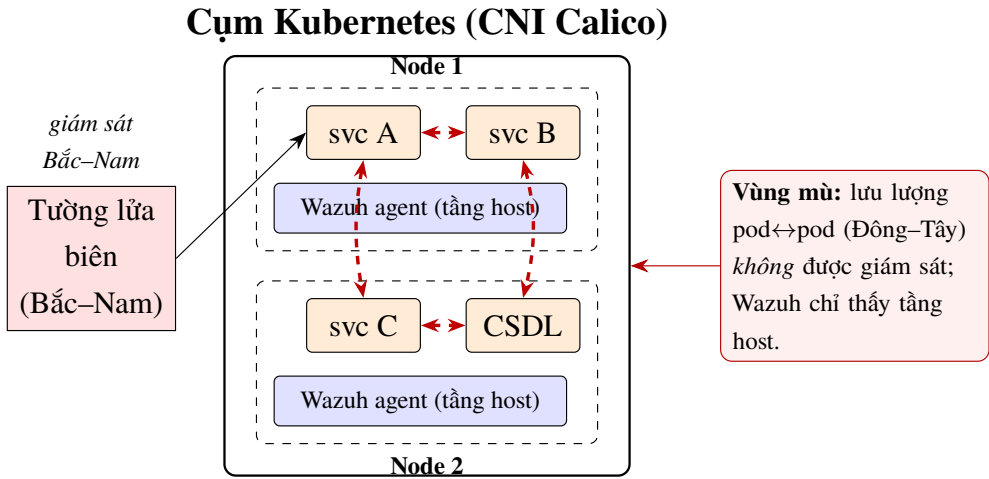
ngại mọi cơ chế cách ly tự động vì rủi ro chặn nhầm dịch vụ hợp lệ ở quy mô lớn; còn lập trình viên mong muốn giải pháp “trong suốt” với mã ứng dụng. Những phát hiện này về sau định hình trực tiếp các yêu cầu phi chức năng (không xâm lấn, an toàn khi phản hồi) ở Mục 1.7.

Bảng 1.1. Các bên liên quan được phỏng vấn và mối quan tâm chính.

Vai trò	Viết tắt	Mối quan tâm chính
Kỹ sư an ninh	SecOps	Phát hiện tấn công nội cụm, ứng phó sự cố, bằng chứng kiểm toán
Kỹ sư nền tảng	DevOps	Chi phí tài nguyên, độ phức tạp vận hành, khả năng tái lập
Kỹ sư vận hành	SRE	Tác động hiệu năng, độ ổn định, tránh báo động giả gây gián đoạn
Lập trình viên	Dev	Không phải sửa mã ứng dụng, không phụ thuộc ngôn ngữ

1.2.3. Hiện trạng giám sát và các điểm đau

Tổng hợp từ ba nguồn khảo sát, hiện trạng giám sát an ninh nội cụm bộc lộ một số điểm đau (*pain point*) nổi bật, được liệt kê ở Bảng 1.2. Điểm đau bao trùm và nghiêm trọng nhất là *mù lưu lượng Đông–Tây*: vì Wazuh và các công cụ hiện có chỉ quan sát ở tầng máy chủ, đội an ninh không trả lời được câu hỏi cơ bản “dịch vụ nào đang nói chuyện với dịch vụ nào”. Hệ quả dây chuyền là không thể phát hiện hành vi quét cổng hay dịch chuyển ngang khi kẻ tấn công đã ở *bên trong* cụm, cũng không thể ước lượng bán kính ảnh hưởng khi một pod bị nghi ngờ chiếm quyền. Đáng chú ý, đây vừa là bài toán an ninh vừa là bài toán vận hành: chính sự thiếu vắng bản đồ giao tiếp khiến đội nền tảng không dám siết NetworkPolicy của Calico, tạo thành một vòng luẩn quẩn giữa khả năng quan sát và khả năng kiểm soát. Hình 1.4 minh họa hiện trạng này: lớp giám sát hiện có bao phủ được trục Bắc–Nam, nhưng toàn bộ lưu lượng Đông–Tây giữa các pod nằm trong một *vùng mù*.



Hình 1.4. Hiện trạng giám sát và vùng mù Đông–Tây tại doanh nghiệp.

Bảng 1.2. Các điểm đau ghi nhận từ khảo sát hiện trạng.

Mã	Điểm đau	Mô tả
P1	Mù lưu lượng Đông–Tây	Không thấy giao tiếp pod-tới-pod; chỉ giám sát ở tầng host (Wazuh) và biên mạng
P2	Thiếu bản đồ phụ thuộc	Không biết dịch vụ nào gọi dịch vụ nào nên không siết được NetworkPolicy
P3	Tác nhân theo pod bất khả thi	Cài agent vào hơn 1000 deployment là tốn kém, phụ thuộc ngôn ngữ, khó bảo trì
P4	Ứng phó thủ công, chậm	Khi nghi ngờ sự cố, việc cách ly pod làm thủ công, không có quy trình tự động có kiểm soát
P5	Thiếu bằng chứng kiểm toán	Không có dấu vết “ai kết nối tới ai” trong cụm phục vụ điều tra và tuân thủ

1.2.4. Tổng hợp nhu cầu

Từ các điểm đau, quá trình khảo sát chưng cất ra một tập *nhu cầu* (Bảng 1.3) — là phát biểu “cần gì” ở mức nghiệp vụ, độc lập với giải pháp kỹ thuật. Tập nhu cầu này định hình *miền bài toán* (domain) thực sự mà đề án phải giải quyết, và sẽ được chuyển hóa thành các yêu cầu chức năng và phi chức năng cụ thể ở Mục 1.7, rồi tiếp tục được phân tích chi tiết thành thiết kế ở Chương 3.

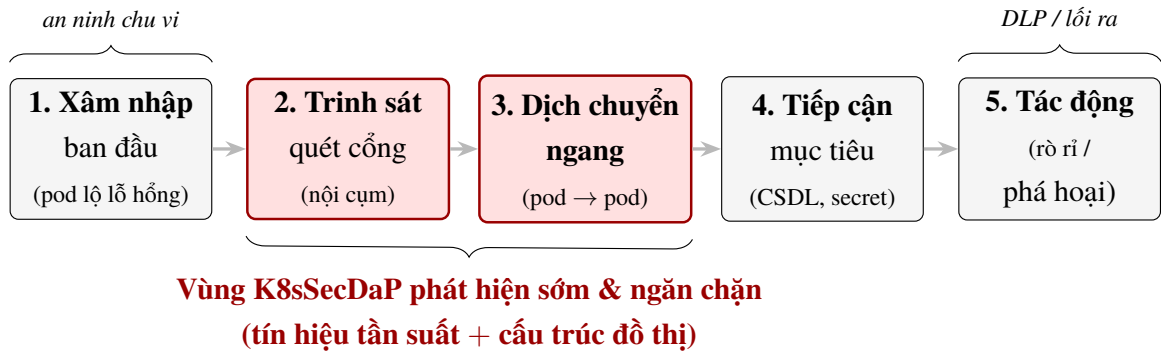
Bảng 1.3. Tổng hợp nhu cầu chưng cất từ các điểm đau.

Mã	Nhu cầu	Đáp ứng điểm đau
N1	Quan sát lưu lượng kết nối toàn cụm với chi phí thấp, <i>không</i> cài tác nhân vào từng pod	P1, P3
N2	Phát hiện trực tuyến hành vi quét cổng và dịch chuyển ngang	P1
N3	Dựng bản đồ giao tiếp giữa các dịch vụ và ước lượng bán kính ảnh hưởng	P2
N4	Ứng phó cách ly tự động <i>có người duyệt</i> , tận dụng NetworkPolicy của Calico sẵn có	P2, P4
N5	Lưu vết kiểm toán đầy đủ phục vụ điều tra và tuân thủ	P5

1.3. Mô hình môi đe dọa: chuỗi tấn công nội cụm

Để hiểu vì sao các nhu cầu trên lại tập trung vào giai đoạn “bên trong” cụm, cần đặt chúng vào một mô hình môi đe dọa. Một cuộc tấn công vào hệ vi dịch vụ thường diễn ra theo một chuỗi giai đoạn (*kill-chain*) như Hình 1.5: sau khi *xâm nhập ban đầu* qua một pod lộ lỗ hổng (giai đoạn 1), kẻ tấn công *trình sát* bằng cách quét cổng các pod và dịch vụ lân cận (giai đoạn 2), rồi *dịch chuyển ngang* từ pod này sang pod khác (giai đoạn 3) để

tiến tới *mục tiêu giá trị cao* như cơ sở dữ liệu hay kho bí mật (giai đoạn 4), và cuối cùng *tác động* bằng việc rò rỉ hay phá hoại dữ liệu (giai đoạn 5).



Hình 1.5. Chuỗi tấn công (kill-chain) nội cụm và vùng phát hiện của đồ án.

Điểm mấu chốt là an ninh chu vi truyền thống chỉ kiểm soát được hai đầu của chuỗi: ngăn xâm nhập ban đầu ở biên (giai đoạn 1) và chặn rò rỉ ra ngoài (giai đoạn 5). Toàn bộ phần *giữa* — trình sát và dịch chuyển ngang — diễn ra hoàn toàn trong lưu lượng Đông–Tây và vô hình trước lớp giám sát hiện tại của doanh nghiệp. Đây chính xác là khoảng mà đồ án nhắm tới: phát hiện *sớm* ở giai đoạn 2–3, khi kẻ tấn công vừa mới bắt đầu dò quét và lan tỏa, để ngăn chặn trước khi chúng chạm tới mục tiêu. Hai giai đoạn này để lại hai loại tín hiệu bổ trợ đã nêu ở Mục 1.1.3 (tần suất kết nối và cấu trúc đồ thị giao tiếp), và đó là nền tảng cho cách tiếp cận phát hiện của hệ thống.

1.3.1. Kịch bản minh họa trong bối cảnh doanh nghiệp

Để cụ thể hóa, hãy xét một kịch bản đặt trong đúng môi trường đã khảo sát. Một pod chạy dịch vụ web đối mặt người dùng tồn tại lỗ hổng và bị kẻ tấn công kiểm soát (giai đoạn 1). Từ chỗ đứng này, kẻ tấn công không tấn công thẳng ra biên — nơi vẫn được tường lửa canh gác — mà quay vào trong: chạy một công cụ quét cổng nhắm tới dải địa chỉ pod của các namespace lân cận để tìm dịch vụ đang mở (giai đoạn 2). Với lớp giám sát hiện tại, hàng trăm lượt kết nối dò quét này hòa lẫn vào lưu lượng Đông–Tây và *không* sinh ra cảnh báo nào, bởi Wazuh chỉ thấy sự kiện ở tầng host. Phát hiện được mẫu đang mở, kẻ tấn công lần lượt kết nối sang một pod xử lý nội bộ rồi sang pod cơ sở dữ liệu (giai đoạn 3–4), hình thành một chuỗi liên lạc pod–tới-pod bất thường mà không công cụ nào trong cụm ghi nhận.

Đặt cạnh nhau, kịch bản này làm lộ rõ hai cơ hội phát hiện mà hệ thống đề xuất khai thác. Thứ nhất, ở giai đoạn trình sát, pod tấn công tạo ra một *số lượng kết nối tăng vọt* trong thời gian ngắn — một dị thường về tần suất mà bộ đếm Count-Min Sketch nắm bắt được ngay khi sự kiện vừa sinh ra. Thứ hai, ở giai đoạn dịch chuyển ngang, chuỗi pod bị chiếm tạo ra một *cấu trúc liên thông bất thường* trên đồ thị giao tiếp mà giải thuật đồ thị phát hiện được, đồng thời cho phép ước lượng bán kính ảnh hưởng để ưu tiên xử lý.

Chính việc một sự kiện đơn lẻ đã đủ để kích hoạt tín hiệu tần suất, còn cấu trúc đồ thị bổ sung bức tranh quan hệ, là lý do hệ thống có thể can thiệp *trước* khi kẻ tấn công chạm tới mục tiêu — điều mà lớp giám sát theo lô, hướng host hiện tại không làm được.

1.4. Khảo sát hiện trạng và khoảng trống nghiên cứu

1.4.1. Bằng chứng từ ba bài tổng quan có hệ thống gần đây

Nhu cầu thực tế ghi nhận ở doanh nghiệp không phải là một trường hợp cá biệt. Để đặt nó vào bức tranh chung và bảo đảm đề tài không xuất phát từ giả định chủ quan, đề án đối chiếu với **ba bài khảo sát có hệ thống** (*systematic literature review*, SLR) gần nhất về an toàn thông tin của hệ vi dịch vụ trên Kubernetes. SLR là phương pháp tổng quan tài liệu theo quy trình minh bạch (xác định câu hỏi, sàng lọc theo tiêu chí, tổng hợp có hệ thống), nên kết luận của chúng có độ tin cậy cao hơn các bài tổng quan thông thường. Ba công trình dưới đây bổ sung cho nhau cả về quy mô khảo sát lẫn góc nhìn:

Hutasuhut và cộng sự (2024) [9] sàng lọc 487 công bố trong giai đoạn sau năm 2020 và chọn lại 87 bài bằng phương pháp snowball. Nhóm tác giả kết luận rằng vẫn thiếu vắng đáng kể các chuẩn bảo mật riêng và các mô hình toàn diện được thiết kế *đặc biệt* cho vi dịch vụ, đồng thời tồn tại một khoảng cách kiến thức rõ rệt giữa giới học thuật và giới thực hành công nghiệp. Khoảng cách này trùng khớp với điều quan sát được ở doanh nghiệp: các đội vận hành biết mình thiếu khả năng quan sát nội cụm nhưng không có một mô hình tham chiếu sẵn để áp dụng.

Berardi và cộng sự (2022) [3] khảo sát 290 công bố và xác định bốn khoảng trống cốt lõi: thiếu hướng dẫn áp dụng *security-by-design* cho kiến trúc vi dịch vụ; thiếu mô hình mối đe dọa chuyên biệt; thiếu hướng dẫn xử lý bề mặt tấn công sinh ra từ sự đa dạng công nghệ; và các vấn đề an toàn khi di trú từ hệ đơn khối sang vi dịch vụ. Đáng chú ý, ba trong bốn khoảng trống này liên quan trực tiếp tới mô hình mối đe dọa và bề mặt tấn công — đúng phần mà đề án tập trung làm rõ qua chuỗi kill-chain và hai mẫu hành vi quét cổng, dịch chuyển ngang.

Pereira-Vale và cộng sự (2021) [12] thực hiện một bài tổng quan đa nguồn (*multivocal review*) kết hợp 36 công bố học thuật với 34 nguồn xám (báo cáo công nghiệp, tài liệu kỹ thuật của các nhà cung cấp lớn). Kết quả cho thấy một sự lệch pha hai chiều: nhiều giải pháp được công nghiệp triển khai chưa có cơ sở lý thuyết được kiểm chứng, trong khi nhiều phân tích lý thuyết lại chưa được hiện thực hóa trong hệ thống thực. Chính sự lệch pha này củng cố cho định hướng của đề án — gắn chặt mỗi giải thuật cốt lõi với cơ sở lý thuyết và một hiện thực đo được trên cụm thật.

1.4.2. Đối chiếu với báo cáo công nghiệp gần đây

Để đối chiếu kết luận học thuật với thực tế triển khai, đề án tham chiếu hai báo cáo công nghiệp gần nhất:

- *The State of Kubernetes Security 2024* của Red Hat [13], dựa trên khảo sát 600 chuyên gia DevOps và an toàn thông tin: **67%** tổ chức đã trì hoãn hoặc làm chậm phát triển ứng dụng do lo ngại an ninh; **45%** gặp sự cố ở giai đoạn *runtime* trong 12 tháng gần nhất; **30%** bị phạt và **46%** chịu mất doanh thu hoặc khách hàng sau sự cố.
- *Kubernetes Security Report 2025* của Wiz [18] bổ sung số liệu thời gian: một cluster K8s mới tạo trên dịch vụ managed (AKS, EKS) bị tấn công thử lần đầu trong vòng 18–28 phút sau khi sẵn sàng, cho thấy “đầu vào đối kháng” không phải tình huống hiếm gặp mà là điều kiện vận hành mặc định.

Bảng 1.4 tổng hợp các con số tiêu biểu từ các nguồn khảo sát trên, cho thấy an ninh Kubernetes vừa là vấn đề phổ biến vừa có hậu quả kinh doanh rõ rệt — một luận cứ định lượng củng cố cho nhu cầu thực tế đã ghi nhận ở Mục 1.2.

Bảng 1.4. Một số số liệu tiêu biểu về an ninh Kubernetes từ các khảo sát gần đây.

Nguồn	Số liệu	Ý nghĩa
Hutasuhut 2024 [9]	487 → 87 bài	Thiếu chuẩn bảo mật riêng và mô hình toàn diện cho vi dịch vụ
Berardi 2022 [3]	290 bài, 4 khoảng trống	Thiếu security-by-design và mô hình mối đe dọa chuyên biệt
Red Hat 2024 [13]	67% / 45% / 30% / 46%	Trì hoãn phát hành / sự cố runtime / bị phạt / mất doanh thu
Wiz 2025 [18]	18–28 phút	Thời gian tối thiểu để quét đầu tiên của một cụm mới

1.4.3. Khoảng trống cụ thể mà đề án nhắm tới

Khi đặt ba nguồn khảo sát cạnh nhau — nhu cầu thực tế của doanh nghiệp, các bài tổng quan học thuật và các công cụ công nghiệp — chúng hội tụ về cùng một khoảng trống. Từ phía *doanh nghiệp*, đó là sự mù lờ lượng Đông–Tây và vòng luẩn quẩn quan sát – kiểm soát. Từ phía *học thuật*, đó là sự thiếu vắng mô hình mối đe dọa chuyên biệt và kiến trúc tham chiếu cho vi dịch vụ. Từ phía *công cụ*, đó là việc mỗi công cụ chỉ giải quyết một mảnh và hiếm khi lấy cấu trúc dữ liệu luồng làm trọng tâm phát hiện.

Tổng hợp lại, đề án xác định một *trục* khoảng trống cụ thể: thiếu một **kiến trúc tham chiếu hoàn chỉnh** kết nối từ tầng nhân (thu thập sự kiện mạng), qua tầng cấu trúc dữ liệu (giải thuật tần suất, đồ thị), tới tầng dịch vụ (phát hiện sự cố, phản ứng) và giao diện vận hành (dashboard và console). Hầu hết các giải pháp công cụ hiện có (như Cilium

Hubble, Falco, Tetragon) tập trung vào một mảnh và không cho thấy cách các mảnh đó được *ráp lại* thành một quy trình giám sát Zero-Trust khép kín. Đề án nhằm bù đắp đúng khoảng trống này bằng cách xây dựng và đánh giá một hệ tham chiếu đầu cuối, được thiết kế để các thành phần lý thuyết và kỹ thuật có thể được tách rời và tái sử dụng — vừa giải quyết bài toán cụ thể của doanh nghiệp, vừa đóng góp một mẫu hình có thể khái quát hóa.

1.5. Mục tiêu, đối tượng và phạm vi

1.5.1. Mục tiêu nghiên cứu

1. Thu thập sự kiện kết nối TCP ở **tầng nhân Linux bằng eBPF** (`kprobe tcp_v4_connect`, `BPF_MAP_TYPE_LPM_TRIE`, `BPF_MAP_TYPE_RINGBUF`) trên mọi nút với chi phí thấp.
2. Hình thức hóa và giải quyết các bài toán cấu trúc dữ liệu luồng cốt lõi: ước lượng tần suất (Count-Min Sketch), phát hiện thành phần liên thông mạnh (Tarjan), khả-đạt-k-bước (BFS), phân loại địa chỉ IP theo tiền tố (LPM Trie).
3. Thiết kế kiến trúc phần mềm phân tầng, cấu hình hóa theo YAML, dùng mẫu thiết kế Strategy + Registry cho phép đổi giải thuật không cần sửa mã nguồn.
4. Xây dựng hệ thống SOC nội bộ (NATS, PostgreSQL, dịch vụ incident, bộ sinh NetworkPolicy theo cơ chế *generate-for-review*) thực hiện vòng phát hiện→phản hồi có người duyệt.
5. **Đánh giá định lượng** trên cluster K8s thật: tỉ lệ phát hiện, tỉ lệ báo động giả, chi phí eBPF, độ trễ phát hiện→phản hồi.

1.5.2. Đối tượng và phạm vi

Đối tượng nghiên cứu: hành vi quét cổng và dịch chuyển ngang trong mạng nội cụm K8s; các cấu trúc dữ liệu luồng và đồ thị phục vụ phát hiện; cơ chế thực thi chính sách mạng (NetworkPolicy).

Phạm vi:

- Kiến trúc Zero-Trust cho môi trường K8s on-premise (không bao gồm cloud-managed như EKS/GKE).
- Phát hiện các mẫu tấn công tiêu biểu: quét cổng (*port scan*), dịch chuyển ngang (*lateral movement*) liên nút.
- Toàn bộ thành phần do sinh viên tự cài đặt, ngoại trừ hạ tầng quan trắc và message bus dùng bản chuẩn thượng nguồn.

Giới hạn:

- Phản ứng ở chế độ *generate-for-review*: hệ thống tạo bản nháp NetworkPolicy, người vận hành phê duyệt; không tự động áp đặt.
- Chỉ hỗ trợ IPv4 (`tcp_v4_connect`); đánh giá ở quy mô lớn là hướng phát triển.

Bảng 1.5 tóm tắt ranh giới phạm vi để tránh hiểu nhầm về kỳ vọng đối với hệ

thống.

Bảng 1.5. Ranh giới phạm vi của đề án.

Trong phạm vi	Ngoài phạm vi
Thu thập kết nối TCP/IPv4 ở tầng nhân	Lưu lượng UDP, ICMP, IPv6
Phát hiện quét cổng và dịch chuyển ngang	Phát hiện mã độc, khai thác lỗ hổng tầng ứng dụng
Cụm Kubernetes on-premise	Dịch vụ K8s do đám mây quản lý (EK-S/GKE/AKS)
Phản hồi cách ly có người duyệt	Tự động cách ly không người duyệt
Đánh giá trên cụm ba nút	Đánh giá tải ở quy mô hàng nghìn nút

1.6. Khảo sát giải pháp và công cụ liên quan

Để định vị đề tài, mục này khảo sát hai nhóm giải pháp: an ninh *truyền thống* và an ninh *cloud-native*, rồi đối chiếu các công cụ tiêu biểu đang được công nghiệp dùng. Phần so sánh hiệu năng định lượng chi tiết với các công cụ này được đặt ở Chương 5; ở đây tập trung vào khác biệt về *cách tiếp cận*.

1.6.1. An ninh truyền thống so với an ninh cloud-native

An ninh truyền thống dựa trên ba trụ: tường lửa/IDS đặt ở biên mạng, tác nhân giám sát cài trên từng máy chủ (host-agent, ví dụ Wazuh trong môi trường khảo sát), và hệ SIEM phân tích nhật ký theo lô. Mô hình này phù hợp với kiến trúc một khối nhưng bộc lộ giới hạn rõ rệt trong môi trường vi dịch vụ: nó không quan sát được lưu lượng Đông–Tây nội cụm, không bám theo vòng đời pod ngắn, và phản ứng theo lô nên trễ. Bảng 1.6 đối chiếu hai cách tiếp cận trên các trục then chốt.

Bảng 1.6. So sánh an ninh truyền thống và an ninh cloud-native (tầng nhân).

Trục	Truyền thống	Cloud-native (đề án)
Vị trí quan sát	Biên mạng / máy chủ (Bắc–Nam)	Tầng nhân mỗi nút (Đông–Tây)
Cách thu thập	Host-agent, mirror gói, theo lô	eBPF không xâm lấn, một DaemonSet/nút
Độ phủ nội cụm	Hầu như không thấy pod–tới–pod	Toàn bộ kết nối pod–tới–pod
Phụ thuộc ứng dụng	Có (agent theo ngôn ngữ)	Không (ở tầng nhân)
Mô hình vận hành	Cấu hình thủ công, dễ trôi dạt	Khai báo GitOps, ảnh bất biến, tái lập
Vòng phản hồi	Phân tích lô, xử lý thủ công	Phát hiện trực tuyến, cách ly có người duyệt

1.6.2. Các công cụ an ninh cloud-native tiêu biểu

Trong hệ sinh thái cloud-native đã xuất hiện nhiều công cụ khai thác eBPF hoặc BPF-LSM. *Falco* [17] phát hiện bất thường theo luật trên dòng lời gọi hệ thống; *Tetragon* [10] cung cấp quan sát và cưỡng chế ở không gian nhân qua LSM; *Cilium* [4] là một CNI dựa trên eBPF với khả năng quan sát luồng (Hubble) và NetworkPolicy L3–L7; *KubeArmor* [1] cưỡng chế hành vi tiến trình/tệp/mạng bằng BPF-LSM. Các công cụ NIDS truyền thống như Snort [14] và Bro/Zeek [11] phân tích gói ở mức mạng nhưng không gắn với ngữ cảnh điều phối của Kubernetes. Bảng 1.7 đối chiếu các công cụ này với đề án.

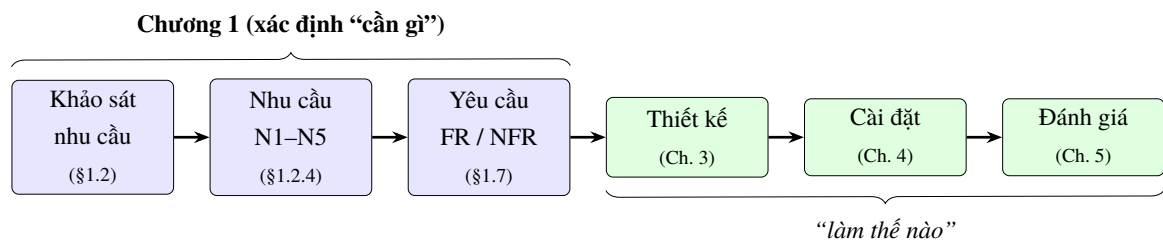
Bảng 1.7. Đối chiếu một số công cụ liên quan với đề án.

Công cụ	Trọng tâm	eBPF	CTDL luồng
Falco	Phát hiện theo luật trên syscall	Có	Không
Tetragon	Quan sát + cưỡng chế qua LSM	Có	Không
Cilium/Hubble	CNI + quan sát luồng + policy L3–L7	Có	Không
KubeArmor	Cưỡng chế hành vi qua BPF-LSM	Có	Không
Wazuh	SIEM/host-IDS theo nhật ký	Không	Không
Snort, Bro/Zeek	NIDS phân tích gói ở mức mạng	Không	Không
Đề án	Phát hiện quét cổng/dịch chuyển ngang	Có	Có (CMS, đồ thị)

Điểm chung của các công cụ trên là khai thác eBPF/LSM để quan sát hoặc cưỡng chế ở tầng nhân. Điểm khác biệt — cũng là khoảng trống đề án lấp — là rất ít công cụ đặt *cấu trúc dữ liệu luồng* (Count-Min Sketch) và *giải thuật đồ thị* (Tarjan, BFS) làm trọng tâm phát hiện quét cổng và dịch chuyển ngang, đồng thời ráp các mảnh thành một quy trình phát hiện→phản hồi khép kín có người duyệt.

1.7. Phân tích yêu cầu, đánh giá khả thi và đóng góp

Từ tập nhu cầu nghiệp vụ ở Mục 1.2.4, mục này chưng cất thành các *yêu cầu* cụ thể — bước chuyển từ “cần gì” sang “hệ thống phải làm gì”. Đây là đầu ra của giai đoạn phân tích yêu cầu ở mức tổng quan; việc phân tích chi tiết thành ca sử dụng, sơ đồ và thiết kế được tiếp nối ở Chương 3. Hình 1.6 tóm tắt mạch truy vết xuyên suốt: từ khảo sát nhu cầu, qua đặc tả yêu cầu, tới thiết kế, cài đặt và đánh giá — trong đó chương này phụ trách phần “cần gì”.



Hình 1.6. Quy trình kỹ nghệ yêu cầu và mạch truy vết từ nhu cầu tới các chương sau.

1.7.1. Yêu cầu chức năng

Bảng 1.8 liệt kê các yêu cầu chức năng (*functional requirement*), mỗi yêu cầu truy vết được về nhu cầu đã nêu.

Bảng 1.8. Yêu cầu chức năng của hệ thống.

Mã	Yêu cầu chức năng	Nhu cầu
FR-01	Thu thập sự kiện kết nối TCP toàn cụm ở tầng nhân (eBPF)	N1
FR-02	Phân loại địa chỉ theo vùng mạng (pod/dịch vụ/nút/ngoài)	N2
FR-03	Phát hiện quét cổng theo tần suất kết nối (Count-Min Sketch)	N2
FR-04	Dựng đồ thị giao tiếp; phát hiện liên thông mạnh và bán kính ảnh hưởng	N3
FR-05	Tạo và làm giàu sự cố (ánh xạ địa chỉ nguồn → pod)	N3, N5
FR-06	Sinh bản nhập NetworkPolicy cách ly (egress-deny)	N4
FR-07	Quy trình phê duyệt có người duyệt trước khi áp chính sách	N4
FR-08	Áp chính sách qua API Kubernetes và xác minh hiệu lực	N4
FR-09	Ghi nhật ký kiểm toán mọi sự cố và hành động	N5
FR-10	Bảng điều khiển và console vận hành cho người dùng	N3, N5

1.7.2. Yêu cầu phi chức năng

Bảng 1.9 liệt kê các yêu cầu phi chức năng (*non-functional requirement*) — những ràng buộc về chất lượng quyết định tính khả dụng của giải pháp ở quy mô doanh nghiệp.

Bảng 1.9. Yêu cầu phi chức năng của hệ thống.

Mã	Yêu cầu phi chức năng
NFR-01	Chi phí thấp: phần thu thập ở tầng nhân chiếm phần nhỏ một lõi CPU mỗi nút
NFR-02	Khả mở rộng: hoạt động được trên cụm hàng nghìn pod / nhiều nút
NFR-03	Không xâm lấn: không cài tác nhân vào ứng dụng, độc lập ngôn ngữ
NFR-04	Tái lập được: triển khai khai báo bằng GitOps với ảnh bất biến
NFR-05	Kiểm toán được: mọi hành động phản hồi đều để lại dấu vết
NFR-06	Tự chủ nội bộ: chạy trọn vẹn trên hạ tầng on-premise, không phụ thuộc đám mây
NFR-07	An toàn khi phản hồi: cách ly chỉ được áp sau khi có người phê duyệt

1.7.3. Đánh giá tính khả thi

Trước khi cam kết thiết kế, cần đánh giá tính khả thi của giải pháp đề xuất trên ba khía cạnh. Về *kỹ thuật*: các nút của cụm chạy nhân Linux hiện đại có sẵn thông tin BTF, đủ điều kiện để nạp chương trình eBPF CO-RE; CNI Calico đã hiện diện sẵn nên cơ chế cường chế NetworkPolicy cho vòng phản hồi là sẵn dùng mà không phải thay đổi hạ tầng mạng; chi phí quan sát ở tầng nhân được đo là rất nhỏ (Chương 5), phù hợp với ràng buộc NFR-01. Về *kinh tế*: toàn bộ thành phần là mã nguồn mở, không phát sinh chi phí giấy phép theo pod — một lợi thế quyết định khi quy mô vượt một nghìn deployment, nơi mô hình tác nhân-theo-pod trở nên đắt đỏ. Về *vận hành*: mô hình DaemonSet cho phép triển khai một lần trên mỗi nút và tự lan theo cụm; vòng phản hồi có người duyệt tránh rủi ro tự động cách ly nhằm ở quy mô lớn, đáp ứng đúng lo ngại của đội SRE đã ghi nhận khi khảo sát. Cả ba khía cạnh đều cho thấy giải pháp khả thi trong đúng bối cảnh doanh nghiệp đã khảo sát.

Tính khả thi đi kèm một số giả định và rủi ro cần được nhận diện sớm (Bảng 1.10); việc nêu rõ chúng giúp khoanh vùng đúng những điều kiện mà kết luận của đề án còn đúng và định hướng cho phần giới hạn cùng hướng phát triển ở Chương 5.

Bảng 1.10. Giả định, rủi ro và biện pháp giảm thiểu.

Mã	Giả định / rủi ro	Biện pháp giảm thiểu
R1	Nhân nút có BTF cho CO-RE	Kiểm tra điều kiện nhân; sinh <code>vmlinux.h</code> lúc xây ảnh
R2	Quét gửi SYN thô né connect()	Ghi nhận giới hạn; bổ sung điểm móc khác là hướng phát triển
R3	Client hợp lệ tạo nhiều kết nối → báo giả	Đếm theo cửa sổ + người duyệt trước khi cách ly
R4	Cách ly nhằm gây gián đoạn dịch vụ	Cơ chế <i>generate-for-review</i> , có người phê duyệt
R5	Đồ thị giao tiếp lớn dần theo thời gian	Chạy Tarjan/BFS theo chu kỳ, độ phức tạp tuyến tính

1.7.4. Đóng góp của đề án

Trên cơ sở phân tích nhu cầu và yêu cầu ở trên, đề án đặt mục tiêu mang lại bốn đóng góp chính. *Thứ nhất*, một kiến trúc tham chiếu đầu cuối cho giám sát Zero-Trust nội cụm, kết nối liền mạch từ tầng nhân (eBPF) qua tầng cấu trúc dữ liệu (Count-Min Sketch, đồ thị) tới tầng dịch vụ và giao diện vận hành — lấp đúng khoảng trống “thiếu kiến trúc ráp hoàn chỉnh” đã chỉ ra. *Thứ hai*, việc đặt cấu trúc dữ liệu luồng và giải thuật đồ thị làm trọng tâm phát hiện, kèm phân tích lý thuyết và ví dụ chạy tay (Chương 2). *Thứ ba*, một

vòng phản hồi cách ly có người duyệt tận dụng NetworkPolicy của Calico — biến phát hiện thành hành động ngăn chặn cụ thể mà không cần thay đổi hạ tầng mạng sẵn có. *Thứ tư*, một bộ số liệu đánh giá định lượng trên cụm Kubernetes thật (tỉ lệ phát hiện, báo động giả, chi phí eBPF, độ trễ phát hiện→phản hồi), giúp kiểm chứng tính thực tiễn của cách tiếp cận.

1.8. Phương pháp nghiên cứu

Đồ án kết hợp nghiên cứu lý thuyết với phương pháp xây dựng – đánh giá (*design science*): không dừng ở phân tích mà hiện thực một hệ thống thật rồi đo đạc để kiểm chứng giả thuyết. Bốn nhóm hoạt động được tổ chức theo đúng mạch các chương sau.

Nghiên cứu lý thuyết. Phân tích các bài báo gốc của từng giải thuật cốt lõi cùng các đảm bảo toán học của chúng (giới hạn sai số của Count-Min Sketch, tính đúng và độ phức tạp của Tarjan, BFS, LPM-Trie), làm cơ sở cho lựa chọn thiết kế ở Chương 2.

Phân tích và thiết kế hệ thống. Từ tập yêu cầu của chương này, dựng các tạo phẩm phân tích – thiết kế theo chuẩn kỹ nghệ phần mềm: sơ đồ use-case và đặc tả, sơ đồ phân rã chức năng, sơ đồ luồng dữ liệu, sơ đồ tuần tự, ERD và thiết kế giao diện (Chương 3).

Cài đặt và kiểm thử. Hiện thực hệ thống với ngôn ngữ phù hợp từng tầng: C cho chương trình eBPF, C++17 cho thư viện cấu trúc dữ liệu và pipeline, Go cho các dịch vụ SOC; kiểm thử đơn vị cho các thành phần then chốt (Chương 4).

Triển khai và đánh giá. Triển khai trên một cụm Kubernetes thật theo mô hình GitOps, chạy các kịch bản tấn công tự động và đo các chỉ số định lượng — tỉ lệ phát hiện, tỉ lệ báo động giả, chi phí eBPF và độ trễ phát hiện→phản hồi (Chương 5).

1.9. Kết luận chương

Chương này đã thực hiện trọn vẹn giai đoạn chuẩn bị của một quy trình kỹ nghệ phần mềm trước khi thiết kế: đi từ bối cảnh, khảo sát nhu cầu thực tế, tới phân tích yêu cầu và đánh giá khả thi. Từ bối cảnh dịch chuyển kiến trúc một khối sang vi dịch vụ trên Kubernetes, chương chỉ ra một bề mặt tấn công Đông–Tây mới mà an ninh chu vi truyền thống gần như không quan sát được. Khảo sát tại một doanh nghiệp vận hành Kubernetes nội bộ quy mô lớn đã lượng hóa khoảng trống này thành các điểm đau cụ thể — mà trung tâm là sự *mù lù lượng Đông–Tây* của lớp giám sát hướng host hiện tại — rồi chưng cất thành tập nhu cầu và, qua phân tích, thành các yêu cầu chức năng và phi chức năng có thể truy vết. Mô hình mối đe dọa theo chuỗi kill-chain làm rõ vì sao việc phát hiện sớm ở giai đoạn trinh sát và dịch chuyển ngang là điểm can thiệp hiệu quả nhất.

Đối chiếu với ba bài tổng quan có hệ thống, hai báo cáo công nghiệp và các công cụ cloud-native hiện có, chương xác định một khoảng trống cụ thể: thiếu một kiến trúc tham chiếu hoàn chỉnh ráp nối từ tầng nhân tới tầng dịch vụ và giao diện vận hành, lấy

cấu trúc dữ liệu luồng và giải thuật đồ thị làm trọng tâm phát hiện. Trên cơ sở đó, chương đã phát biểu bốn đóng góp chính, mục tiêu, đối tượng và phạm vi của đề tài. Toàn bộ các yêu cầu và căn cứ này là đầu vào trực tiếp cho việc phân tích, cụ thể hóa thành thiết kế ở Chương 3, sau khi đã trang bị cơ sở lý thuyết và công nghệ lõi ở Chương 2.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

Chương này trình bày cơ sở lý thuyết của các giải thuật cốt lõi và các công nghệ nền tảng được sử dụng trong đồ án. Bối cảnh kiến trúc (sự dịch chuyển từ một khối sang vi dịch vụ), các mẫu tấn công cần phát hiện và lý do chọn quan sát ở tầng nhân đã được trình bày ở Chương 1; chương này tập trung đi sâu vào *cách* hiện thực. Bám sát đúng đường đi của một sự kiện trong hệ thống, chương lần lượt làm rõ: cơ chế thu thập ở tầng nhân bằng eBPF (Mục 2.1), ước lượng tần suất bằng Count-Min Sketch để phát hiện quét cổng (Mục 2.2), phân tích đồ thị và phân loại địa chỉ (Mục 2.3), và cuối cùng là nền tảng công nghệ triển khai (Mục 2.4).

2.1. Thu thập sự kiện ở tầng nhân bằng eBPF

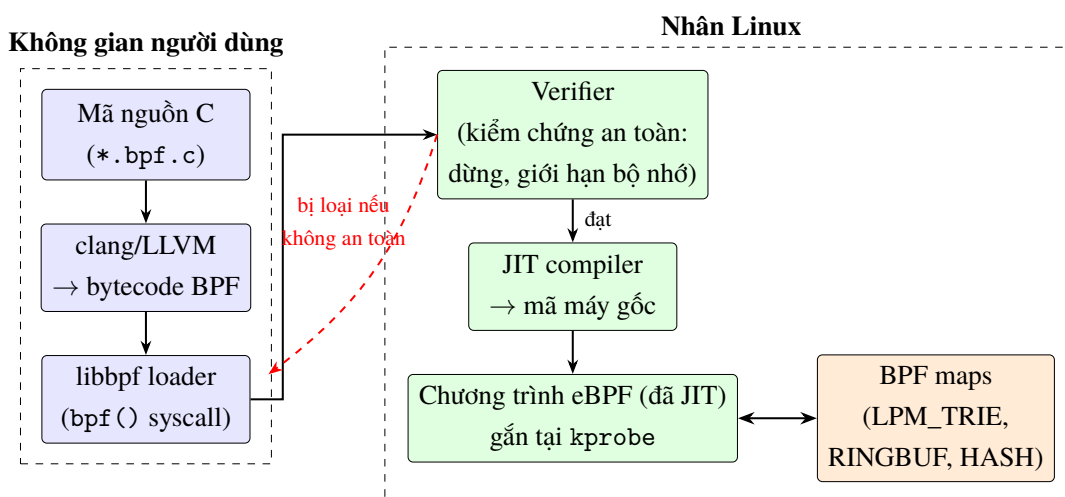
2.1.1. eBPF và mô hình thực thi an toàn trong nhân Linux



Hình 2.1. Logo eBPF.

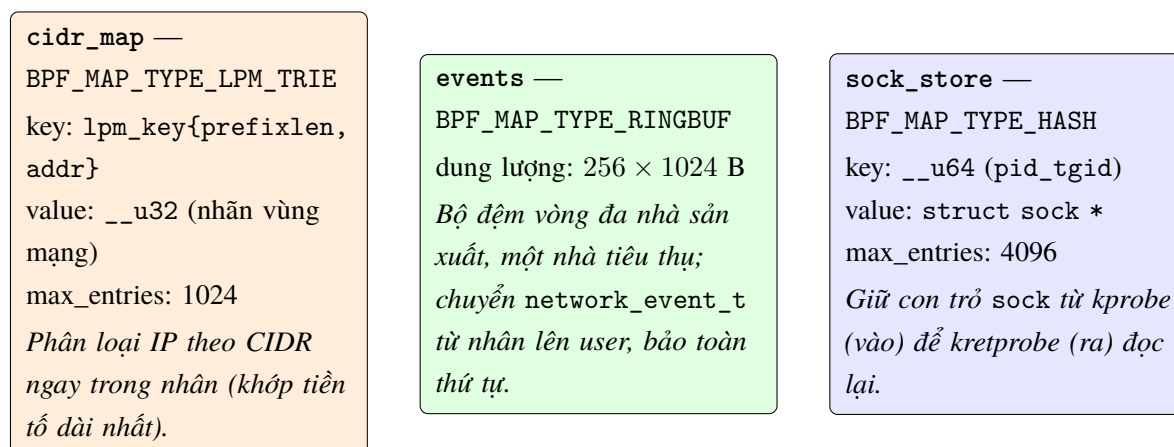
eBPF (*extended Berkeley Packet Filter*) là một máy ảo nhỏ chạy ngay bên trong nhân Linux, cho phép nạp và thực thi các chương trình do người dùng cung cấp tại nhiều điểm móc (*hook*) của nhân mà không cần biên dịch lại nhân hay nạp mô-đun nhân [8]. Điểm mấu chốt khiến eBPF phù hợp cho bài toán giám sát an ninh là cơ chế *verifier*: trước khi một chương trình eBPF được nạp, bộ kiểm chứng của nhân phân tích tĩnh toàn bộ đồ thị luồng điều khiển để bảo đảm chương trình luôn kết thúc (không có vòng lặp vô hạn), không truy cập bộ nhớ ngoài phạm vi cho phép, và không thực hiện thao tác nguy hiểm. Sau khi vượt qua bộ kiểm chứng, chương trình được biên dịch tức thời (*JIT*) sang mã máy gốc nên chạy với hiệu năng gần như mã nhân biên dịch sẵn. So với việc viết một mô-đun nhân truyền thống — vốn có thể làm sập (*panic*) toàn bộ hệ điều hành nếu mắc lỗi và phải biên dịch lại theo từng phiên bản nhân — eBPF mang lại sự an toàn và khả năng triển khai linh hoạt hơn hẳn, đây chính là lý do nó trở thành nền tảng cho phần lớn công cụ quan sát an ninh hiện đại trên Kubernetes. Hình 2.2 tóm tắt vòng đời của một chương trình eBPF: mã C được biên dịch sang bytecode BPF bằng clang/LLVM, nạp vào nhân qua lời gọi `bpf()`, đi qua bộ kiểm chứng (nếu không an toàn sẽ bị từ chối ngay), được JIT

sang mã máy gốc, rồi gắn tại điểm móc và tương tác với các BPF map.



Hình 2.2. Mô hình thực thi của một chương trình eBPF.

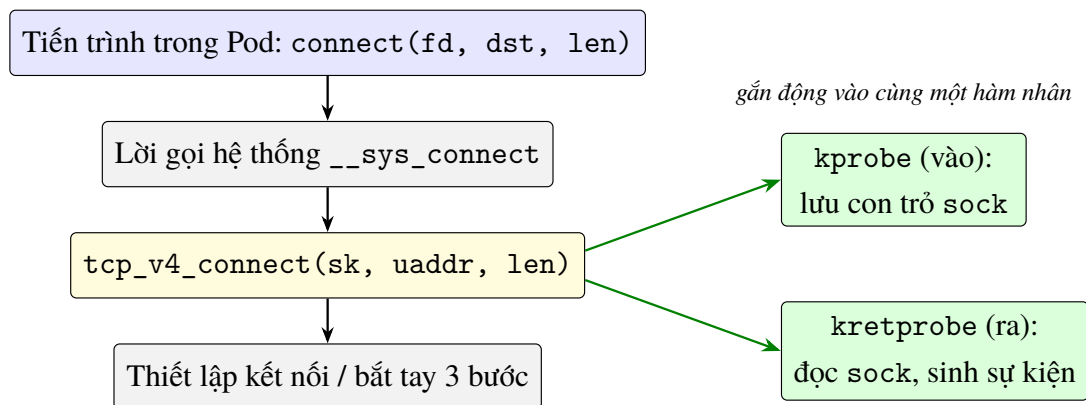
Một chương trình eBPF trao đổi dữ liệu với không gian người dùng và lưu trạng thái qua các cấu trúc gọi là *BPF map*. Bộ thu thập của đồ án dùng ba map, mỗi map một vai trò (Hình 2.3). *cidr_map* kiểu `BPF_MAP_TYPE_LPM_TRIE` phân loại địa chỉ IP theo dải mạng (CIDR) ngay trong nhân theo nguyên tắc khớp tiền tố dài nhất (Mục 2.3), nhờ đó lọc bỏ sớm những kết nối không thuộc mạng pod, giảm tải cho tầng người dùng. *events* kiểu `BPF_MAP_TYPE_RINGBUF` là bộ đệm vòng đa nhà sản xuất, một nhà tiêu thụ, dùng để chuyển sự kiện từ nhân lên không gian người dùng; so với *perf buffer* thế hệ trước (cấp phát bộ đệm theo từng CPU và dễ mất thứ tự sự kiện giữa các CPU), ring buffer bảo toàn thứ tự toàn cục và dùng bộ nhớ hiệu quả hơn, phù hợp cho luồng kết nối mật độ cao. Cuối cùng, *sock_store* kiểu `BPF_MAP_TYPE_HASH` giữ con trỏ sock mà chương trình lưu lúc vào hàm để đọc lại lúc hàm trả về — chi tiết kỹ thuật cần thiết sẽ được làm rõ ngay sau đây.



Hình 2.3. Ba BPF map của bộ thu thập và vai trò tương ứng.

2.1.2. Lựa chọn điểm móc và tính khả chuyển nhờ CO-RE

Nhân Linux cho phép gắn chương trình eBPF tại nhiều tầng khác nhau, mỗi tầng có sự đánh đổi riêng. XDP và tc hoạt động rất sớm ở tầng trình điều khiển mạng, lý tưởng cho việc lọc/loại bỏ gói với thông lượng lớn nhưng chỉ thấy gói thô, khó truy được ngữ cảnh tiến trình. *Tracepoint* là các điểm móc tĩnh do nhân định nghĩa sẵn, ổn định nhưng kém linh hoạt. *kprobe* là điểm móc động cho phép chèn vào gần như bất kỳ hàm nào của nhân. Đồ án chọn gắn kprobe (và kretprobe tương ứng) vào hàm `tcp_v4_connect` — hàm được nhân gọi mỗi khi một tiến trình khởi tạo kết nối TCP/IPv4 qua lời gọi `connect()`. Tại điểm này, chương trình đọc được đầy đủ địa chỉ nguồn, địa chỉ đích, cổng đích và mã tiến trình, vốn là toàn bộ thông tin cần thiết để phát hiện hành vi quét cổng và dịch chuyển ngang. Mô hình quan sát ở đây mang tính nền tảng: vì mọi kết nối TCP của mọi pod trên một nút đều phải đi qua nhân của nút đó, một chương trình eBPF đặt tại `tcp_v4_connect` sẽ quan sát được trọn vẹn lưu lượng kết nối mà không cần cài đặt tác nhân vào từng container. Hình 2.4 cho thấy vị trí của hai điểm móc trên đường gọi hàm nhân, còn Bảng 2.1 đối chiếu các lựa chọn điểm móc.



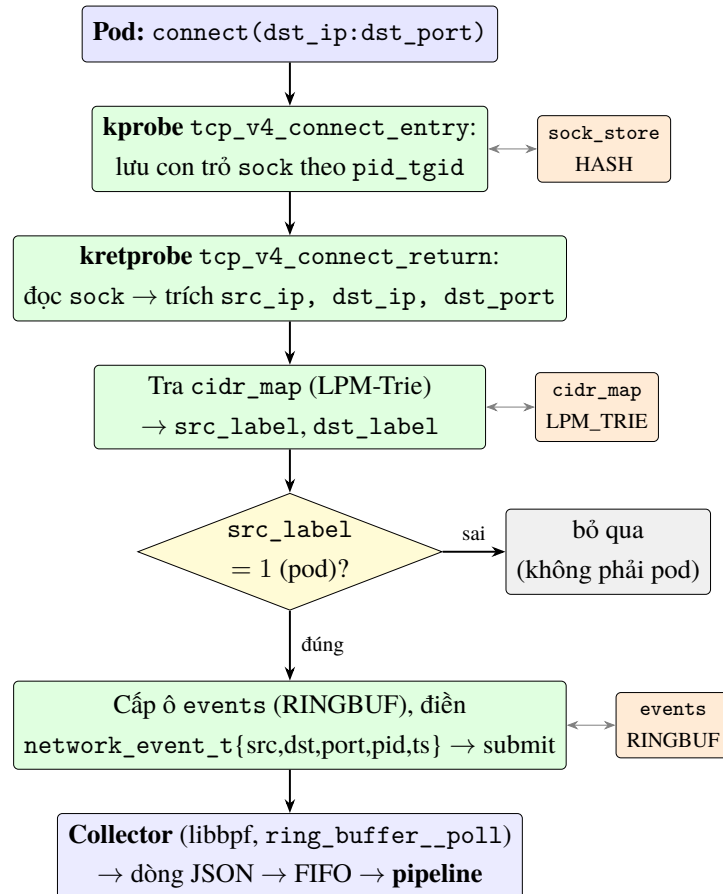
Hình 2.4. Vị trí gắn kprobe/kretprobe trong ngăn xếp TCP.

Bảng 2.1. So sánh các điểm móc eBPF cho bài toán quan sát kết nối.

Điểm móc	Tầng	Đặc điểm / phù hợp
XDP / tc	Trình điều khiển / hàng đợi	Rất sớm, thông lượng cao, nhưng chỉ thấy gói thô, khó truy ngữ cảnh tiến trình
tracepoint	Điểm tĩnh của nhân	Ổn định theo phiên bản nhưng cố định, kém linh hoạt
kprobe	Hàm nhân (động)	Gắn được vào <code>tcp_v4_connect</code>; đọc đủ <code>src/dst/-port/pid</code> — lựa chọn của đồ án

Cơ chế thu thập được tách làm hai nửa vì lý do kỹ thuật: tại thời điểm *vào* hàm (*kprobe*), địa chỉ đích chưa chắc đã được điền đầy đủ vào cấu trúc `sock`, nên

chương trình chỉ lưu con trỏ sock theo khóa pid_tgid vào sock_store; tại thời điểm *trả về* (kretprobe), chương trình đọc lại con trỏ này, trích ra địa chỉ nguồn, địa chỉ đích và cổng đích, tra cidr_map để xác định nhãn vùng mạng, và chỉ khi nguồn thuộc mạng pod (src_label = 1) mới cấp một ô trong events (ring buffer) để điền cấu trúc network_event_t gồm {src_ip, dst_ip, dst_port, pid, timestamp_ns} rồi đẩy lên không gian người dùng. Bộ lọc theo nhãn ngay trong nhân này loại bỏ phần lớn lưu lượng không liên quan tại nguồn, giảm tải đáng kể cho tầng phân tích. Hình 2.5 mô tả toàn bộ đường đi của một sự kiện từ lời gọi connect() trong pod tới khi tới được bộ máy pipeline.



Hình 2.5. Đường đi của một sự kiện kết nối qua eBPF.

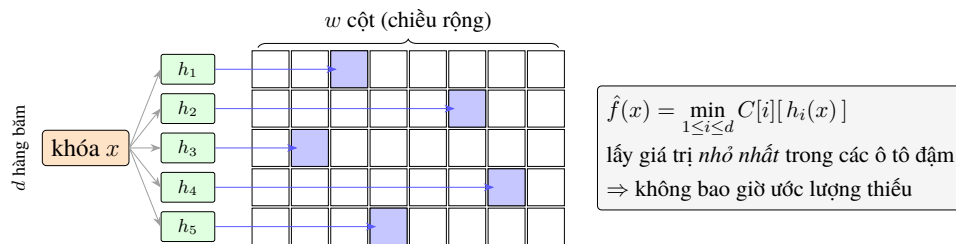
Để một chương trình eBPF biên dịch một lần có thể chạy trên nhiều phiên bản nhân khác nhau, đồ án dựa vào cơ chế CO-RE (*Compile Once – Run Everywhere*) kết hợp với thông tin kiểu BTF (*BPF Type Format*) mà nhân hiện đại đính kèm. Thay vì giả định bố cục bộ nhớ của các cấu trúc nhân tại thời điểm biên dịch (điều buộc phải có header trùng khớp với từng nút), chương trình truy cập trường dữ liệu thông qua các phép định vị lại (*relocation*) được giải tại thời điểm nạp dựa trên BTF của nhân đích. Nhờ đó cùng một tệp đối tượng eBPF chạy được trên cụm có nhân không đồng nhất, giảm đáng kể chi phí vận hành. Cuối cùng, cần ghi nhận một giới hạn vốn có của lựa chọn điểm mốc này: vì kprobe trên tcp_v4_connect chỉ kích hoạt khi có lời gọi connect(), các kiểu quét gửi

gói SYN thô (ví dụ nmap -sS chạy với quyền root) sẽ không đi qua đường này và do đó không bị quan sát; phân tích định lượng về giới hạn này được trình bày ở Chương 5.

2.2. Ước lượng tần suất trên luồng: Count-Min Sketch

Bài toán cốt lõi mà tăng phát hiện phải giải là: trên một luồng sự kiện kết nối gần như vô hạn và tốc độ cao, ước lượng *tần suất xuất hiện* của mỗi địa chỉ nguồn để nhận ra nguồn nào đang tạo ra số lượng kết nối bất thường. Lưu trữ chính xác tần suất bằng bảng băm đòi hỏi bộ nhớ tỉ lệ thuận với số phần tử phân biệt — không khả thi khi số địa chỉ và cổng có thể rất lớn và biến động liên tục. Count-Min Sketch (CMS) [6] giải quyết vấn đề này bằng một cấu trúc xác suất sử dụng bộ nhớ dưới tuyến tính, đánh đổi một sai số ước lượng có thể kiểm soát được để lấy bộ nhớ cố định và thời gian cập nhật/truy vấn hằng số.

Về cấu trúc, CMS là một ma trận đếm kích thước $d \times w$ đi kèm d hàm băm độc lập đôi một, mỗi hàm ánh xạ một khóa vào một trong w cột (đồ án dùng họ hàm băm phi mật mã MurmurHash [2] vì tốc độ cao và phân bố tốt). Thao tác $\text{record}(x)$ tăng một đơn vị tại ô $(i, h_i(x))$ với mọi hàng i ; thao tác $\text{estimate}(x)$ trả về *giá trị nhỏ nhất* trong d ô đó. Tính chất nền tảng của cấu trúc là nó *không bao giờ ước lượng thiếu*: do va chạm băm chỉ có thể làm các ô bị cộng thêm, ta luôn có $\hat{f}(x) \geq f(x)$. Mặt khác, sai số vượt lên trên bị chặn theo xác suất: với chiều rộng $w = \lceil e/\varepsilon \rceil$ và chiều sâu $d = \lceil \ln(1/\delta) \rceil$, ước lượng thỏa mãn $\hat{f}(x) \leq f(x) + \varepsilon \|\mathbf{f}\|_1$ với xác suất ít nhất $1 - \delta$. Phát biểu và chứng minh đầy đủ của cận này, cùng phân tích so sánh với Misra–Gries và Count Sketch, đã được trình bày trong Báo cáo học phần CNPM 1 nên ở đây chỉ nêu lại kết quả phục vụ phần thiết kế. Hình 2.6 minh họa trực quan cấu trúc, còn Giải thuật 1 tóm tắt hai thao tác cơ bản.



Hình 2.6. Cấu trúc Count-Min Sketch.

Giải thuật 1 Count-Min Sketch — cập nhật và truy vấn

```
1: function RECORD( $x$ )
2:   for  $i \leftarrow 1$  to  $d$  do
3:      $C[i][h_i(x) \bmod w] \leftarrow C[i][h_i(x) \bmod w] + 1$ 
4:   end for
5: end function
6: function ESTIMATE( $x$ )
7:   return  $\min_{1 \leq i \leq d} C[i][h_i(x) \bmod w]$ 
8: end function
```

Để thấy rõ cơ chế, Bảng 2.2 minh hoạ trạng thái ma trận sau một luồng ngắn chạy tay.

Bảng 2.2. Trạng thái ma trận CMS ($d = 2, w = 4$) sau luồng $S = (a, b, a, c, a, b)$, với $h_1: a \rightarrow 1, b \rightarrow 3, c \rightarrow 1$ và $h_2: a \rightarrow 2, b \rightarrow 0, c \rightarrow 3$.

	cột 0	cột 1	cột 2	cột 3
hàng h_1	0	4	0	2
hàng h_2	2	0	3	1

Khi đó $\hat{f}(a) = \min(C[1][1], C[2][2]) = \min(4, 3) = 3 = f(a)$ (chính xác), và $\hat{f}(c) = \min(C[1][1], C[2][3]) = \min(4, 1) = 1 = f(c)$: dù ô $C[1][1]$ bị va chạm giữa a và c (giá trị 4), phép lấy nhỏ nhất vẫn cho ước lượng đúng nhờ hàng còn lại — đây chính là trực giác đằng sau tính chất không-ước-lượng-thiếu.

Trong hệ thống của đồ án, CMS được dùng với khóa là *địa chỉ IP nguồn*: mỗi sự kiện kết nối làm tăng bộ đếm của địa chỉ nguồn tương ứng, và khi giá trị ước lượng của một nguồn vượt một ngưỡng định trước trong một cửa sổ thời gian, hệ thống phát một cảnh báo `port_scan`. Cấu hình thực tế dùng ma trận 5×2048 với ngưỡng 40 trên cửa sổ trượt kiểu *tumbling* dài 60 giây. Cần nhấn mạnh một đặc điểm thiết kế có hệ quả đến độ chính xác: bộ đếm khóa theo địa chỉ nguồn nên nó đếm *tổng lưu lượng kết nối* của nguồn đó, chứ không đếm *số cổng đích phân biệt*. Lựa chọn này khiến bộ phát hiện thực chất là một bộ phát hiện theo tốc độ kết nối; ưu, nhược điểm và các hệ quả định lượng của nó (bỏ sót khi quét chậm, báo giả khi một client hợp lệ tạo nhiều kết nối) được phân tích kỹ ở Chương 5.

2.3. Phân tích đồ thị và phân loại địa chỉ trong mạng nội cụm

Ngoài việc đếm tần suất từng nguồn riêng lẻ, hệ thống còn dựng một *đồ thị truyền thông* trong đó mỗi đỉnh là một địa chỉ và mỗi cung biểu diễn một kết nối đã quan sát

được. Trên đồ thị này, hai bài toán đồ thị kinh điển được vận dụng để phát hiện những mẫu hành vi mà phép đếm tần suất đơn lẻ không nhìn thấy, kèm theo một bài toán phân loại địa chỉ phục vụ cho cả tầng nhân lẫn tầng phân tích.

2.3.1. Tarjan — phát hiện thành phần liên thông mạnh

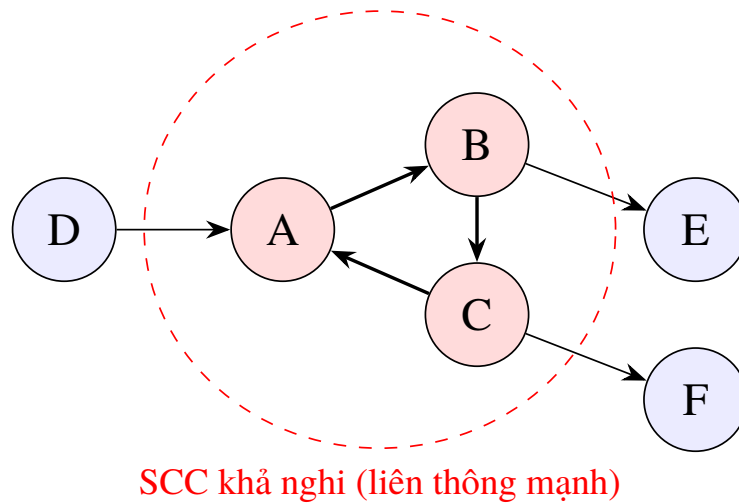
Một thành phần liên thông mạnh (*Strongly Connected Component*, SCC) của đồ thị có hướng là một tập đỉnh cực đại trong đó mọi cặp đỉnh đều có thể đến được nhau. Trong ngữ cảnh an ninh nội cụm, một SCC không tầm thường (kích thước lớn hơn một) giữa các pod là một dấu hiệu đáng ngờ: nó cho thấy một nhóm pod đang liên lạc vòng tròn lẫn nhau, mẫu hình thường gặp ở các mạng điều khiển (*command-and-control*) hoặc ở giai đoạn dịch chuyển ngang khi kẻ tấn công đã kiểm soát nhiều điểm. Đồ án sử dụng giải thuật Tarjan [16] để liệt kê toàn bộ SCC trong một lần duyệt theo chiều sâu (DFS): mỗi đỉnh được gán một chỉ số khám phá và một giá trị *low-link* — chỉ số nhỏ nhất có thể quay về được qua các cung ngược — đồng thời được đẩy vào một ngăn xếp; khi một đỉnh có *low-link* bằng chính chỉ số khám phá của nó, toàn bộ các đỉnh phía trên nó trong ngăn xếp tạo thành một SCC. Giải thuật chạy trong thời gian tuyến tính $O(V + E)$ theo số đỉnh và số cung, đủ nhanh để chạy định kỳ trên đồ thị truyền thông đang lớn dần của cụm. Giải thuật 2 trình bày ý tưởng cốt lõi của một lần duyệt.

Giải thuật 2 Tarjan — liệt kê thành phần liên thông mạnh

```

1:  $index \leftarrow 0$ ; ngăn xếp  $S \leftarrow \emptyset$ 
2: function STRONGCONNECT( $v$ )
3:    $v.index \leftarrow index$ ;  $v.low \leftarrow index$ ;  $index \leftarrow index + 1$ 
4:   đẩy  $v$  vào  $S$ ;  $v.onStack \leftarrow \mathbf{true}$ 
5:   for all cung  $(v, w)$  do
6:     if  $w$  chưa được thăm then
7:       STRONGCONNECT( $w$ );  $v.low \leftarrow \min(v.low, w.low)$ 
8:     else if  $w.onStack$  then
9:        $v.low \leftarrow \min(v.low, w.index)$ 
10:    end if
11:  end for
12:  if  $v.low = v.index$  then
13:    lấy các đỉnh khỏi  $S$  cho tới  $v \Rightarrow$  một thành phần liên thông mạnh
14:  end if
15: end function

```



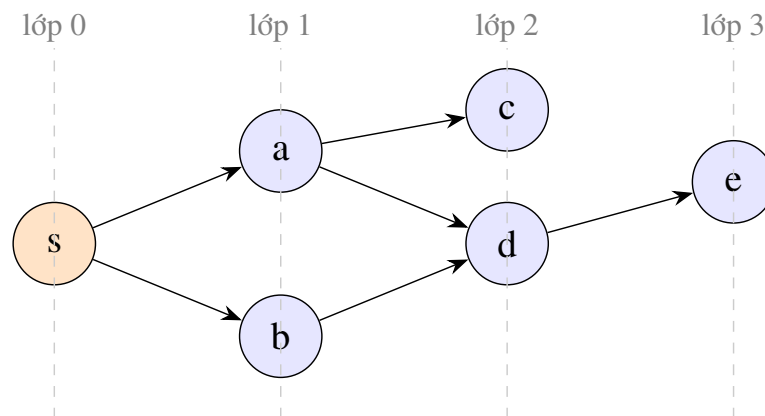
Hình 2.7. Ví dụ thành phần liên thông mạnh (SCC) trong đồ thị truyền thông.

2.3.2. BFS — bán kính ảnh hưởng theo k bước

Khi một pod bị nghi ngờ đã bị chiếm, một câu hỏi vận hành quan trọng là: từ pod đó, kẻ tấn công có thể với tới những đâu trong một số bước giới hạn? Hình thức hóa, ta cần tính tập khả-đạt- k -bước $R_k(s) = \{v : \text{tồn tại đường đi từ } s \text{ tới } v \text{ với độ dài } \leq k\}$. Vì đồ thị truyền thông không trọng số và ta quan tâm tới khoảng cách theo số bước, duyệt theo chiều rộng (BFS) là lựa chọn tự nhiên và tối ưu: BFS mở rộng đồ thị theo từng lớp khoảng cách tăng dần, nên chỉ cần dừng ở lớp thứ k là thu được toàn bộ tập khả-đạt mong muốn trong thời gian $O(V + E)$ [5]. Kích thước của $R_k(s)$ chính là *bán kính ảnh hưởng* (*blast radius*) của nguồn s — một chỉ số trực quan giúp người vận hành ưu tiên xử lý: một nguồn vừa vượt ngưỡng quét cổng lại vừa có bán kính ảnh hưởng lớn là sự cố cần can thiệp khẩn cấp. Hệ thống sử dụng $k = 3$ bước trong cấu hình mặc định; Giải thuật 3 mô tả phép duyệt theo lớp có giới hạn độ sâu.

Giải thuật 3 BFS giới hạn k bước — bán kính ảnh hưởng $R_k(s)$

```
1:  $R \leftarrow \{s\}$ ; hàng đợi  $Q \leftarrow [(s, 0)]$ 
2: while  $Q \neq \emptyset$  do
3:    $(u, d) \leftarrow$  lấy đầu hàng đợi  $Q$ 
4:   if  $d = k$  then
5:     bỏ qua, xét phần tử kế tiếp
6:   end if
7:   for all láng giềng  $v$  của  $u$  do
8:     if  $v \notin R$  then
9:        $R \leftarrow R \cup \{v\}$ ; thêm  $(v, d+1)$  vào  $Q$ 
10:    end if
11:  end for
12: end while
13: return  $R$ 
```



$R_3(s) = \{s, a, b, c, d, e\}$ — bán kính ảnh hưởng trong ≤ 3 bước

Hình 2.8. Bán kính ảnh hưởng theo BFS k bước từ nguồn s .

2.3.3. LPM Trie — phân loại địa chỉ theo tiền tố dài nhất

Cả tầng nhân lẫn tầng phân tích đều cần trả lời nhanh câu hỏi “địa chỉ này thuộc vùng mạng nào” để phân biệt lưu lượng nội bộ pod, lưu lượng dịch vụ, lưu lượng nút và lưu lượng bên ngoài. Đây chính là bài toán khớp tiền tố dài nhất (*Longest Prefix Match*): cho một tập dải mạng CIDR có gán nhãn, hãy tìm dải *cụ thể nhất* (tiền tố dài nhất) chứa một địa chỉ cho trước. LPM Trie giải bài toán này bằng một cây nhị phân theo bit của tiền tố: mỗi dải mạng được chèn dọc theo đường đi tương ứng với các bit tiền tố của nó kèm nhãn, còn thao tác tra cứu đi từ gốc theo các bit của địa chỉ và ghi nhớ nhãn tại nút khớp sâu nhất. So với cách băm toàn bộ địa chỉ (không biểu diễn được quan hệ bao hàm giữa các dải) hay cây Radix tổng quát, LPM Trie cho thời gian tra cứu chặn theo độ dài địa

chỉ và biểu diễn tự nhiên thứ bậc bao hàm của các dải. Trong đồ án, cùng một bảng phân loại được dùng ở hai nơi: trong nhân qua BPF_MAP_TYPE_LPM_TRIE để lọc sớm, và trong pipeline qua bộ phân loại ip_classifier. Bảng phân loại gán nhãn cho các dải tiêu biểu: 10.244.0.0/16 là mạng pod, 10.96.0.0/12 là mạng dịch vụ, 192.168.0.0/16 là mạng nút, và 0.0.0.0/0 là lưu lượng bên ngoài; nhờ đó mỗi sự kiện đều mang ngữ cảnh vùng mạng phục vụ cho việc lọc và phân tích về sau. Giải thuật 4 mô tả thao tác tra cứu nhãn.

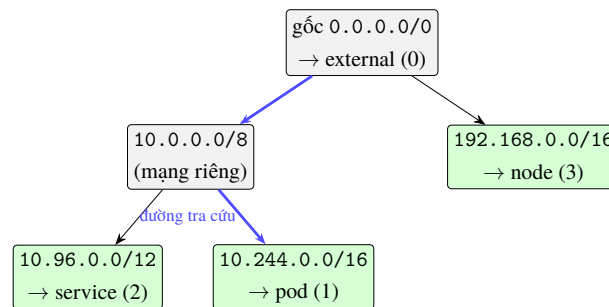
Giải thuật 4 LPM Trie — tra cứu nhãn theo tiền tố dài nhất

```

1: function LOOKUP(địa chỉ  $a$ )
2:    $node \leftarrow root$ ;  $label \leftarrow \text{NONE}$ 
3:   for all bit  $b$  của  $a$ , từ cao xuống thấp do
4:     if  $node$  có gán nhãn then
5:        $label \leftarrow node.label$ 
6:     end if
7:     if  $node.child[b]$  rỗng then
8:       dừng vòng lặp
9:     end if
10:     $node \leftarrow node.child[b]$ 
11:  end for
12:  return  $label$ 
13: end function

```

▷ nhãn của tiền tố khớp sâu nhất

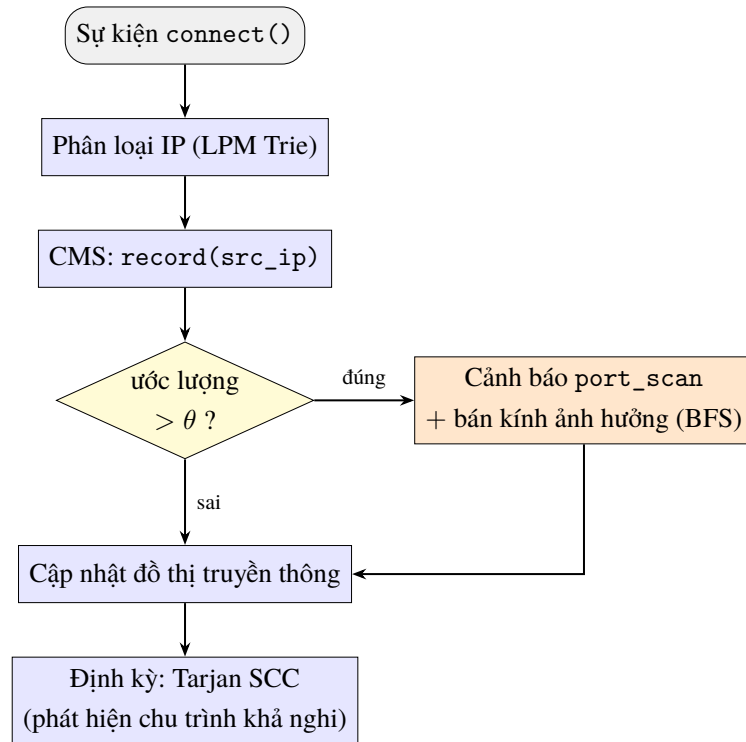


Tra cứu 10.244.0.5: duyệt theo từng bit tiền tố; nút khớp *sâu nhất* = 10.244.0.0/16 ⇒ nhãn **pod (1)** (tiền tố dài nhất thắng).

Hình 2.9. Ví dụ phân loại CIDR bằng LPM-Trie.

Bốn giải thuật trên không hoạt động rời rạc mà phối hợp trên cùng một dòng xử lý sự kiện. Hình 2.10 tóm tắt cách chúng kết hợp: mỗi sự kiện kết nối lần lượt được phân loại theo vùng địa chỉ (LPM Trie), được đếm tần suất theo nguồn (CMS) để kiểm tra ngưỡng quét cổng, đồng thời được dùng để bồi đắp đồ thị truyền thông — nơi giải thuật Tarjan và phép duyệt BFS chạy để phát hiện chu trình khả nghi và ước lượng bán kính ảnh hưởng.

Cách bố trí này cho phép phát hiện trực tuyến với chi phí thấp ngay khi sự kiện vừa được sinh ra.



Hình 2.10. Lưu đồ xử lý một sự kiện kết nối qua bốn giải thuật cốt lõi.

2.4. Nền tảng công nghệ sử dụng

Một đặc trưng của đề án là toàn bộ hệ thống được xây dựng trên một nền tảng (*platform*) điện toán đám mây gốc (*cloud-native*) thay vì chạy trực tiếp trên máy chủ vật lý theo cách truyền thống. Việc lựa chọn nền tảng này không chỉ là quyết định kỹ thuật phụ trợ mà còn quy định cách thức ứng dụng đặc thù của đề án vận hành, do đó mục này trình bày chi tiết từng tầng nền tảng và kết lại bằng một sơ đồ tổng thể (Hình 2.22) làm rõ điểm khác biệt so với các kiến trúc đi trước. Bảng 2.3 tóm tắt công dụng cụ thể của từng công cụ trong hệ thống.

Bảng 2.3. Các công cụ/công nghệ nền tảng và công dụng trong hệ thống.

Công cụ	Công dụng trong hệ thống
Kubernetes	Điều phối container: lập lịch, mở rộng, tự phục hồi; DaemonSet đặt bộ thu thập trên mọi nút
CNI Calico	Mạng pod và <i>thực thi</i> NetworkPolicy — nền cho vòng phản hồi cách ly
containerd	Runtime chạy container
Nhân Linux 6.8 + BTF	Cung cấp eBPF và thông tin kiểu BTF cho CO-RE
eBPF / libbpf	Thu thập sự kiện kết nối ở tầng nhân (kprobe tcp_v4_connect)
NATS JetStream	Trục thông điệp xuất bản—đăng ký giữa pipeline và các dịch vụ SOC
PostgreSQL	Lưu trữ bền vững sự cố, cảnh báo, hành động, nhật ký kiểm toán (ACID, JSONB)
Prometheus / Grafana	Thu thập số liệu và trực quan hóa cho người vận hành
ArgoCD (GitOps)	Đồng bộ trạng thái cụm về đúng khai báo trong Git (prune, self-heal)
Tekton	Tích hợp liên tục (CI) bằng pipeline chạy ngay trong cụm: xây, kiểm thử và đóng gói ảnh OCI
Harbor	Registry ảnh container riêng: lưu trữ, quét lỗ hổng và ký ảnh trước khi triển khai
clang/LLVM, bpftool	Biên dịch chương trình eBPF, sinh <code>vmLinux.h</code> , thao tác BPF

Phần dưới đây trình bày từng công nghệ theo cùng một mạch: *nó là gì, đồ án dùng nó làm gì, và vì sao chọn nó thay cho các giải pháp thay thế*. Các tầng nhân Linux, eBPF/libbpf và clang/LLVM/bpftool đã được phân tích sâu ở Mục 2.1 nên không nhắc lại ở đây.

2.4.1. Kubernetes — nền điều phối container



Hình 2.11. Logo Kubernetes.

Kubernetes là hệ điều phối (*orchestrator*) container mã nguồn mở, khởi nguồn từ Google và hiện do CNCF quản trị; nó tự động hóa việc lập lịch, mở rộng, tự phục hồi và quản lý vòng đời của các container trên một cụm nhiều nút. Đơn vị triển khai nhỏ nhất là *pod*, và mọi trạng thái mong muốn đều được mô tả khai báo bằng tệp YAML.

Trong đồ án, Kubernetes là nền chạy toàn bộ hệ thống: nó điều phối các dịch vụ SOC và pipeline phân tích như những *Deployment* co giãn, đồng thời — và đây là điểm mấu chốt — dùng đối tượng *DaemonSet* để bảo đảm bộ thu thập eBPF chạy đúng *một bản trên mỗi nút*, tự động theo cụm khi thêm/bớt nút. Nhờ vậy độ phủ quan sát tầng nhân là toàn cục mà không cần thao tác thủ công.

Sở dĩ chọn Kubernetes thay cho các công cụ điều phối nhẹ hơn như Docker Compose hay Docker Swarm là vì hai lý do đặc thù của bài toán: thứ nhất, chỉ Kubernetes có nguyên hình *DaemonSet* đúng ngữ nghĩa “một tiến trình nền trên mỗi nút” mà mô hình thu thập yêu cầu; thứ hai, Kubernetes định nghĩa chuẩn *NetworkPolicy* — nền tảng để khép kín vòng phản hồi cách ly ở Chương 3. Đây cũng là chuẩn thực tế của ngành, giúp đồ án bám sát môi trường triển khai thật.

2.4.2. CNI Calico — mạng pod và thực thi *NetworkPolicy*



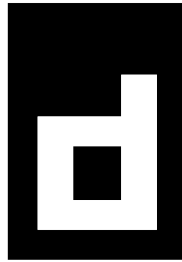
Hình 2.12. Logo Project Calico.

Calico là một trình cắm mạng (*Container Network Interface*, CNI) cung cấp khả năng kết nối mạng giữa các pod và, quan trọng hơn, *thực thi* chính sách mạng *NetworkPolicy*. Khác với một số CNI chỉ lo định tuyến gói, Calico hiện thực một bộ máy chính sách ở mỗi nút để cho phép hoặc chặn lưu lượng theo nhãn, không gian tên và dải địa chỉ.

Vai trò của Calico trong đồ án nằm ở *tầng phản hồi*: khi một nguồn bị xác định là đang quét cổng, hệ thống sinh ra một *NetworkPolicy* kiểu chặn lối ra (*egress-deny*) và Calico là thành phần thực sự áp đặt việc chặn đó trong mạng nội cụm. Nói cách khác, Calico biến một quyết định phát hiện thành một hành động ngăn chặn cụ thể — đúng tinh thần “phát hiện đi đôi với phòng ngừa” của tên đề tài.

Calico được chọn vì nó hỗ trợ đầy đủ và ổn định *NetworkPolicy* (kể cả chính sách egress), trong khi một số CNI phổ biến như Flannel không thực thi chính sách mạng. So với Cilium — một CNI dựa trên eBPF cũng rất mạnh — Calico cho mô hình chính sách rõ ràng, tài liệu trưởng thành và đủ cho phạm vi đồ án, đồng thời tránh chồng lấn với phần eBPF do chính đồ án tự hiện thực ở tầng thu thập.

2.4.3. containerd — runtime chạy container



Hình 2.13. Logo containerd.

containerd là runtime container (*container runtime*) cấp thấp, một dự án CNCF, đảm nhiệm việc kéo ảnh, quản lý vòng đời và chạy container theo chuẩn OCI. Trong cụm, nó nằm ngay dưới Kubernetes và là nơi thực sự khởi chạy các container của mọi pod, bao gồm cả pod thu thập eBPF (vốn cần quyền đặc thù để truy cập tầng nhân). Đồ án dùng containerd vì đây là runtime mặc định, gọn nhẹ và được Kubernetes hỗ trợ trực tiếp qua giao diện CRI, không cần lớp trung gian như Docker Engine.

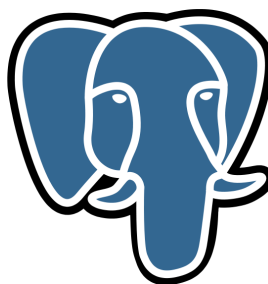
2.4.4. NATS JetStream — trực thông điệp xuất bản/đăng ký



Hình 2.14. Logo NATS.

NATS là một hệ thống truyền thông điệp (*messaging*) hiệu năng cao theo mô hình xuất bản–đăng ký (*publish/subscribe*); JetStream là lớp bổ sung khả năng lưu trữ bền vững và phát lại thông điệp. Trong kiến trúc của đồ án, NATS JetStream là *trục xương sống* nối tầng phân tích với các dịch vụ SOC: pipeline phát các cảnh báo lên các chủ đề (*subject*), còn aggregator, alert-bridge và incident-service đăng ký nhận. Cách ghép nối lỏng này tách rời bên sinh sự kiện khỏi bên tiêu thụ, cho phép mở rộng hoặc thay thế từng dịch vụ độc lập. NATS được ưu tiên hơn Kafka hay RabbitMQ vì nhẹ, độ trễ thấp và chi phí vận hành nhỏ — phù hợp với khối lượng cảnh báo của hệ thống mà không cần bộ máy nặng của Kafka.

2.4.5. PostgreSQL — cơ sở dữ liệu quan hệ



Hình 2.15. Logo PostgreSQL.

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, nổi tiếng về độ tin cậy và tuân thủ chuẩn. Đồ án dùng PostgreSQL làm kho lưu trữ bền vững cho sự cố (*incident*), cảnh báo, hành động phản hồi và *nhật ký kiểm toán* (*audit log*) — những dữ liệu đòi hỏi tính toàn vẹn cao. Hai đặc tính quyết định việc lựa chọn là: bảo đảm giao dịch ACID (cần thiết để nhật ký kiểm toán không bị mất hay mâu thuẫn) và kiểu dữ liệu JSONB cho phép lưu linh hoạt các trường ngữ cảnh bán cấu trúc (ví dụ thông tin pod đính kèm mỗi sự cố) mà vẫn truy vấn hiệu quả. So với các cơ sở dữ liệu tài liệu như MongoDB, PostgreSQL giữ được ràng buộc quan hệ giữa sự cố – cảnh báo – hành động; so với MySQL, nó hỗ trợ JSONB và chỉ mục biểu thức tốt hơn cho nhu cầu này.

2.4.6. Prometheus — thu thập số liệu



Hình 2.16. Logo Prometheus.

Prometheus là hệ thu thập số liệu (*metrics*) theo mô hình kéo (*pull*) và lưu trữ dưới dạng chuỗi thời gian, một dự án CNCF. Định kỳ nó truy vấn các điểm cuối `/metrics` mà mỗi dịch vụ phơi ra và lưu lại theo thời gian. Trong đồ án, Prometheus thu thập số liệu về tốc độ sự kiện, độ trễ xử lý và sức khỏe các dịch vụ SOC; chính những số liệu này về sau được dùng làm dữ liệu *đánh giá* định lượng ở Chương 5. Prometheus được chọn vì là chuẩn thực tế cho giám sát trong hệ sinh thái cloud-native, tích hợp sẵn với Kubernetes và có mô hình truy vấn (PromQL) mạnh mà vẫn nhẹ.

2.4.7. Grafana — trực quan hóa và bảng điều khiển



Hình 2.17. Logo Grafana.

Grafana là công cụ trực quan hóa dữ liệu thành các bảng điều khiển (*dashboard*) tương tác. Nó đọc số liệu từ Prometheus (và các nguồn khác) rồi hiển thị thành biểu đồ thời gian thực cho người vận hành. Trong hệ thống, Grafana là mặt giám sát trực quan: chuyên viên phân tích theo dõi tình hình cụm, tốc độ cảnh báo và sức khỏe dịch vụ qua các bảng do Grafana dựng. Grafana được chọn vì ghép cặp tự nhiên với Prometheus, hỗ trợ nhiều loại biểu đồ và phân quyền xem theo vai trò — phù hợp với nhu cầu của một bảng điều khiển an ninh.

2.4.8. Docker — đóng gói ảnh container



Hình 2.18. Logo Docker.

Docker là công cụ đóng gói ứng dụng cùng toàn bộ phụ thuộc thành một *ảnh* (*image*) bất biến theo chuẩn OCI, bảo đảm phần mềm chạy nhất quán giữa máy phát triển và cụm thật. Trong đề án, mỗi thành phần (collector, pipeline, sáu dịch vụ SOC) được đóng gói thành một ảnh Docker riêng; tính bất biến của ảnh là nền tảng cho khả năng tái lập của hệ thống. Docker được chọn vì là định dạng đóng gói phổ biến nhất, tương thích trực tiếp với runtime containerd của cụm.

2.4.9. Tekton — tích hợp liên tục gốc Kubernetes



Hình 2.19. Logo Tekton.

Tekton là khung tích hợp liên tục (CI) gốc *Kubernetes*: mỗi bước xây dựng, kiểm thử và đóng gói ảnh được mô tả khai báo và chạy ngay trong cụm dưới dạng pod. Trong đồ án, Tekton đảm nhiệm việc tự động xây và đóng gói ảnh OCI cho từng thành phần mỗi khi mã nguồn thay đổi. Tekton được chọn thay cho các dịch vụ CI bên ngoài (như GitHub Actions) vì nó chạy nội cụm, không phụ thuộc hạ tầng ngoài, giúp toàn bộ pipeline khép kín và kiểm toán được — phù hợp với một hệ thống an ninh đề cao tính tự chủ.

2.4.10. Harbor — kho ảnh container riêng



Hình 2.20. Logo Harbor.

Harbor là kho ảnh (*registry*) container mã nguồn mở, một dự án CNCF, bổ sung các tính năng an ninh chuỗi cung ứng so với một registry thông thường. Trong đồ án, Harbor là nơi lưu trữ các ảnh do Tekton tạo ra; trước khi một ảnh được phép triển khai, Harbor quét lỗ hổng và cho phép ký ảnh để bảo đảm tính toàn vẹn. Harbor được chọn thay cho một registry công cộng (như Docker Hub) vì nó là kho *riêng*, có kiểm soát truy cập và quét bảo mật — yêu cầu thiết yếu khi vận hành một hệ thống an ninh.

2.4.11. ArgoCD — triển khai theo mô hình GitOps



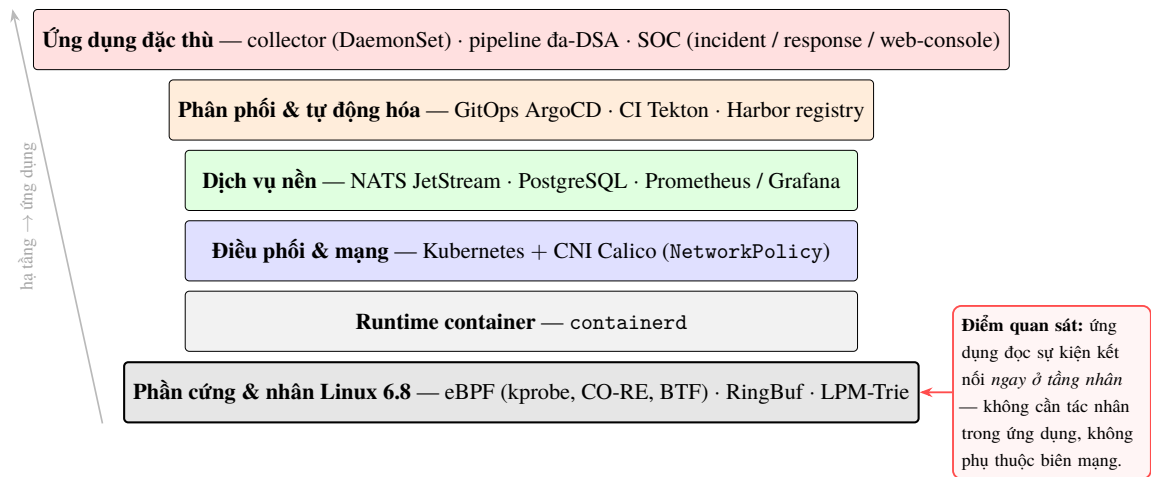
Hình 2.21. Logo Argo (ArgoCD).

ArgoCD là công cụ triển khai liên tục (CD) theo mô hình GitOps: trạng thái mong muốn của cụm được khai báo trong một kho Git, và ArgoCD liên tục đồng bộ cụm thật về đúng khai báo đó, tự dọn dẹp tài nguyên thừa và tự chữa lành khi có sai lệch. Trong đồ án, ArgoCD triển khai toàn bộ hệ thống từ các manifest trong Git, nên mọi thay đổi đều có dấu vết và có thể khôi phục. ArgoCD được chọn vì mô hình khai báo của nó làm cho hệ thống *tái lập được* và *kiểm toán được* — một yêu cầu phi chức năng quan trọng đã nêu ở Chương 1.

Ba công cụ Tekton, Harbor và ArgoCD tạo thành một chuỗi cung ứng phần mềm gốc *Kubernetes* khép kín trong cụm: Tekton xây ảnh, Harbor lưu và quét ảnh, ArgoCD triển khai theo Git. Kết hợp với mô hình ảnh bất biến của Docker, chuỗi này là cơ sở kỹ thuật cho tính tái lập và kiểm toán của toàn nền tảng.

2.4.12. Kiến trúc nền tảng tổng thể và khác biệt với cách tiếp cận truyền thống

Hình 2.22 tổng hợp toàn bộ các tầng nền tảng và đặt trên đó là ứng dụng đặc thù của đồ án (bộ thu thập, pipeline, các dịch vụ SOC). Điểm cốt lõi mà sơ đồ muốn làm nổi bật nằm ở mũi tên nhấn mạnh tầng nhân: ứng dụng quan sát lưu lượng *ngay tại tầng nhân* của mỗi nút, thay vì ở biên mạng hay bên trong mã ứng dụng.



Hình 2.22. Sơ đồ nền tảng tổng thể: các tầng từ nhân Linux tới ứng dụng đặc thù.

Sự khác biệt so với các kiến trúc truyền thống có thể nhìn trên bốn trục. Thứ nhất, về *vị trí quan sát*: an ninh chu vi truyền thống đặt thiết bị giám sát ở biên mạng (hướng Bắc – Nam) và gần như không thấy lưu lượng nội bộ giữa các dịch vụ, trong khi nền tảng này quan sát toàn bộ lưu lượng Đông – Tây ngay ở tầng nhân, đúng tinh thần Zero-Trust [15]. Thứ hai, về *cách thu thập*: thay vì cài tác nhân vào từng ứng dụng (xâm lấn, phụ thuộc ngôn ngữ) hay phản chiếu gói toàn mạng (tốn kém), eBPF cho phép thu thập không xâm lấn và độ phủ toàn nút chỉ với một DaemonSet. Thứ ba, về *mô hình vận hành*: hạ tầng truyền thống thường cấu hình thủ công và trạng thái trôi dạt theo thời gian, còn nền tảng này khai báo bằng GitOps với ảnh bất biến nên tái lập được và kiểm toán được. Thứ tư, về *vòng phản hồi*: thay vì phân tích theo lô rồi xử lý sự cố thủ công như các hệ SIEM cổ điển, hệ thống phát hiện trực tuyến bằng cấu trúc dữ liệu luồng và khép kín vòng phản hồi bằng NetworkPolicy có người duyệt. Chính bốn khác biệt này tạo nên giá trị của việc xây dựng ứng dụng an ninh *trên* một nền tảng cloud-native hiện đại, và là tiền đề cho phần thiết kế hệ thống ở Chương 3.

2.5. Kết luận chương

Chương này đã thiết lập đầy đủ nền tảng lý thuyết và công nghệ cho phần còn lại của đề án. Bốn trụ lý thuyết được lựa chọn không phải ngẫu nhiên mà bám sát đúng đường đi của một sự kiện trong hệ thống: sự kiện kết nối được *thu thập* ở tầng nhân bằng eBPF (Mục 2.1), được *đếm tần suất* bằng Count-Min Sketch để phát hiện quét cổng (Mục 2.2), rồi được *phân tích quan hệ* bằng các giải thuật đồ thị Tarjan và BFS, đồng thời *phân loại* theo vùng địa chỉ bằng LPM-Trie (Mục 2.3). Với mỗi giải thuật, chương đã trình bày ý tưởng, công thức, mã giả và ví dụ chạy tay; riêng phần ước lượng tần suất chỉ nêu lại kết quả cô đọng, còn toàn bộ chứng minh đầy đủ (giới hạn lỗi, cận dưới không gian, so sánh Misra–Gries và Count Sketch) đã được phát triển chi tiết trong Báo cáo học phần tốt nghiệp (CNPM 1) và được tham chiếu khi cần.

Bên cạnh các giải thuật, chương cũng đã giới thiệu từng công nghệ nền tảng — từ Kubernetes, Calico, containerd đến NATS, PostgreSQL, Prometheus/Grafana và chuỗi cung ứng phần mềm — cùng lý do lựa chọn, và kết lại bằng một bức tranh tổng thể về nền tảng cùng bốn điểm khác biệt so với các kiến trúc truyền thống. Những cơ sở này là tiền đề trực tiếp cho việc phân tích và thiết kế hệ thống ở Chương 3.

CHƯƠNG 3

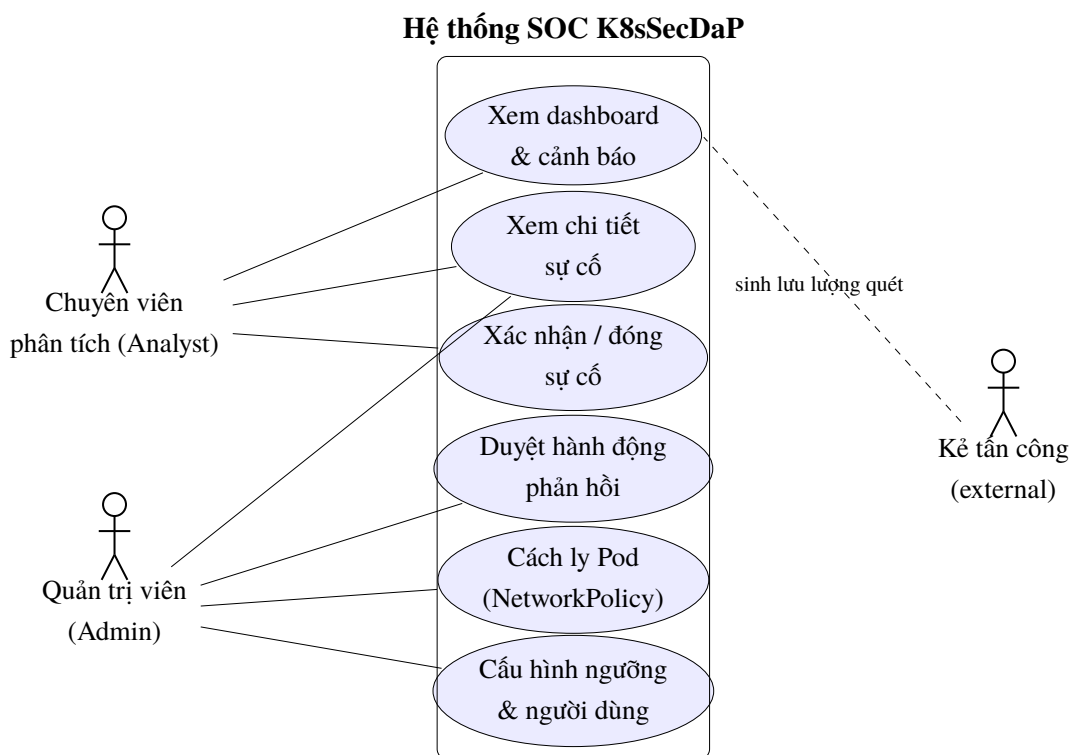
PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này đi từ mô hình mối đe dọa và các yêu cầu suy ra từ nó, qua kiến trúc phần mềm và luồng dữ liệu, tới thiết kế cơ sở dữ liệu, quy trình phản hồi và thiết kế giao diện vận hành. Mục tiêu là làm rõ *tại sao* hệ thống được tổ chức như vậy trước khi trình bày cách hiện thực ở Chương 4.

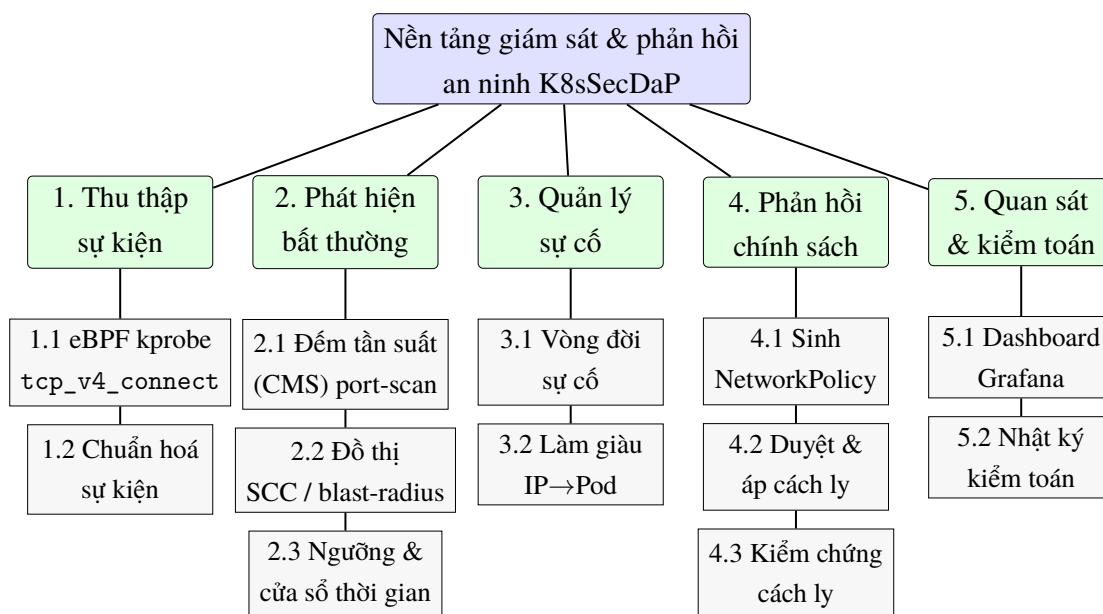
3.1. Mô hình mối đe dọa và yêu cầu hệ thống

Thiết kế của hệ thống được dẫn dắt bởi một mô hình mối đe dọa cụ thể thay vì các giả định chung chung. *Tài sản* cần bảo vệ là các pod và dịch vụ trong namespace mục tiêu của cụm. *Kẻ tấn công* được giả định là một pod đã bị chiếm quyền điều khiển bên trong cụm — tình huống hoàn toàn thực tế khi một dịch vụ phơi ra Internet bị khai thác — và từ chỗ đứng đó kẻ tấn công tiến hành quét cổng theo chiều dọc (nhiều cổng trên một đích) và theo chiều ngang (một cổng trên nhiều đích) nhằm lập bản đồ mạng nội bộ, rồi dịch chuyển ngang tới các pod khác. *Khả năng quan sát* của hệ thống dựa trên một sự thật kiến trúc: mọi kết nối TCP của mọi pod đều phải đi qua nhân Linux của nút chứa nó, do đó một bộ thu thập eBPF đặt ở tầng nhân quan sát được trọn vẹn lưu lượng mà không cần thay đổi ứng dụng. Mô hình giả định CNI có hỗ trợ NetworkPolicy, nhân có BTF, lưu lượng là IPv4, và kẻ tấn công chỉ có quyền bên trong pod chứ không có quyền root trên nút; các tình huống nằm ngoài mô hình — tấn công ở tầng ứng dụng (L7), khai thác lỗ hổng nhân, hay kẻ tấn công đã thoát khỏi container — được ghi nhận tường minh là giới hạn phạm vi.

Từ mô hình trên, các tác nhân và chức năng của hệ thống được xác định. Người vận hành an ninh (*analyst*) tương tác với hệ thống để theo dõi cảnh báo, điều tra sự cố và phê duyệt hành động phản hồi; hệ thống tự động đảm nhiệm việc thu thập, phát hiện, tạo sự cố và sinh bản nháp chính sách. Hình 3.1 mô tả các trường hợp sử dụng (*use case*) chính theo các tác nhân, còn Hình 3.2 phân rã hệ thống thành các nhóm chức năng. Từ đó, các *yêu cầu chức năng* cốt lõi gồm: thu thập sự kiện kết nối ở tầng nhân; phát hiện quét cổng và dịch chuyển ngang theo thời gian thực; tạo và làm giàu sự cố; sinh chính sách cách ly theo cơ chế có người duyệt; và cung cấp giao diện theo dõi/xử lý. Các *yêu cầu phi chức năng* quan trọng gồm: chi phí thu thập đủ thấp để chạy thường trực trên mọi nút, độ trễ phát hiện→phản hồi ở mức giây, và khả năng tái lập trên cụm thật — những tiêu chí này được kiểm chứng định lượng ở Chương 5.



Hình 3.1. Sơ đồ use-case của hệ thống.



Hình 3.2. Sơ đồ phân rã chức năng (BFD) của nền tảng giám sát an ninh.

Bảng 3.1 đặc tả từng chức năng lá của sơ đồ phân rã — nêu rõ nhiệm vụ cụ thể của mỗi nút lá, làm cầu nối giữa phân rã chức năng và thiết kế thành phần ở các mục sau.

Bảng 3.1. Đặc tả các chức năng lá của sơ đồ phân rã chức năng (BFD).

Mã	Chức năng lá	Nhiệm vụ
1.1	eBPF kprobe tcp_v4_connect	Bắt mọi lời gọi connect() TCP/IPv4 ở tầng nhân trên mỗi nút
1.2	Chuẩn hoá sự kiện	Trích src/dst/cổng/pid/thời gian, lọc theo nhãn vùng mạng, xuất JSON
2.1	Đếm tần suất (CMS)	Cập nhật Count-Min Sketch theo địa chỉ nguồn, ước lượng số kết nối
2.2	Đồ thị SCC / blast-radius	Dựng đồ thị giao tiếp, chạy Tarjan và BFS để tìm chu trình và bán kính
2.3	Ngưỡng & cửa sổ thời gian	So sánh tần suất với ngưỡng trên cửa sổ trượt, quyết định phát cảnh báo
3.1	Vòng đời sự cố	Tạo, gộp trùng, cập nhật trạng thái (new/ack/resolved) của sự cố
3.2	Làm giàu IP→Pod	Ánh xạ địa chỉ nguồn về tên pod và namespace qua API Kubernetes
4.1	Sinh NetworkPolicy	Tạo bản nháp chính sách egress-deny gắn nhãn lên pod tấn công
4.2	Duyệt & áp cách ly	Trình bản nháp cho người duyệt; khi đồng ý thì áp qua API Kubernetes
4.3	Kiểm chứng cách ly	Xác minh pod tấn công mất kết nối đi ra sau khi áp chính sách
5.1	Dashboard Grafana	Trực quan hóa số liệu và cảnh báo theo thời gian thực cho người vận hành
5.2	Nhật ký kiểm toán	Ghi lại mọi thao tác trên sự cố/hành động phục vụ điều tra và tuân thủ

3.1.1. Đặc tả use-case

Bảng 3.2 liệt kê các use-case của hệ thống cùng tác nhân tương ứng. Tác nhân *Chuyên viên phân tích* (Analyst) và *Quản trị viên* (Admin) là người dùng; *Hệ thống* đại diện cho các chức năng tự động (thu thập, phát hiện, sinh sự cố) chạy không cần can thiệp. Sau bảng tổng hợp là đặc tả chi tiết của các use-case quan trọng nhất.

Bảng 3.2. Danh sách use-case và tác nhân.

Mã	Use-case	Tác nhân
UC-01	Xem dashboard và cảnh báo	Analyst, Admin
UC-02	Điều tra chi tiết sự cố	Analyst, Admin
UC-03	Ghi nhận / đóng sự cố	Analyst, Admin
UC-04	Duyệt và áp hành động phản hồi	Admin
UC-05	Cấu hình ngưỡng phát hiện và người dùng	Admin
UC-06	Tra cứu nhật ký kiểm toán	Analyst, Admin
UC-S1	Thu thập, phát hiện và sinh sự cố (tự động)	Hệ thống

Bảng 3.3. Đặc tả use-case UC-02 — Điều tra chi tiết sự cố.

Mã / Tên	UC-02 — Điều tra chi tiết sự cố
Tác nhân	Chuyên viên phân tích (Analyst)
Mô tả	Người vận hành mở một sự cố để xem ngữ cảnh đã làm giàu, các cảnh báo liên quan và bán kính ảnh hưởng nhằm đánh giá mức độ nghiêm trọng.
Tiền điều kiện	Đã đăng nhập; tồn tại ít nhất một sự cố trong hệ thống.
Luồng chính	- Analyst chọn một sự cố từ danh sách. - Hệ thống truy vấn cơ sở dữ liệu lấy chi tiết sự cố. - Hệ thống hiển thị siêu dữ liệu (địa chỉ nguồn, pod, nút, mức độ, nhãn). - Hệ thống hiển thị danh sách cảnh báo liên quan và đồ thị bán kính ảnh hưởng. - Analyst xem và đánh giá ngữ cảnh sự cố.
Luồng thay thế	2a. Không tìm thấy sự cố: hệ thống báo lỗi và quay lại danh sách.
Hậu điều kiện	Sự cố được hiển thị đầy đủ; sẵn sàng cho thao tác ghi nhận/đóng hoặc duyệt phản hồi.

Bảng 3.4. Đặc tả use-case UC-04 — Duyệt và áp hành động phản hồi.

Mã / Tên	UC-04 — Duyệt và áp hành động phản hồi
Tác nhân	Quản trị viên (Admin)
Mô tả	Admin xem xét bản nháp NetworkPolicy cách ly do hệ thống sinh ra và quyết định phê duyệt hay từ chối; khi phê duyệt, chính sách được áp vào cụm.
Tiền điều kiện	Đã đăng nhập với quyền Admin; sự cố có một bản nháp hành động ở trạng thái chờ duyệt.
Luồng chính	1. Admin mở sự cố và chọn hành động phản hồi chờ duyệt. 2. Hệ thống hiển thị xem trước nội dung NetworkPolicy sẽ áp. 3. Admin nhập lý do và bấm Phê duyệt. 4. Hệ thống áp chính sách qua API Kubernetes (Calico thực thi). 5. Hệ thống xác minh hiệu lực và cập nhật trạng thái hành động. 6. Hệ thống ghi toàn bộ thao tác vào nhật ký kiểm toán.
Luồng thay thế	3a. Admin từ chối: hệ thống đánh dấu từ chối, ghi kiểm toán, không áp chính sách. 4a. Lỗi áp chính sách: hệ thống ghi lỗi, giữ trạng thái chờ và thông báo.
Hậu điều kiện	Pod tấn công bị cách ly (nếu phê duyệt); mọi quyết định đều có dấu vết kiểm toán.

Bảng 3.5. Đặc tả use-case UC-05 — Cấu hình ngưỡng phát hiện và người dùng.

Mã / Tên	UC-05 — Cấu hình ngưỡng phát hiện và người dùng
Tác nhân	Quản trị viên (Admin)
Mô tả	Admin điều chỉnh tham số phát hiện (ngưỡng, độ dài cửa sổ, kích thước sketch) và quản lý tài khoản/vai trò người dùng.
Tiền điều kiện	Đã đăng nhập với quyền Admin.
Luồng chính	1. Admin mở trang cấu hình. 2. Admin sửa tham số phát hiện hoặc thông tin người dùng/vai trò. 3. Hệ thống kiểm tra hợp lệ và lưu cấu hình. 4. Hệ thống áp dụng cấu hình mới cho pipeline và ghi kiểm toán.
Luồng thay thế	3a. Dữ liệu không hợp lệ: hệ thống báo lỗi và yêu cầu nhập lại.
Hậu điều kiện	Cấu hình mới có hiệu lực; thay đổi được lưu vết.

Bảng 3.6. Đặc tả use-case UC-01 — Xem dashboard và cảnh báo.

Mã / Tên	UC-01 — Xem dashboard và cảnh báo
Tác nhân	Chuyên viên phân tích (Analyst), Quản trị viên (Admin)
Mô tả	Người vận hành theo dõi tình hình an ninh nội cụm qua bảng điều khiển Grafana và danh sách cảnh báo/sự cố mới nhất.
Tiền điều kiện	Đã đăng nhập; hệ thống đang thu thập và phát hiện.
Luồng chính	1. Người vận hành mở trang tổng quan. 2. Hệ thống truy vấn số liệu (Prometheus) và danh sách sự cố mới. 3. Hệ thống hiển thị các biểu đồ thời gian thực và bảng hành động chờ duyệt. 4. Người vận hành nắm bắt tình hình và chọn mục cần điều tra.
Luồng thay thế	2a. Nguồn số liệu tạm gián đoạn: hiển thị cảnh báo thiếu dữ liệu.
Hậu điều kiện	Bức tranh tổng quan được hiển thị; sẵn sàng cho điều tra chi tiết (UC-02).

Bảng 3.7. Đặc tả use-case UC-03 — Ghi nhận / đóng sự cố.

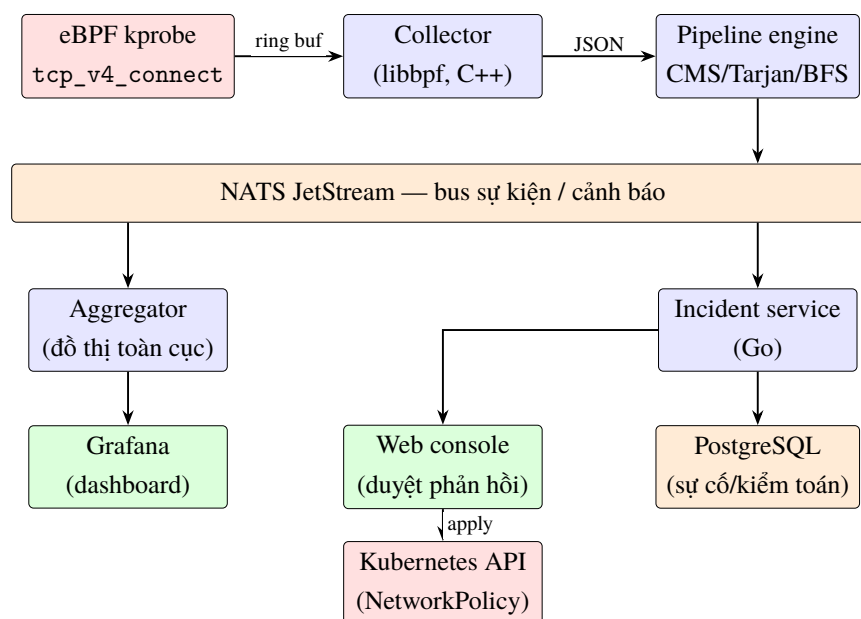
Mã / Tên	UC-03 — Ghi nhận / đóng sự cố
Tác nhân	Chuyên viên phân tích (Analyst)
Mô tả	Sau khi điều tra, người vận hành ghi nhận (<i>ack</i>) sự cố đang xử lý hoặc đóng (<i>resolve</i>) sự cố đã giải quyết.
Tiền điều kiện	Đã đăng nhập; sự cố tồn tại và đang ở trạng thái mở.
Luồng chính	1. Người vận hành mở chi tiết sự cố. 2. Người vận hành chọn “Ack” hoặc “Resolve”. 3. Hệ thống cập nhật trạng thái sự cố trong cơ sở dữ liệu. 4. Hệ thống ghi thao tác vào nhật ký kiểm toán.
Luồng thay thế	3a. Sự cố đã đổi trạng thái bởi người khác: hệ thống thông báo và làm mới.
Hậu điều kiện	Trạng thái sự cố được cập nhật; thao tác có dấu vết kiểm toán.

3.2. Kiến trúc phần mềm và luồng dữ liệu

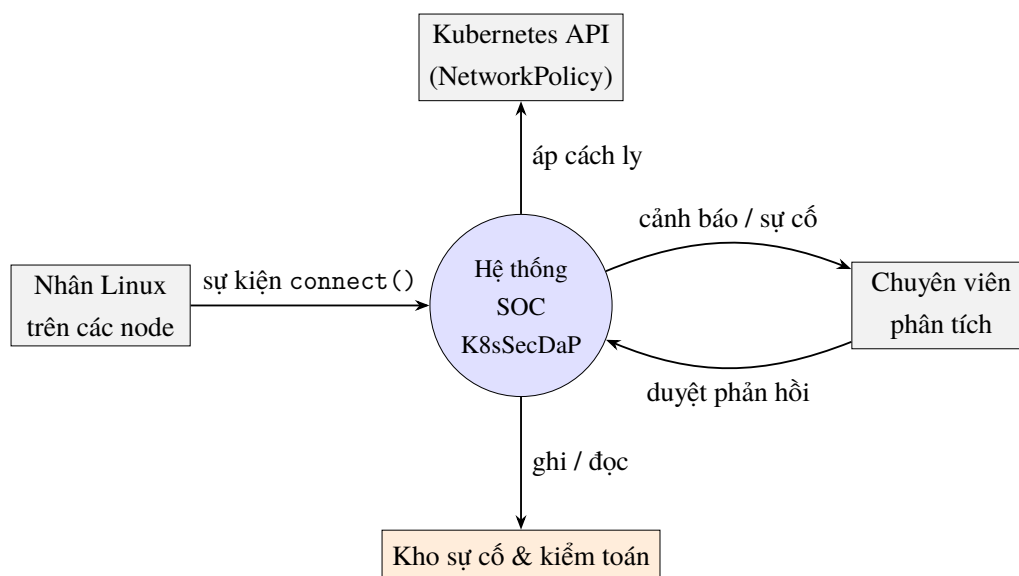
Hệ thống được tổ chức theo kiến trúc phân lớp, mỗi lớp một trách nhiệm rõ ràng và giao tiếp với nhau qua các giao diện ổn định. Hình 3.3 trình bày kiến trúc ở mức container (theo mô hình C4): ở mỗi nút, một bộ thu thập eBPF chạy dưới dạng DaemonSet bắt sự kiện kết nối và đẩy lên bộ máy pipeline qua bộ đệm vòng; bộ máy pipeline áp các giải thuật phát hiện và phát cảnh báo lên trục thông điệp NATS; phía trung tâm, bộ tổng hợp (*aggregator*) tương quan và khử trùng lặp cảnh báo trên phạm vi toàn cụm, dịch vụ sự cố tạo và làm giàu sự cố rồi lưu vào PostgreSQL, còn bộ sinh phản hồi tạo bản nháp

NetworkPolicy; cuối cùng, người vận hành theo dõi qua Grafana và thao tác qua console web. Cách phân tách này cho phép thay thế hoặc nâng cấp từng thành phần độc lập, đồng thời cô lập phần chạy đặc quyền (bộ thu thập tầng nhân) khỏi phần logic nghiệp vụ.

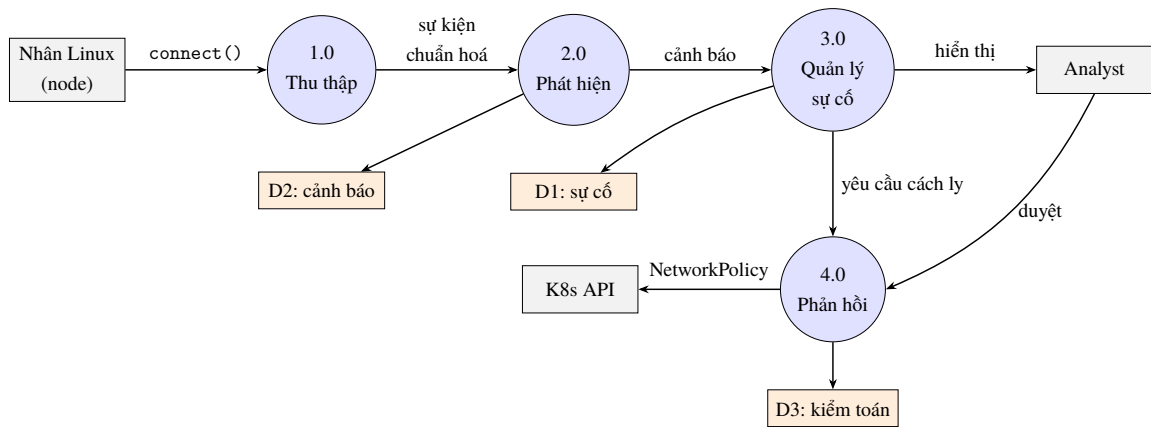
Luồng dữ liệu xuyên suốt hệ thống được mô tả ở hai mức. Hình 3.4 cho cái nhìn ngữ cảnh: dữ liệu khởi nguồn từ lưu lượng kết nối trong nhân, chảy qua hệ thống xử lý và kết thúc ở người vận hành dưới dạng cảnh báo, sự cố và hành động phản hồi. Hình 3.5 phân rã chi tiết hơn các tiến trình bên trong và các kho dữ liệu mà chúng đọc/ghi. Riêng tuyến đường “trên mỗi nút → xử lý tập trung” — đặc trưng của mô hình DaemonSet — được làm rõ ở Hình 3.7: phần chạy phân tán trên từng nút chỉ đảm nhiệm thu thập và phát hiện cục bộ, còn việc hợp nhất, lưu trữ và hiển thị được tập trung hóa.



Hình 3.3. Kiến trúc mức container (mô hình C4).

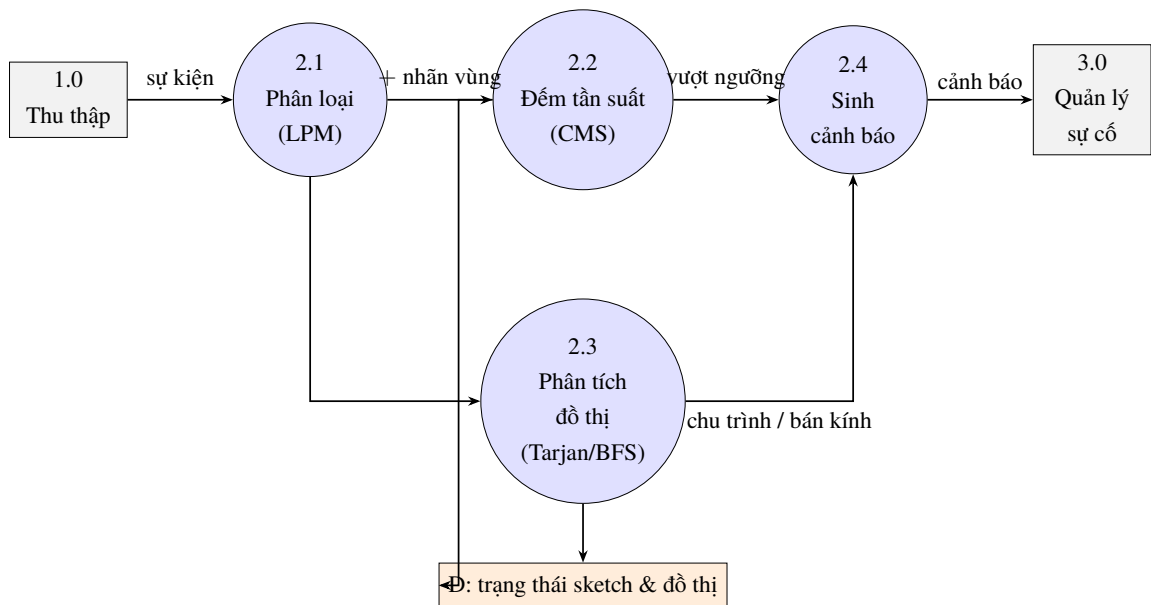


Hình 3.4. Sơ đồ luồng dữ liệu mức ngữ cảnh (DFD level 0).



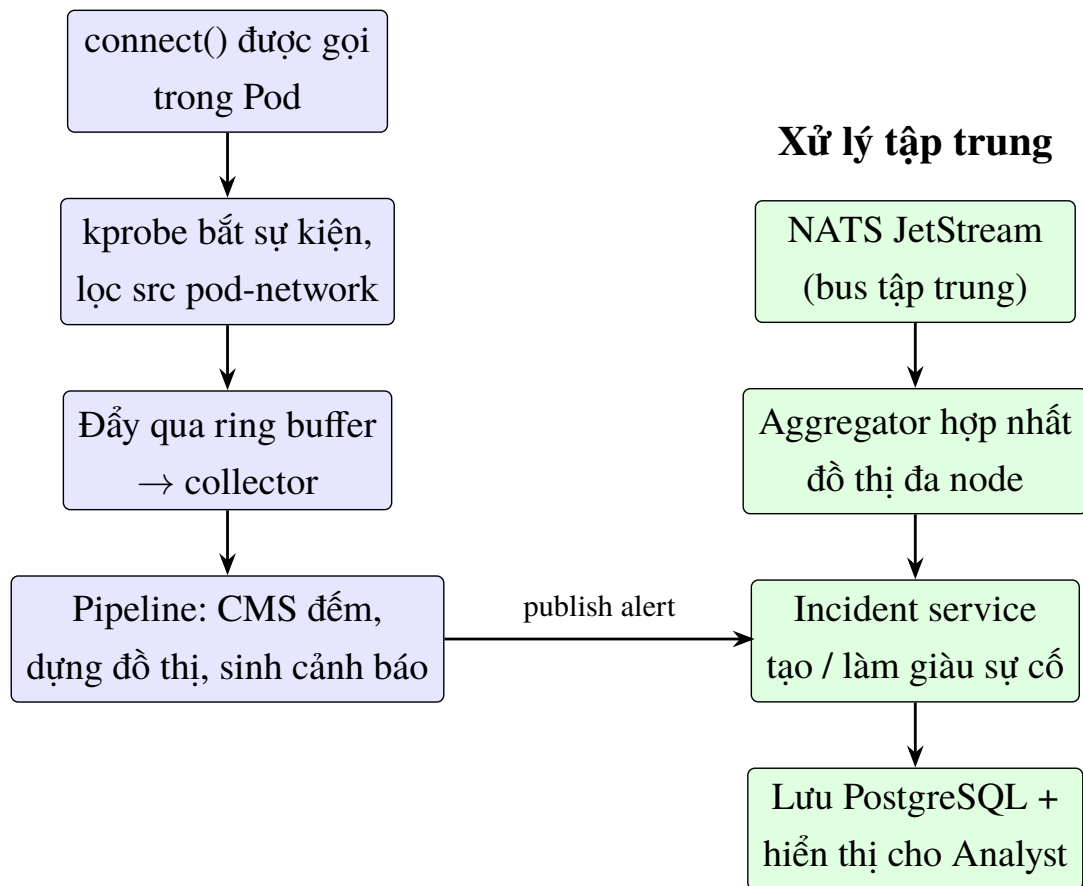
Hình 3.5. Sơ đồ luồng dữ liệu mức 1.

Tiến trình phát hiện (2.0) — phần lõi thuật toán của hệ thống — được phân rã thêm một mức ở Hình 3.6: sự kiện sau khi phân loại địa chỉ (2.1) vừa được đếm tần suất bằng Count-Min Sketch (2.2) vừa được dùng để phân tích đồ thị bằng Tarjan/BFS (2.3); cả hai nhánh đọc/ghi trạng thái sketch và đồ thị, rồi hội tụ ở bước sinh cảnh báo (2.4).



Hình 3.6. Sơ đồ luồng dữ liệu mức 2 (tiến trình phát hiện).

Trên mỗi node — DaemonSet

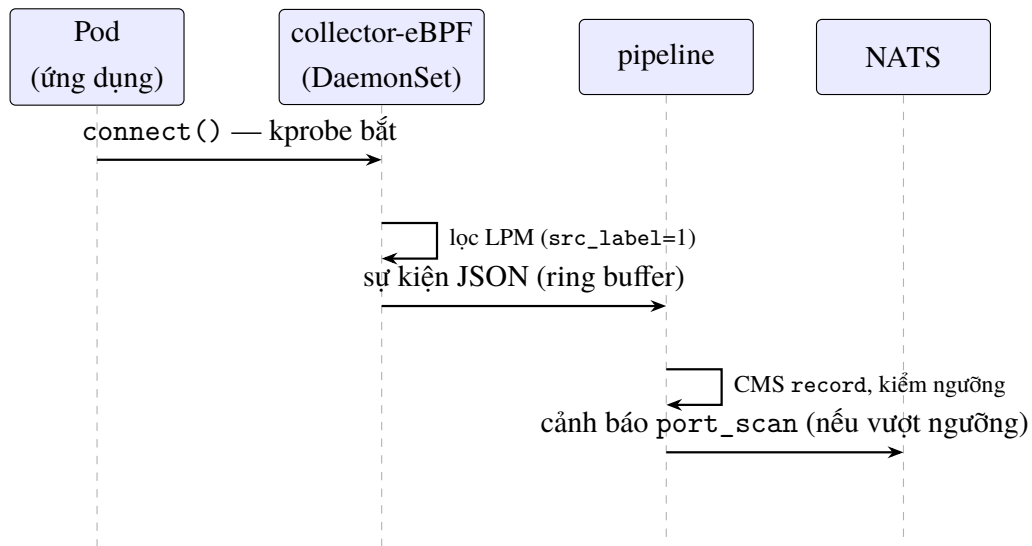


Hình 3.7. Luồng xử lý phân tán trên mỗi nút (DaemonSet) và phần xử lý tập trung.

Một quyết định thiết kế quan trọng ở tầng pipeline là áp dụng mẫu thiết kế Strategy kết hợp Registry [7]: mỗi họ giải thuật (ước lượng tần suất, phát hiện thành phần liên thông mạnh, tính khả-đạt, phân loại địa chỉ) được trừu tượng hóa thành một giao diện chung và đăng ký vào một sổ đăng ký; việc lựa chọn cài đặt cụ thể được điều khiển hoàn toàn bằng tệp cấu hình YAML. Nhờ đó, có thể thay Count-Min Sketch bằng một biến thể khác, hoặc đổi tham số cửa sổ và ngưỡng, mà không phải sửa và biên dịch lại mã nguồn — một đặc tính có giá trị cho cả việc thử nghiệm lẫn vận hành.

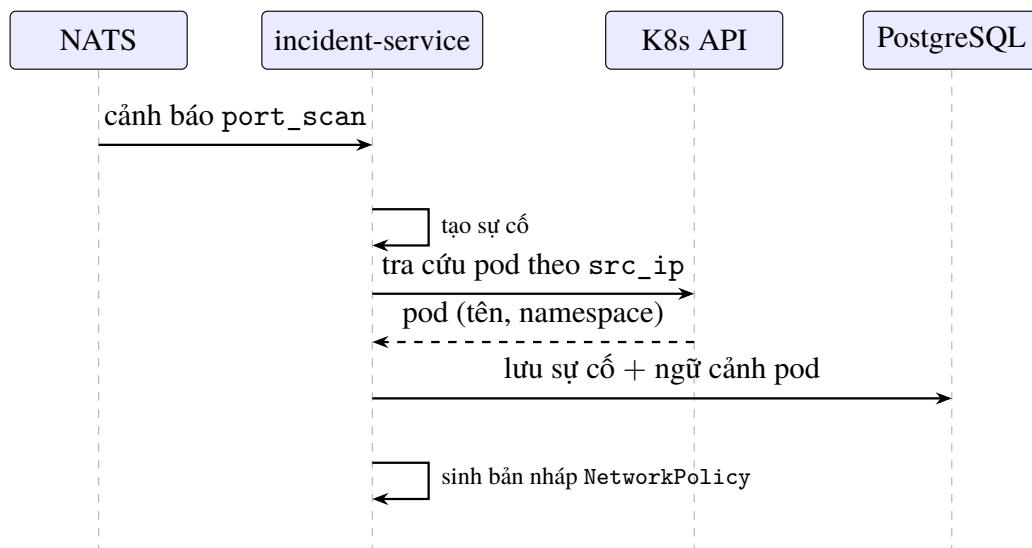
3.2.1. Luồng gọi giữa các dịch vụ

Để làm rõ trình tự tương tác giữa các thành phần, ba sơ đồ tuần tự (*sequence diagram*) dưới đây mô tả ba kịch bản chính. Hình 3.8 là luồng *thu thập*: một lời gọi `connect()` trong pod bị kprobe bắt, bộ thu thập lọc theo nhãn vùng mạng rồi đẩy sự kiện sang pipeline; pipeline cập nhật Count-Min Sketch và phát cảnh báo khi vượt ngưỡng.



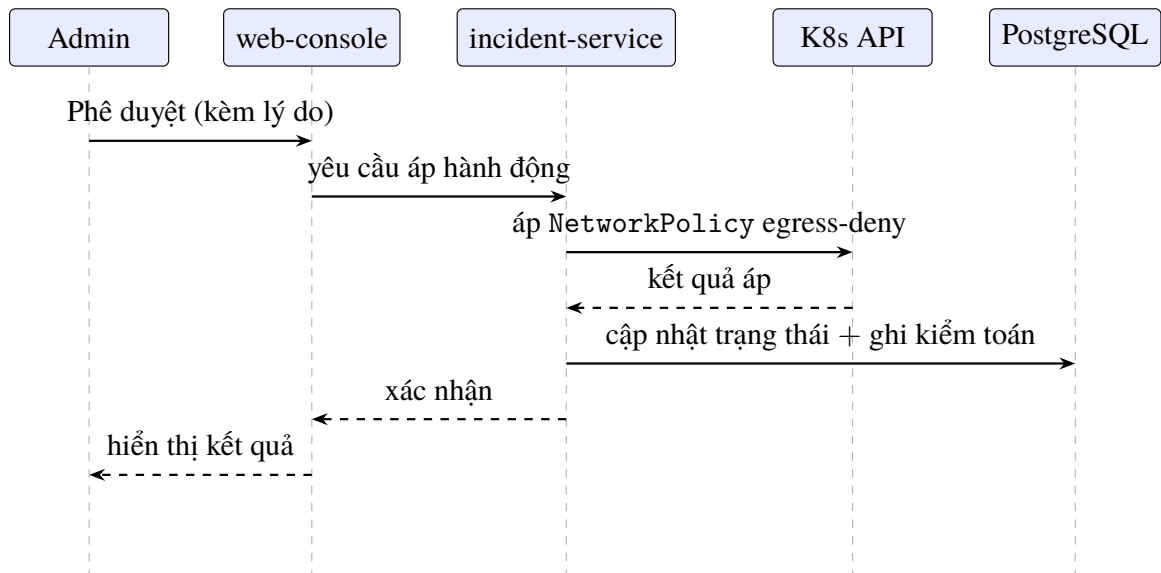
Hình 3.8. Sơ đồ tuần tự: thu thập sự kiện và phát hiện quét cổng.

Hình 3.9 là luồng *tạo và làm giàu sự cố*: dịch vụ sự cố nhận cảnh báo từ NATS, tạo sự cố, tra cứu API Kubernetes để ánh xạ địa chỉ nguồn về đúng pod, lưu sự cố kèm ngưỡng cảnh vào PostgreSQL và sinh bản nháp chính sách cách ly.



Hình 3.9. Sơ đồ tuần tự: tạo và làm giàu sự cố.

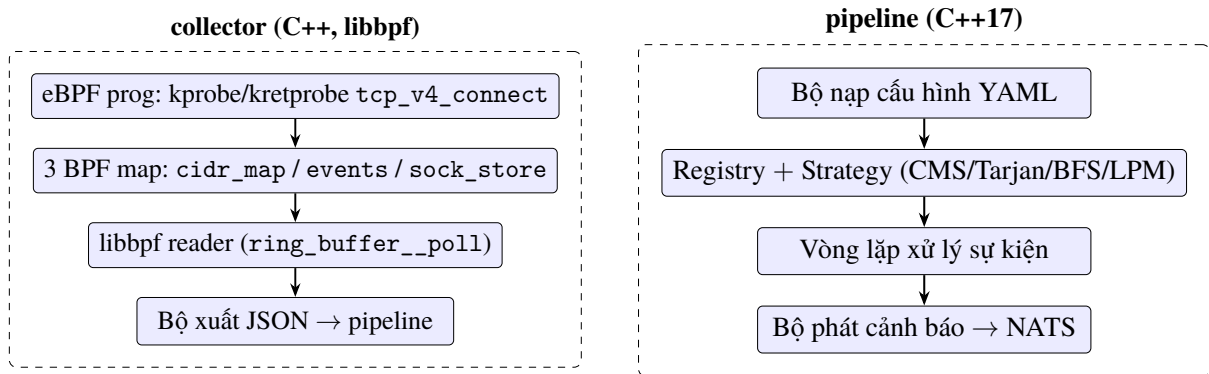
Hình 3.10 là luồng *phê duyệt và áp phản hồi*: quản trị viên phê duyệt qua console, dịch vụ sự cố áp NetworkPolicy qua API Kubernetes, cập nhật trạng thái và ghi nhật ký kiểm toán trước khi phản hồi kết quả về giao diện.



Hình 3.10. Sơ đồ tuần tự: phê duyệt và áp hành động phản hồi.

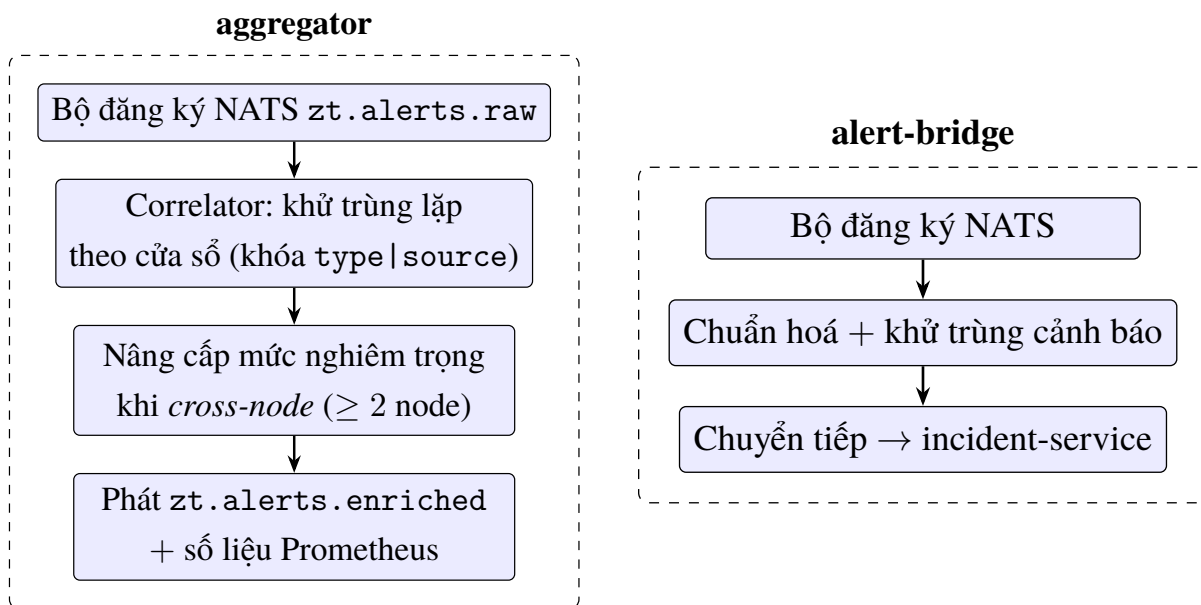
3.2.2. Thiết kế chi tiết từng thành phần

Mỗi thành phần trong kiến trúc được phân rã thành các mô-đun nội bộ với trách nhiệm rõ ràng. Bộ thu thập (Hình 3.11, trái) gồm chương trình eBPF gắn ở tầng nhân, ba BPF map, bộ đọc libbpf và bộ xuất JSON; pipeline (phải) nạp cấu hình YAML, dựng các giải thuật qua Registry, chạy vòng lặp xử lý và phát cảnh báo.



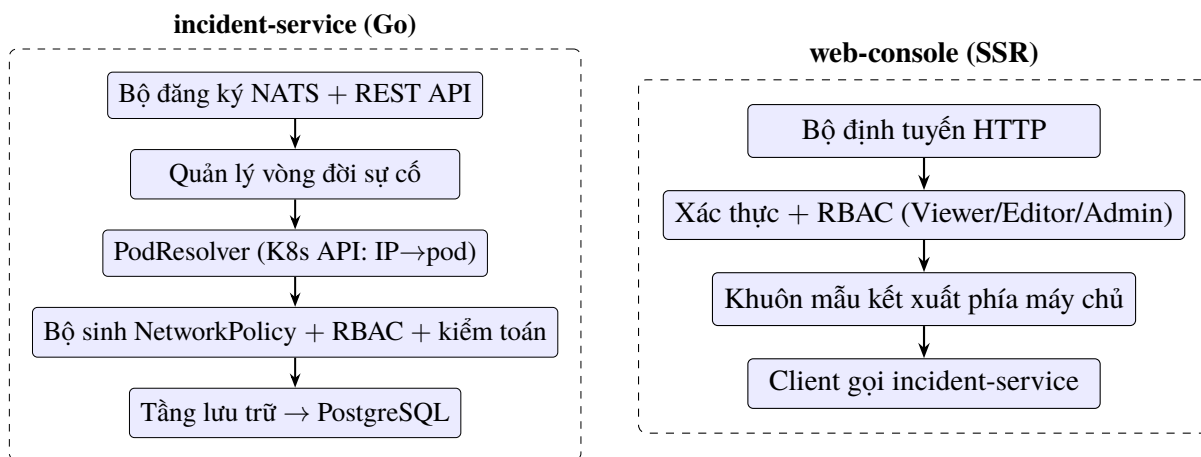
Hình 3.11. Thiết kế thành phần: collector (trái) và pipeline (phải).

Phía trung tâm, bộ tổng hợp (Hình 3.12, trái) tương quan và khử trùng lặp cảnh báo theo cửa sổ thời gian (khóa type|source) và nâng cấp mức nghiêm trọng khi cùng một nguồn xuất hiện trên nhiều node (*cross-node*); còn phân tích đồ thị (Tarjan/BFS tính bán kính ảnh hưởng) chạy ngay trong pipeline mỗi node. Bộ cầu nối cảnh báo (phải) chuẩn hoá và chuyển tiếp cảnh báo tới dịch vụ sự cố.



Hình 3.12. Thiết kế thành phần: aggregator (trái) và alert-bridge (phải).

Dịch vụ sự cố (Hình 3.13, trái) là thành phần phức tạp nhất: nó quản lý vòng đời sự cố, ánh xạ địa chỉ về pod, sinh NetworkPolicy, áp RBAC và ghi kiểm toán, đồng thời lưu trữ vào PostgreSQL; console web (phải) kết xuất giao diện phía máy chủ và gọi dịch vụ sự cố qua API.



Hình 3.13. Thiết kế thành phần: incident-service (trái) và web-console (phải).

3.2.3. Đặc tả phần mềm theo mô-đun

Bảng 3.8 đặc tả tóm tắt từng mô-đun theo ba trục đầu vào — xử lý — đầu ra, giúp xác định rõ ranh giới trách nhiệm và giao diện giữa các thành phần trước khi hiện thực.

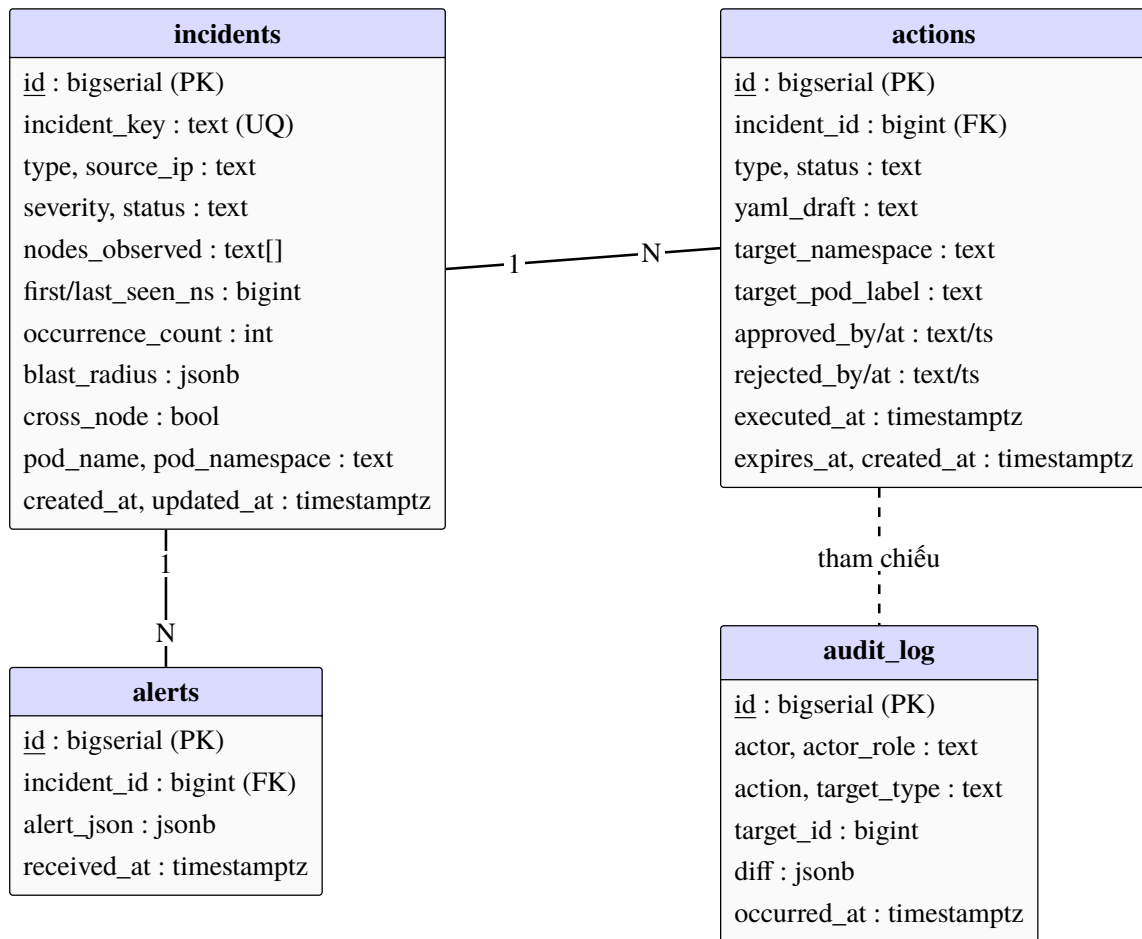
Bảng 3.8. Đặc tả phần mềm theo mô-đun (đầu vào / xử lý / đầu ra).

Mô-đun	Đầu vào	Xử lý	Đầu ra
collector	Sự kiện connect() ở nhân	Lọc LPM, đọc ring buffer	Dòng JSON sự kiện
pipeline	Sự kiện JSON, cấu hình YAML	CMS, kiểm ngưỡng, đồ thị + Tarjan/BFS	Cảnh báo (NDJSON)
alert-bridge	Tệp alerts.json (NDJSON)	Theo dõi đuôi tệp, publish	Cảnh báo lên <code>zt.alerts.raw</code>
aggregator	Cảnh báo thô từ NATS	Khử trùng lặp theo cửa sổ, nâng cấp cross-node	Cảnh báo đã tương quan
incident-service	Cảnh báo, yêu cầu API	Tạo/làm giàu sự cố, sinh policy, RBAC	Sự cố, bản nháp policy, kiểm toán
web-console	Thao tác người dùng (HTTP)	Xác thực, RBAC, kết xuất SSR	Trang HTML, lời gọi API

3.3. Thiết kế cơ sở dữ liệu

Trung tâm vận hành an ninh cần một kho lưu trữ bền vững, nhất quán cho sự cố, cảnh báo, hành động phản hồi và nhật ký kiểm toán. Hệ thống chọn PostgreSQL vì ba lý do bám sát yêu cầu: thứ nhất, các bảo đảm giao dịch ACID giữ cho vòng đời sự cố (tạo, làm giàu, phê duyệt, đóng) luôn ở trạng thái nhất quán ngay cả khi nhiều dịch vụ ghi đồng thời; thứ hai, kiểu dữ liệu JSONB cùng chỉ mục GIN cho phép lưu và truy vấn hiệu quả các trường bán cấu trúc như chi tiết cảnh báo hay danh sách nhãn, vốn rất biến động; thứ ba, PostgreSQL tích hợp tốt với Kubernetes qua StatefulSet và cung cấp công cụ phân tích kế hoạch truy vấn (EXPLAIN ANALYZE) phục vụ tối ưu hóa.

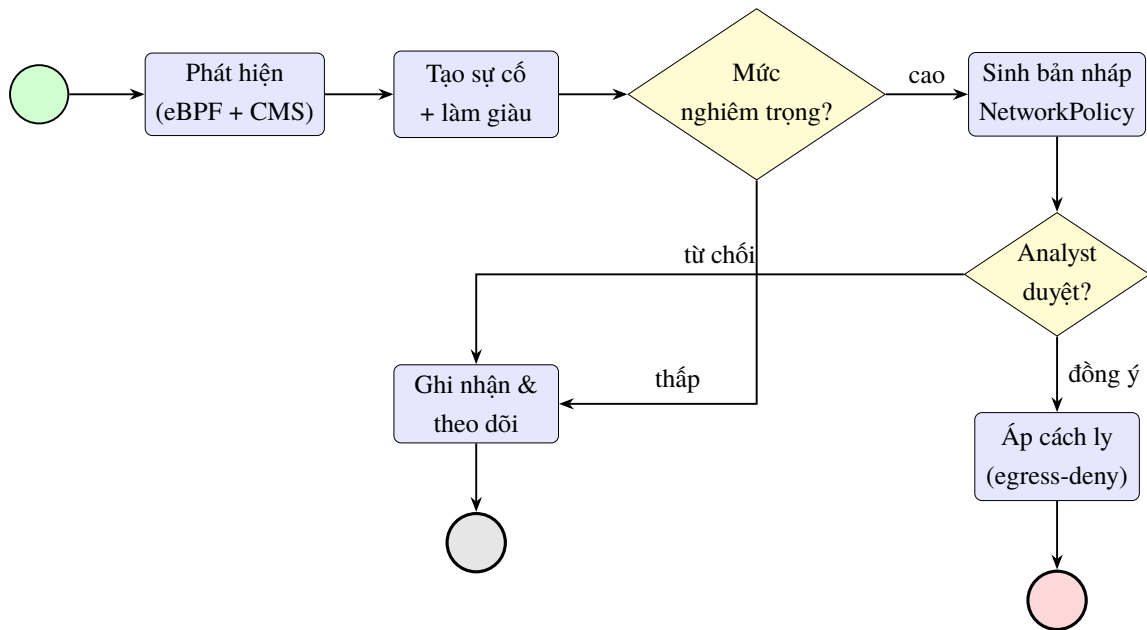
Hình 3.14 trình bày sơ đồ thực thể – quan hệ của lược đồ. Các thực thể trung tâm gồm incidents (sự cố, khóa theo nguồn và có các trường ngữ cảnh pod được làm giàu), alerts (cảnh báo do pipeline phát ra), actions (hành động phản hồi, ví dụ bản nháp NetworkPolicy và trạng thái phê duyệt) và audit_log (nhật ký mọi thao tác phục vụ truy vết). Lược đồ được chuẩn hóa tối dạng chuẩn 3 để loại bỏ dư thừa và bảo đảm toàn vẹn tham chiếu qua các khóa ngoại từ cảnh báo và hành động trở về sự cố. Về chiến lược chỉ mục, các trường thời gian được đánh chỉ mục B-tree để hỗ trợ truy vấn theo khoảng và sắp xếp theo thời gian, trong khi các trường nhãn và chi tiết dạng JSONB dùng chỉ mục GIN; nhờ đó các truy vấn vận hành thường gặp — liệt kê sự cố mới nhất, đếm theo mức độ, truy vết theo người dùng trong một khoảng thời gian — đều thực thi hiệu quả. Định nghĩa bảng đầy đủ (DDL) được quản lý bằng các tệp di trú (*migration*) nhúng kèm dịch vụ sự cố và được trích dẫn trong phần phụ lục.



Hình 3.14. Sơ đồ thực thể – quan hệ (ERD) của cơ sở dữ liệu SOC.

3.4. Thiết kế quy trình phản hồi và giao diện vận hành

Khác với nhiều công cụ tự động áp đặt chính sách, hệ thống của đề án thiết kế vòng phản hồi theo cơ chế *có người duyệt (generate-for-review)* nhằm tránh rủi ro một cảnh báo sai dẫn tới việc cách ly nhầm dịch vụ hợp lệ. Hình 3.15 mô tả quy trình nghiệp vụ: khi một nguồn vượt ngưỡng, hệ thống tạo sự cố và sinh bản nháp NetworkPolicy cách ly; bản nháp này được trình cho người vận hành xem xét và chỉ được áp dụng vào cụm sau khi được phê duyệt, kèm ghi nhật ký kiểm toán đầy đủ. Thiết kế này đặt con người vào đúng điểm ra quyết định quan trọng, đồng thời vẫn giữ được tốc độ nhờ phần máy đã chuẩn bị sẵn mọi dữ kiện.



Hình 3.15. Quy trình nghiệp vụ phát hiện → phê duyệt → cách ly (BPMN).

Giao diện vận hành được thiết kế ở *mức bố cục thành phần* (mockup) trong mục này; ảnh chụp ứng dụng chạy thật được trình bày ở Chương 4. Giao diện phục vụ ba vai trò với quyền hạn tăng dần: *Viewer* chỉ xem dashboard và sự cố, *Editor* có thể ghi nhận (*ack*) và đóng sự cố, còn *Admin* được phê duyệt hành động phản hồi và quản trị hệ thống. Về mặt kiến trúc trình bày, hệ thống dùng hai bề mặt hỗ trợ nhau: Grafana cung cấp các dashboard kỹ thuật chỉ-đọc cho đội trực vận hành, còn một console web kết xuất phía máy chủ phục vụ thao tác xử lý sự cố — lựa chọn kết xuất phía máy chủ giúp giao diện đơn giản, ít mã JavaScript và dễ kiểm toán hơn so với một ứng dụng đơn trang.

Về bố cục, console gồm *ba màn hình chính*, kết xuất phía máy chủ (Flask/Jinja2) với cập nhật cục bộ bằng HTMX. Thanh tiêu đề “ZT-SOC Admin Console” và thanh điều hướng (*Incidents, Reports*) xuất hiện ở mọi trang. Các bản phác (*wireframe*) dưới đây trình bày ở mức bố cục thành phần — đúng theo mã nguồn của console — chứ không phải giao diện chạy thật (ảnh chụp thật ở Chương 4).

Màn hình *Incidents* (Hình 3.16) gồm hai bảng: “Pending actions” liệt kê các hành động phản hồi đang chờ duyệt với hai nút *Approve/Reject* ngay trên mỗi dòng (gửi bằng HTMX, cập nhật tại chỗ), và “Incidents” liệt kê các sự cố theo các cột mã, loại, địa chỉ nguồn, mức độ, trạng thái và thời điểm xuất hiện, kèm liên kết xem chi tiết.

ZT-SOC Admin Console
Incidents Reports

Pending actions (2)

ID	Incident	Type	Status	Created	Decide
					Approve Reject
					Approve Reject

Incidents (12)

ID	Type	Source IP	Severity	Status	First seen	→
	port_scan	10.244.x.y	high	new	...	view →
	port_scan	10.244.x.y	high	new	...	view →
	port_scan	10.244.x.y	high	new	...	view →
	port_scan	10.244.x.y	high	new	...	view →
	port_scan	10.244.x.y	high	new	...	view →

Hình 3.16. Wireframe màn hình Incidents (bảng hành động chờ duyệt + bảng sự cố).

Màn hình *chi tiết sự cố* (Hình 3.17) là trung tâm thao tác: bảng siêu dữ liệu (địa chỉ nguồn, mức độ, trạng thái, các nút quan sát được, cờ liên-nút, số lần lặp, thời điểm đầu/cuối), bảng *Actions* với nút duyệt, biểu mẫu *Layer-2 follow-up* để lên lịch thao tác pod (delete_pod/shutdown_pod kèm namespace, tên pod, lý do), và *nhật ký kiểm toán* nhúng ngay trong trang.

ZT-SOC Admin Console
Incidents Reports

← back to incidents

Incident #1042 — port_scan

Source IP : 10.244.2.31
Severity : high
Status : new
Nodes observed : worker-1
Cross-node : false
Occurrence count : 47
First / Last seen : ...

Actions

ID	Type	Status	Created	Decide
				Approve Reject

Layer-2 follow-up

Operation : [delete_pod ∨]
Namespace : [zt-targets]
Pod name : [attacker]
Reason : [...]

Schedule

Audit log

Time	Actor (role)	Action	Diff

Hình 3.17. Wireframe màn hình chi tiết sự cố.

Màn hình *Reports* (Hình 3.18) tổng hợp báo cáo theo tuần: số sự cố, trung vị thời gian phản hồi (MTTR), số sự kiện kiểm toán; phân tích theo mức độ, nguồn tấn công nhiều nhất và loại hành động; cuối trang là biểu mẫu xuất nhật ký kiểm toán phục vụ tuân thủ (định dạng CSV/JSON).

ZT-SOC Admin Console Incidents Reports

Weekly report — 2026-W23

ISO week (YYYY-WW): [...] [View](#) [Generate & upload \(MinIO\)](#)

Headline

Window : From → To
 Total incidents : N
 MTTR median (s) : ...
 Audit events : ...

Severity breakdown

Severity | Count
 critical | ...
 high | ...
 medium | ...

Top attackers

Source IP | Count
 10.244.x.y | ...

Actions breakdown

Type | Count
 quarantine_pod | ...

Compliance / forensic audit dump

From [date] To [date] Actor [...] Format [CSV ▾] [Download](#)

Hình 3.18. Wireframe màn hình Reports (báo cáo tuần + xuất nhật ký kiểm toán).

3.5. Kết luận chương

Chương này đã chuyển các yêu cầu và bài toán nêu ở Chương 1 thành một thiết kế hệ thống cụ thể. Bắt đầu từ phân tích yêu cầu và mô hình mối đe dọa, chương đã trình bày sơ đồ use-case cùng sơ đồ phân rã chức năng, rồi đặc tả kiến trúc phần mềm qua sơ đồ mức container và các sơ đồ luồng dữ liệu, làm rõ vai trò của từng dịch vụ và đường đi của dữ liệu trong hệ thống. Phần thiết kế cơ sở dữ liệu xác định các thực thể và quan hệ phục vụ lưu vết sự cố, cảnh báo, hành động và nhật ký kiểm toán. Cuối cùng, chương đặc tả quy trình phản hồi có người duyệt và thiết kế giao diện vận hành ở mức bố cục thành phần. Toàn bộ thiết kế này là bản vẽ trực tiếp cho phần cài đặt và triển khai ở Chương 4.

CHƯƠNG 4

CÀI ĐẶT, TRIỂN KHAI VÀ THỬ NGHIỆM HỆ THỐNG

Chương này trình bày *toàn bộ* những gì đã được hiện thực và vận hành: từ hạ tầng nền tảng, qua ba thành phần lõi của vòng phát hiện – phản hồi (bộ thu thập eBPF, bộ máy pipeline và trung tâm vận hành an ninh – SOC), tới ứng dụng giao diện cho người vận hành, và cuối cùng là việc chạy thử nghiệm end-to-end trên cụm Kubernetes thật. Theo đúng trọng tâm của đề tài, phần trình bày *đào sâu* vào bộ thu thập, pipeline và SOC – nơi đặt thuật toán phát hiện và logic phản hồi – kèm sơ đồ luồng, mã giả và trích đoạn mã nguồn/cấu hình thật; phần hạ tầng và triển khai chỉ được *giới thiệu vắn tắt* nhưng vẫn liệt kê đủ thành phần. Những đoạn mã dài hơn được đưa vào phần Phụ lục. Phần đánh giá *định lượng* (tỷ lệ phát hiện, FPR, chi phí, độ trễ) tách riêng sang Chương 5; còn thiết kế kiến trúc/thành phần (sơ đồ) đã trình bày ở Chương 3 và sẽ được tham chiếu lại khi cần.

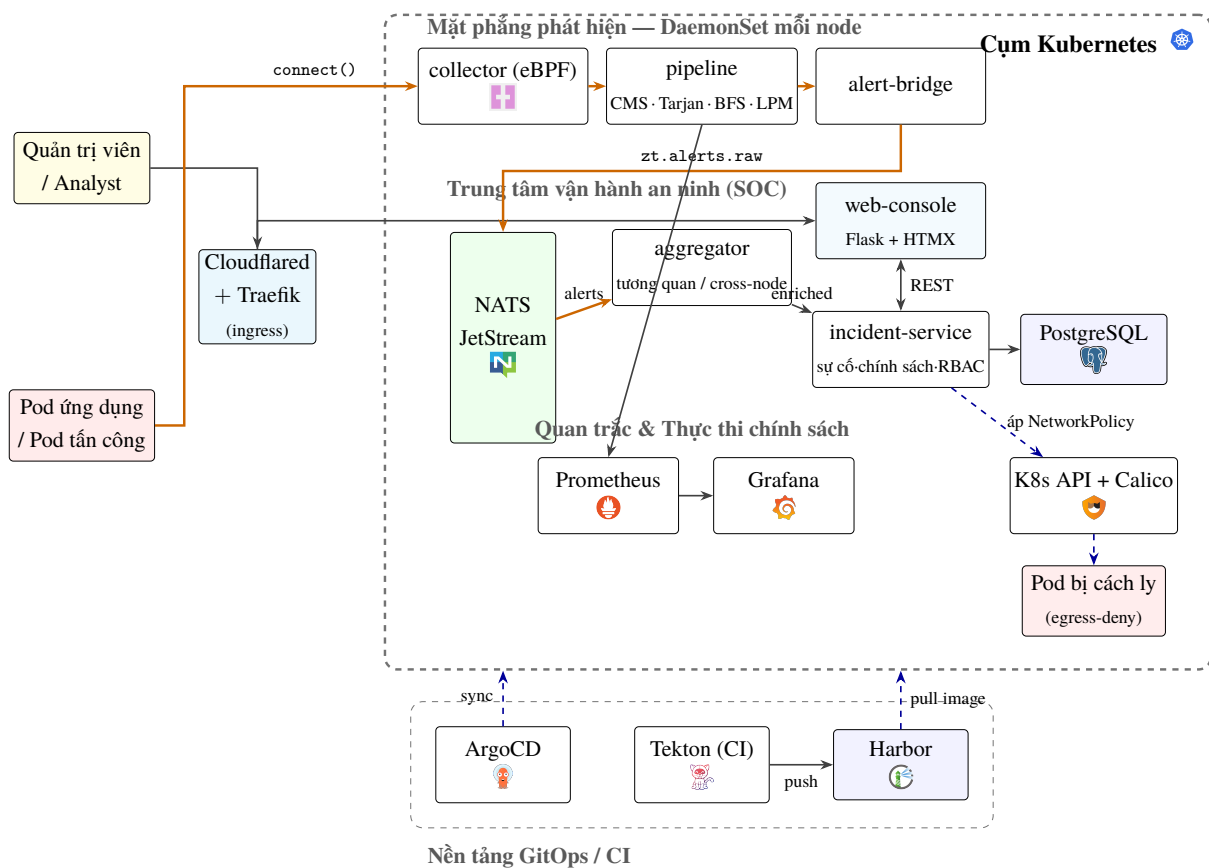
4.1. Tổng quan hiện thực

Hệ thống được tổ chức thành nhiều kho mã nguồn độc lập, mỗi kho phụ trách một tầng và được đóng gói thành ảnh container riêng để xây dựng, kiểm thử, nâng cấp tách bạch. Bảng 4.1 ánh xạ mỗi thành phần tới kho mã, ngôn ngữ, đối tượng Kubernetes và tệp mã nguồn cốt lõi; đây cũng là bản đồ tra cứu cho toàn bộ chương.

Bảng 4.1. Bản đồ hiện thực: thành phần → kho mã, ngôn ngữ, đóng gói và tệp cốt lõi.

Thành phần	Kho mã	Ngôn ngữ	Đối tượng K8s	Tệp cốt lõi
Bộ thu thập eBPF	collector	C (nhân) + C++	DaemonSet (đặc quyền)	tcp_connect.bpf.c, collector.cpp
Bộ máy pipeline	pipeline	C++17	DaemonSet (sidecar)	engine.cpp, libdsa/
Cầu nối cảnh báo	soc	Go	DaemonSet (sidecar)	alert-bridge/main.go
Bộ tổng hợp	soc	Go	Deployment	aggregator/correlator.go
Dịch vụ sự cố	soc	Go	Deployment	incident-service/internal/*
Console web	soc	Python (Flask)	Deployment	web-console/app.py
Lưu trữ	soc	SQL	StatefulSet	store/migrations/*.sql
Hạ tầng & triển khai	infra, deploy	Ansible, YAML	ArgoCD, Tekton	ansible/, argocd/

Luồng dữ liệu xuyên suốt các thành phần này như sau: lời gọi `connect()` của một pod → bộ thu thập eBPF (tầng nhân) → pipeline (phát hiện) → cảnh báo qua NATS → bộ tổng hợp (khử trùng lặp, tương quan) → dịch vụ sự cố (tạo sự cố, sinh chính sách cách ly) → người vận hành duyệt trên console → áp `NetworkPolicy` vào cụm. Hình 4.1 cho cái nhìn tổng quát về toàn bộ kiến trúc: mặt phẳng phát hiện chạy phân tán trên mỗi node (DaemonSet), trung tâm SOC xử lý tập trung, tầng quan trắc và thực thi chính sách, cùng nền tảng GitOps/CI ở dưới. Các mục tiếp theo trình bày lần lượt từng mắt xích.



Hình 4.1. Sơ đồ tổng quát kiến trúc hệ thống.

4.2. Môi trường và công cụ triển khai

Hệ thống được hiện thực và kiểm thử trên một cụm Kubernetes thật gồm ba node (một node control-plane và hai node worker), chạy hệ điều hành Ubuntu 24.04 với nhân Linux phiên bản 6.8 có sẵn BTF, runtime container containerd và mạng pod do CNI Calico đảm nhiệm (có hỗ trợ NetworkPolicy). Mỗi tầng được hiện thực bằng ngôn ngữ phù hợp với ràng buộc của tầng đó: chương trình tầng nhân viết bằng C và biên dịch sang đối tượng eBPF bằng clang/LLVM; bộ đọc và bộ máy pipeline viết bằng C++17 để kiểm soát chặt bộ nhớ và hiệu năng; các dịch vụ SOC viết bằng Go; console web viết bằng Python (Flask). Mỗi thành phần được đóng gói thành ảnh OCI riêng và nạp vào cụm qua đối tượng Kubernetes tương ứng (DaemonSet, Deployment, StatefulSet). Việc tách bạch ngôn ngữ và đóng gói theo tầng giúp cô lập phần chạy đặc quyền ở tầng nhân khỏi phần logic nghiệp vụ.

Tôi dựng cụm tự động bằng Ansible (kho infra): mỗi node được khởi tạo bằng kubeadm, cài containerd, rồi CNI Calico được áp qua role cni_calico. Sau khi dựng xong, tôi kiểm chứng môi trường bằng `kubectl get nodes` (Mã nguồn 4.1): cả ba node ở trạng thái Ready, cùng nhân Linux 6.8 và runtime containerd — xác nhận điều kiện chạy eBPF CO-RE (nhân ≥ 5.8 , có BTF) đã được thỏa mãn đồng nhất trên cụm.

```
1 $ kubectl get nodes -o wide
```

2	NAME	STATUS	ROLES	VERSION	KERNEL -
	VERSION	CONTAINER-RUNTIME			
3	userver-master	Ready	control-plane	v1.32.13	6.8.0-
	generic	containerd://2.2.4			
4	userver-worker-1	Ready	<none>	v1.32.13	6.8.0-
	generic	containerd://2.2.4			
5	userver-worker-home	Ready	<none>	v1.32.13	
	6.8.0-generic	containerd://2.2.4			

Mã nguồn 4.1. Kiểm chứng cụm sau khi dựng bằng Ansible (kết quả kubectl rút gọn).

4.3. Hạ tầng nền tảng và triển khai (giới thiệu)

Phần này được trình bày vắn tắt vì không thuộc trọng tâm đề tài, nhưng vẫn nêu đủ các thành phần để bức tranh triển khai hoàn chỉnh. Toàn bộ trạng thái mong muốn của hệ thống được quản lý theo mô hình GitOps: manifest Kubernetes lưu trong kho Git và một ứng dụng ArgoCD (zt-soc-core) liên tục đồng bộ trạng thái cụm về đúng trạng thái khai báo, với hai tùy chọn *prune* và *self-heal* bật để tự dọn tài nguyên thừa và tự khôi phục khi có sai lệch. Quy trình tích hợp liên tục do pipeline *Tekton* chạy ngay trong cụm đảm nhiệm: clone mã nguồn, dựng ảnh bằng Kaniko, đẩy lên kho ảnh riêng (*Harbor*), rồi cập nhật thể ảnh trong manifest để ArgoCD triển khai. Hạ tầng node (cài đặt kubeadm, containerd, CNI Calico) được tự động hóa bằng Ansible trong kho *infra*.

Cụ thể, gốc của cây GitOps là một Application kiểu “app-of-apps”: nó trở vào thư mục *platform/argocd/apps* và sinh ra một Application con cho mỗi tệp, trong đó thứ tự triển khai được kiểm soát bằng *sync-wave* để CNI luôn tồn tại trước khi pod SOC xin địa chỉ pod-network (Mã nguồn 4.2). Nhờ *selfHeal*, mọi sai lệch thủ công đều bị kéo về đúng trạng thái khai báo trong Git.

```

1 # app-of-apps.yaml: gốc, sinh 1 Application cho mọi tệp
   trong apps/
2 kind: Application
3 metadata: { name: platform-apps, namespace: argocd }
4 spec:
5   source: { repoURL: ...K8sSecDaP-deploy.git, path:
       platform/argocd/apps }
6   syncPolicy: { automated: { prune: true, selfHeal: true }
       }
7 ---
8 # Thu tu trien khai bang sync-wave (CNI truoc, SOC sau, lop
   du lieu cuoi):
9 #   zt-cni-calico   sync-wave: "-10"   (prune/selfHeal=

```

```

false - KHONG bao gio prune CNI)
10 #   zt-soc-core      sync-wave: "0"      -> manifests/ của
      K8sSecDaP-soc
11 #   zt-soc-data     sync-wave: "10"     -> data/ (MinIO,
      cronjob bao cao tuan)

```

Mã nguồn 4.2. Gốc GitOps “app-of-apps” và thứ tự sync-wave của các ứng dụng con (platform/argocd/).

Quy trình CI là một Tekton Pipeline tên build-and-deploy gồm bốn Task tuần tự: git-clone → kaniko-build (dựng ảnh OCI không cần Docker daemon) → đẩy ảnh lên kho riêng → update-manifest (vá thể ảnh vào manifest GitOps, commit để ArgoCD đồng bộ). Như vậy một thay đổi mã nguồn được đưa ra cụm hoàn toàn tự động và có thể truy vết qua Git.

Để minh chứng các thành phần này thực sự đang chạy, tôi truy vấn trạng thái cụm (Mã nguồn 4.3): các Application của ArgoCD đều Synced/Healthy, và DaemonSet thu thập zt-pipeline-ebpf hiện diện trên các node worker.

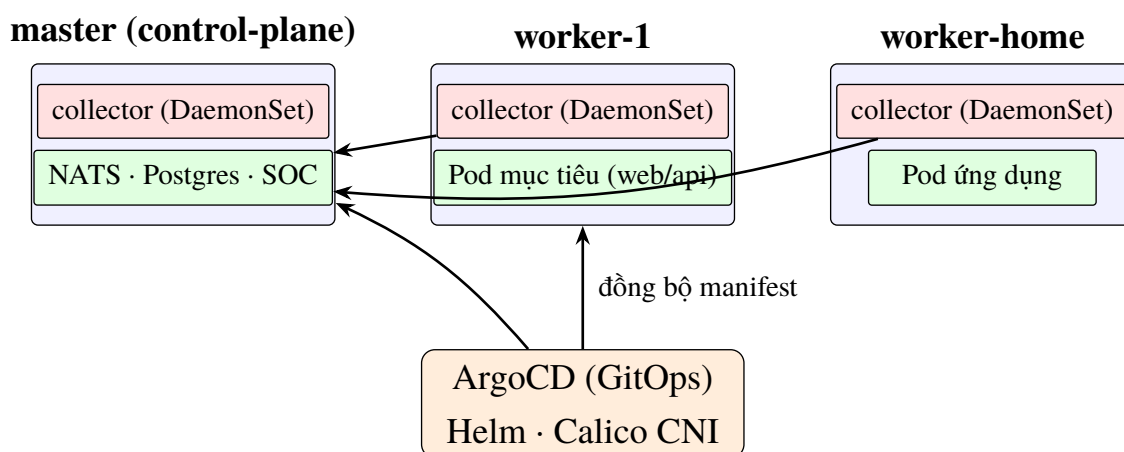
```

1 $ kubectl get applications -n argocd
2 NAME              SYNC STATUS    HEALTH STATUS
3 platform-apps     Synced         Healthy
4 zt-cni-calico      Synced         Healthy
5 zt-soc-core        Synced         Healthy
6 zt-soc-data        Synced         Healthy
7
8 $ kubectl get daemonset -n zt-mapper
9 NAME              DESIRED    CURRENT    READY    NODE
10 zt-pipeline-ebpf  2          2          2        <none>

```

Mã nguồn 4.3. Minh chứng triển khai: ArgoCD đồng bộ và DaemonSet thu thập đang chạy (kubectl rút gọn).

Ở tầng truyền thông, các dịch vụ trao đổi cảnh báo qua trục thông điệp NATS theo các chủ đề có cấu trúc: zt.alerts.raw (đầu ra pipeline mỗi node) và zt.alerts.enriched (sau khi tổng hợp). Ở tầng lưu trữ, PostgreSQL chạy dưới dạng StatefulSet có ổ đĩa bền vững để giữ sự cố, cảnh báo và nhật ký kiểm toán. Hệ thống quan trắc Prometheus/Grafana cung cấp số liệu và dashboard cho đội vận hành. Hình 4.2 tóm tắt cách các thành phần phân bố trên cụm ba node cùng vòng đồng bộ GitOps.



Hình 4.2. Sơ đồ triển khai trên cụm Kubernetes ba node.

4.4. Bộ thu thập eBPF

Bộ thu thập là mắt xích đầu tiên và là thành phần đặc quyền duy nhất chạm vào tầng nhân, nên được trình bày kỹ nhất. Về kiến trúc, nó gồm hai phần (đã mô tả ở Hình 3.11, Chương 3): *chương trình tầng nhân* gắn kprobe/kretprobe vào hàm tcp_v4_connect và lọc sự kiện ngay trong nhân; và *chương trình tầng người dùng* nạp, gắn chương trình eBPF, đọc sự kiện từ bộ đệm vòng rồi xuất ra dòng JSON.

4.4.1. Chương trình tầng nhân

Chương trình tầng nhân khai báo ba BPF map, mỗi map giữ một vai trò rõ rệt:

- `cidr_map` (LPM_TRIE, 1024 mục): ánh xạ tiền tố CIDR sang nhãn (1=pod, 2=service, 3=node, 4=RFC1918, 5=loopback) để phân loại địa chỉ *ngay trong nhân*. Bắt buộc cờ BPF_F_NO_PREALLOC.
- `events` (RINGBUF, 256 KiB): kênh truyền sự kiện một chiều từ nhân lên không gian người dùng, thay cho perf buffer cũ, cho thông lượng cao hơn và không mất thứ tự.
- `sock_store` (HASH, khóa pid_tgid): lưu tạm con trỏ struct sock giữa lúc kprobe vào và kretprobe ra, vì thông tin địa chỉ đích chỉ đầy đủ *sau* khi kết nối được thiết lập.

Cơ chế hai pha (entry → return) là điểm tinh tế nhất: kprobe lúc vào chỉ lưu con trỏ socket theo pid_tgid; kretprobe lúc ra mới đọc địa chỉ nguồn/đích bằng CO-RE (BPF_CORE_READ), *lọc nguồn* (chỉ phát sự kiện có nhãn nguồn = 1, tức xuất phát từ mạng pod – đúng mô hình mỗi đe dọa “pod bị chiếm”), rồi đặt sự kiện vào bộ đệm vòng. Mã nguồn 4.4 trích phần lõi của kretprobe.

```

1 SEC("kretprobe/tcp_v4_connect")
2 int BPF_KRETPROBE(tcp_v4_connect_return, int ret)
3 {
4     if (ret != 0 && ret != -115)    // chỉ xử lý connect()

```

```

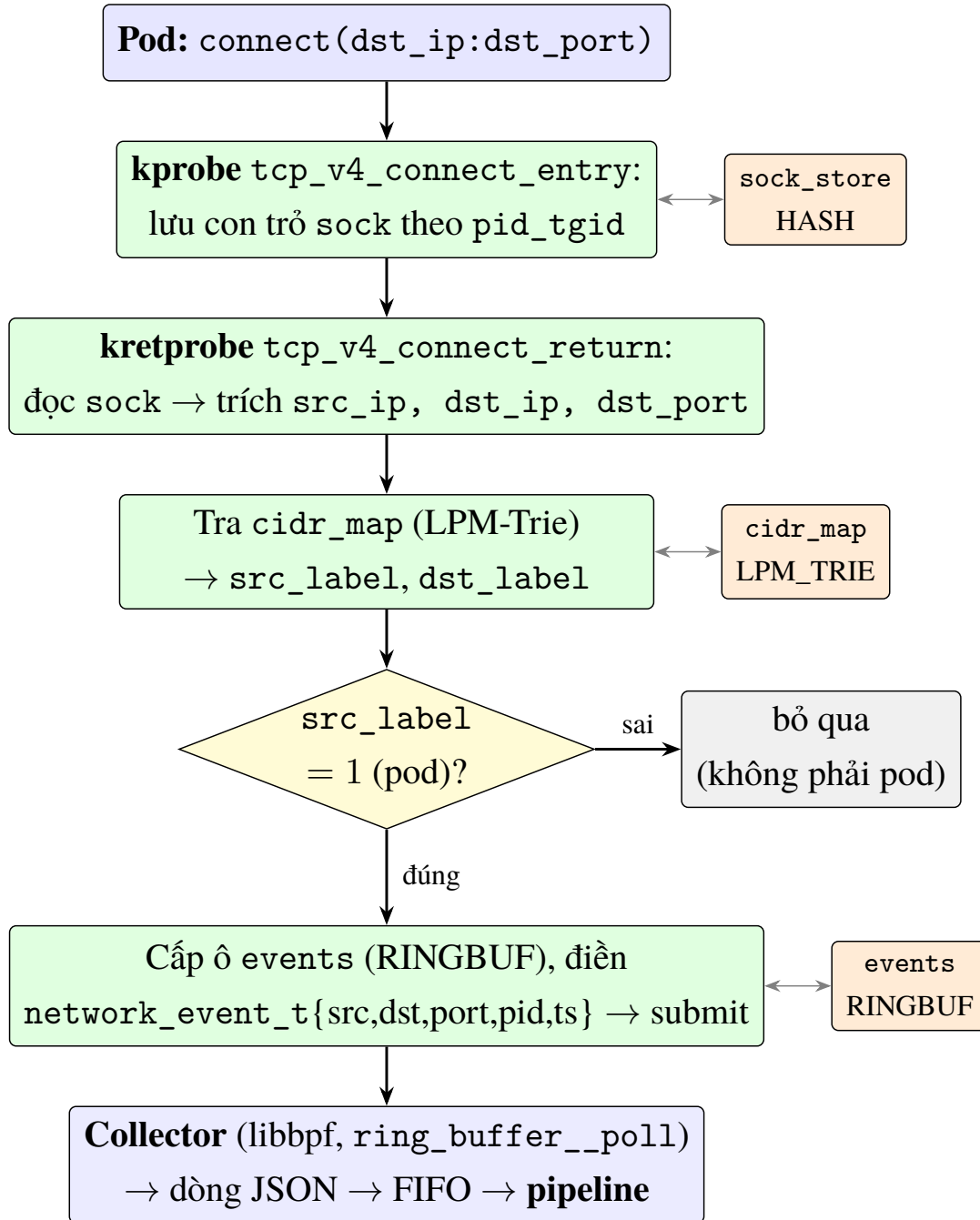
    thành công / EINPROGRESS
5     return 0;
6     __u64 pid_tgid = bpf_get_current_pid_tgid();
7     struct sock **skp = bpf_map_lookup_elem(&sock_store, &
        pid_tgid);
8     if (!skp) return 0;
9     struct sock *sk = *skp;
10    bpf_map_delete_elem(&sock_store, &pid_tgid);
11
12    __u32 dst_ip = 0, src_ip = 0; __u16 dst_port = 0;
13    BPF_CORE_READ_INTO(&dst_ip, sk, __sk_common.skc_daddr)
        ;
14    BPF_CORE_READ_INTO(&dst_port, sk, __sk_common.skc_dport
        );
15    BPF_CORE_READ_INTO(&src_ip, sk, __sk_common.
        skc_rcv_saddr);
16    dst_port = __builtin_bswap16(dst_port);
17
18    // Loc nguồn: chỉ phát sự kiện nếu nguồn thuộc mạng pod
        (nhân == 1).
19    struct lpm_key src_key = { .prefixlen = 32, .addr =
        src_ip };
20    __u32 *src_label = bpf_map_lookup_elem(&cidr_map, &
        src_key);
21    if (!src_label || *src_label != 1)
22        return 0;
23
24    struct network_event_t *evt = bpf_ringbuf_reserve(&
        events, sizeof(*evt), 0);
25    if (!evt) return 0;
26    evt->src_ip = src_ip; evt->dst_ip = dst_ip; evt->
        dst_port = dst_port;
27    evt->pid = pid_tgid >> 32;
28    evt->timestamp_ns = bpf_ktime_get_ns();
29    bpf_ringbuf_submit(evt, 0);
30    return 0;
31 }

```

Mã nguồn 4.4. Trích kretprobe: đọc socket bằng CO-RE, lọc nguồn theo LPM, đẩy sự kiện qua ring buffer (tcp_connect.bpf.c).

Việc lọc nguồn ngay trong nhân là một quyết định thiết kế quan trọng: nó loại

bỏ phần lớn lưu lượng không liên quan (kết nối do chính node phát đi, lưu lượng host) *tại nguồn*, giảm số sự kiện phải truyền lên không gian người dùng và – như phân tích ở Chương 5 – giảm đáng kể tỷ lệ báo động giả. Đường đi đầy đủ của một gói tin từ lời gọi `connect()` tới khi thành sự kiện ở không gian người dùng được mô tả ở Hình 4.3.



Hình 4.3. Đường đi của một sự kiện kết nối qua eBPF.

4.4.2. Chương trình tầng người dùng

Phần tầng người dùng (`collector.cpp`) dùng `libbpf` để: mở tệp đối tượng `tcp_connect.bpf.o`, nạp vào nhân, gắn `kprobe/kretprobe`, nạp bảng *CIDR* vào `cidr_map`, rồi vào vòng lặp đọc bộ đệm vòng. Bảng *CIDR* (Mã nguồn 4.5) chính là tri

thức để chương trình tăng nhân phân loại địa chỉ; mỗi mục là cặp (tiền tố, nhãn) với địa chỉ ở thứ tự byte mạng.

```
1 struct { struct lpm_key key; uint32_t label; const char *
  desc; } entries[] = {
2   { {16, htonl(0x0AF40000)}, 1, "pod_network" },          //
   10.244.0.0/16
3   { {12, htonl(0x0A600000)}, 2, "service_network" },      //
   10.96.0.0/12
4   { {16, htonl(0xC0A80000)}, 3, "node_network" },          //
   192.168.0.0/16
5   { {12, htonl(0xAC100000)}, 4, "private_172" },          //
   172.16.0.0/12
6   { {8,  htonl(0x7F000000)}, 5, "loopback" },              //
   127.0.0.0/8
7 };
8 for (const auto &e : entries)
9   bpf_map_update_elem(map_fd, &e.key, &e.label, BPF_ANY);
```

Mã nguồn 4.5. Nạp bảng CIDR vào LPM_TRIE từ tầng người dùng (collector.cpp).

Mỗi sự kiện đọc được từ bộ đệm vòng được chuyển con trỏ qua hàm callback `handle_event`, ở đó địa chỉ nhị phân được đổi sang chuỗi chấm-thập phân và in ra stdout dưới dạng một dòng JSON (ví dụ `{"src_ip": "10.244.1.5", "dst_port": 8080, ...}`). Vòng lặp chính gọi `ring_buffer_poll` với chu kỳ 100 ms và thoát sạch khi nhận tín hiệu SIGINT/SIGTERM. Việc xuất JSON theo dòng giúp giữ giao diện *đơn giản và thay thế được*: pipeline chỉ cần đọc stdin, không phụ thuộc vào thư viện eBPF.

4.4.3. Đóng gói và triển khai

Để chạy được trên cụm có nhân không đồng nhất, ảnh Docker của bộ thu thập sinh tệp `vmlinux.h` từ thông tin BTF của nhân ngay trong quá trình xây dựng, hiện thực hóa cơ chế CO-RE đã trình bày ở Chương 2. Bộ thu thập được triển khai dưới dạng DaemonSet *đặc quyền* để hiện diện trên mọi node. Trong cùng pod, ba container chạy cạnh nhau và nối với nhau qua một FIFO trên `emptyDir`: collector ghi JSON vào FIFO, pipeline đọc FIFO qua stdin, còn `alert-bridge` đẩy cảnh báo lên NATS. Mã nguồn 4.6 trích phần khai báo container thu thập, bao gồm cơ chế tự khởi động lại và `securityContext` đặc quyền.

```
1 - name: collector
2   image: johnnaeder/ebpf-portscan-k8s-check:collector-ebpf-
   s3
3   command: ["/bin/sh", "-c"]
```

```

4   args:
5     - |
6       FIFO=/stream/events.pipe
7       [ -p "$FIFO" ] || mkfifo -m 0600 "$FIFO"
8       while true; do
9         /app/collector > "$FIFO" || echo "[ebpf] collector
          exited $" >&2
10        sleep 3
11      done
12   volumeMounts:
13     - { name: stream, mountPath: /stream }
14     - { name: btf,      mountPath: /sys/kernel/btf, readOnly:
          true }
15   securityContext:
16     privileged: true          # can de nap + attach BPF, doc
          kernel BTF

```

Mã nguồn 4.6. Trích DaemonSet thu thập: tạo FIFO, vòng tự khởi động lại, quyền đặc quyền nạp eBPF (23-pipeline-ebpf.yaml).

Vì collector chạy đặc quyền và nạp mã eBPF vào nhân, đây là thành phần có quyền cao hơn hẳn ứng dụng thường, nên tôi đặt nó vào một ranh giới tin cậy riêng và làm cứng tách biệt; phần phân tích mô hình tin cậy và các biện pháp làm cứng được trình bày cùng với các giới hạn an ninh ở mục 5.5 (Chương 5).

4.4.4. Kiểm chứng eBPF hoạt động ở tầng nhân

Vì can thiệp tầng nhân là trọng tâm của đồ án, cần một cách *quan sát trực tiếp* chứng minh chương trình eBPF đã thực sự được nạp, kiểm chứng và gắn vào ngăn xếp mạng của nhân – chứ không chỉ là một tiến trình người dùng thông thường. Việc kiểm chứng dựa trên bpftool (truy vấn trạng thái BPF của nhân) và giao diện tracing của nhân, gồm ba bước minh chứng độc lập (Mã nguồn 4.7).

Thứ nhất, bpftool prog show liệt kê chương trình đang nạp: kiểu kprobe gắn vào hàm tcp_v4_connect – đúng điểm móc đã chọn ở Chương 2 – kèm số lệnh đã được trình kiểm chứng (verifier) duyệt và đã biên dịch JIT, khẳng định mã chạy *trong* nhân chứ không phải trong không gian người dùng. *Thứ hai*, bpftool map show liệt kê đúng ba bản đồ BPF mà bộ thu thập sử dụng: cidr_map kiểu lpm_trie (lọc dải pod-network), events kiểu ringbuf (kênh chuyển sự kiện lên không gian người dùng) và sock_store kiểu hash (ghép pha vào/ra của connect). *Thứ ba*, đồ nội dung cidr_map xác nhận các CIDR pod-network đã được nạp, và đọc luồng sự kiện cho thấy mỗi lời gọi connect() trong cụm sinh đúng một bản ghi sự kiện – tức là cơ chế tracing đang chạy theo thời gian

thực.

```
1 # (1) Hai chương trình eBPF đã nạp: kprobe + kretprobe gắn
   tcp_v4_connect, đã JIT
2 $ sudo bpftool prog show
3 97069: kprobe name tcp_v4_connect_entry gpl run_cnt
   101167
4         xlated 112B jited 72B memlock 4096B map_ids
   19041 btfd_id 3973
5 97070: kprobe name tcp_v4_connect_return gpl run_cnt
   101167
6         xlated 760B jited 412B memlock 4096B map_ids
   19041,19039,19040 btfd_id 3973
7
8 # (2) Ba bản đồ BPF của bộ thu thập
9 $ sudo bpftool map show
10 19039: lpm_trie name cidr_map flags 0x1 key 8B value
   4B max_entries 1024
11 19040: ringbuf name events flags 0x0
   max_entries 262144
12 19041: hash name sock_store flags 0x0 key 8B value
   8B max_entries 4096
13
14 # (3) Các CIDR pod-network đã nạp vào bộ lọc LPM
15 $ sudo bpftool map dump name cidr_map | head
16 [{ "key": { "prefixlen": 12, "addr": 24586 }, "value": 2 },
17  { "key": { "prefixlen": 16, "addr": 62474 }, "value": 1 }]
18
19 # (4) Tracing thời gian thực: mỗi connect() sinh một sự
   kiện JSON trên collector stdout
20 $ kubectl -n zt-mapper logs zt-pipeline-ebpf-tx94l -c
   collector --tail=1
21 {"src":"10.244.204.189","dst":"10.244.204.161","dport":80,"
   src_label":1}
```

Mã nguồn 4.7. Kiểm chứng chương trình và bản đồ eBPF ở tầng nhân (chạy bpftool trên node).

Bốn đầu ra trên là minh chứng đầy đủ cho chuỗi “nạp → kiểm chứng → JIT → gắn móc → bắt sự kiện” diễn ra ở tầng nhân. Hình 4.4 là ảnh chụp đầu ra bpftool thật trên node user-server-1: hai chương trình kprobe/kretprobe gắn vào tcp_v4_connect (đã xlated/jited, số lần chạy run_cnt hơn 10^5) và đúng ba bản

đồ cidr_map (lpm_trie), events (ringbuf), sock_store (hash). Quy trình chạy lại đầy đủ được tổng hợp trong Phụ lục F.

```
johnnaeder@userver-worker-1:~$ sudo bpftool prog show | grep -A4 tcp_v4_connect

[sudo] password for johnnaeder:
97069: kprobe name tcp_v4_connect_entry tag cfc74b011cf654ef gpl run_time_ns 50627849 run_cnt 101167
      loaded_at 2026-06-06T05:33:20+0000 uid 0
      xlated 112B jited 72B memlock 4096B map_ids 19041
      btf_id 3973
97070: kprobe name tcp_v4_connect_return tag 7f6e896cc411c407 gpl run_time_ns 163954672 run_cnt 101167
      loaded_at 2026-06-06T05:33:20+0000 uid 0
      xlated 760B jited 412B memlock 4096B map_ids 19041,19039,19040
      btf_id 3973
97076: cgroup_device tag 531db05b114e9af3
johnnaeder@userver-worker-1:~$ sudo bpftool map show

2: prog_array name hid_jump_table flags 0x0
   key 4B value 4B max_entries 1024 memlock 8576B
   owner_prog_type tracing owner jited
19039: lpm_trie name cidr_map flags 0x1
   key 8B value 4B max_entries 1024 memlock 240B
   btf_id 3973
19040: ringbuf name events flags 0x0
   key 0B value 0B max_entries 262144 memlock 275864B
19041: hash name sock_store flags 0x0
   key 8B value 8B max_entries 4096 memlock 329472B
   btf_id 3973
25061: hash name calico_failsafe flags 0x1
   key 4B value 1B max_entries 65535 memlock 1050440B
johnnaeder@userver-worker-1:~$ sudo bpftool map dump name cidr_map | head
[{"
    "key": {
      "prefixLen": 12,
      "addr": 24586
    },
    "value": 2
  }, {
    "key": {
      "prefixLen": 16,
      "addr": 62474
    }
  }
]
```

Hình 4.4. Minh chứng eBPF ở tầng nhân qua bpftool trên node.

4.5. Bộ máy pipeline

Bộ máy pipeline (engine.cpp + thư viện libdsa) là nơi đặt các thuật toán phát hiện. Kiến trúc của nó (Hình 3.11, phải) gồm một bộ *sổ đăng ký* (Registry) dựng các thuật toán theo tên trong cấu hình, và một vòng lặp xử lý sự kiện đọc từ stdin.

4.5.1. Cấu hình theo dữ liệu

Toàn bộ hành vi phát hiện – kích thước Count-Min Sketch, ngưỡng, độ dài cửa sổ, danh sách dải mạng phân loại, luật mức độ nghiêm trọng – đều khai báo trong một tệp YAML nạp vào pod qua ConfigMap (Mã nguồn 4.8), nên có thể điều chỉnh mà không cần biên dịch lại. Đây cũng là nơi định nghĩa ánh xạ CIDR → nhãn mà pipeline dùng cho phân loại LPM ở tầng người dùng (song song với bảng trong nhân của bộ thu thập).

```
1 algorithms:
2   frequency_estimator: { name: count_min_sketch, params: {
```

```

        width: 2048, depth: 5, threshold: 40 } }
3  scc_detector:          { name: tarjan }
4  reachability:         { name: bfs, params: { max_hops: 3 }
    }
5  ip_classifier:         { name: lpm_trie }
6  networks:
7  - { cidr: "10.244.0.0/16", label: 1, name: pod_network }
8  - { cidr: "10.96.0.0/12",  label: 2, name:
    service_network }
9  - { cidr: "0.0.0.0/0",     label: 0, name: external }
10 analysis: { window_seconds: 60, scc_interval_seconds: 10 }
11 severity_rules:
12 - { type: port_scan,      severity: medium }
13 - { type: blast_radius,   severity: high }

```

Mã nguồn 4.8. Trích cấu hình pipeline: thuật toán, ngưỡng, dải mạng, cửa sổ phân tích (pipeline.yaml).

4.5.2. Vòng lặp xử lý sự kiện

Với mỗi sự kiện, vòng lặp thực hiện một chuỗi bước cố định (Giải thuật 4.9): kiểm tra và đặt lại cửa sổ trượt; phân loại địa chỉ bằng LPM; cập nhật bộ đếm Count-Min Sketch theo địa chỉ nguồn và ước lượng tần suất; nếu vượt ngưỡng thì phát cảnh báo port_scan (mỗi nguồn chỉ một lần trong cửa sổ) và tính bán kính ảnh hưởng bằng BFS k bước để phát cảnh báo blast_radius; bồi đắp đồ thị truyền thông; và định kỳ chạy Tarjan để phát hiện thành phần liên thông mạnh (scc_anomaly). Lưu đồ tương ứng ở Hình 4.5.

```

1  for each event e read from stdin:
2      if e.ts - window_start > window_seconds: reset CMS;
        clear alerted; window_start = e.ts
3      src_label = lpm.lookup(e.src_ip)           // phân
        loại địa chỉ nguồn
4      cms.record(e.src_ip); freq = cms.estimate(e.src_ip)
5      if freq > threshold and e.src_ip not in alerted:
6          alerted.add(e.src_ip); emit Alert("port_scan", e.
            src_ip, freq)
7          if graph.has(e.src_ip):
8              blast = bfs.reachable_within(graph, e.src_ip,
                max_hops)
9              emit Alert("blast_radius", e.src_ip, blast)
10         graph.add_edge(e.src_ip, e.dst_ip)       // bồi đắp
            đồ thị truyền thông

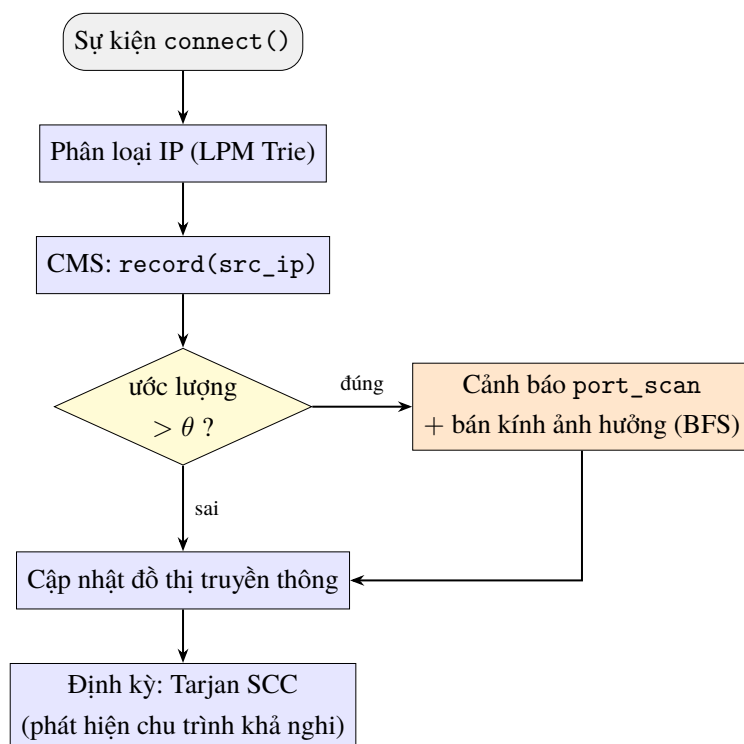
```

```

11     if total_events % (scc_interval*10) == 0:
12         sccs = tarjan.find_sccs(graph)
13         if new_nontrivial_scc(sccs): emit Alert("
            scc_anomaly", scc)

```

Mã nguồn 4.9. Vòng lặp xử lý sự kiện của pipeline (rút gọn từ engine.cpp).



Hình 4.5. Lưu đồ xử lý một sự kiện trong pipeline.

Ở đây tôi đặt cả phân tích *tần suất* (CMS) lẫn phân tích *đồ thị* (Tarjan SCC, BFS bán kính ảnh hưởng) *ngay trong pipeline mỗi node*, chứ không phải ở tầng tổng hợp. Tầng tổng hợp (mục 4.6) chỉ làm tương quan và khử trùng lặp cảnh báo trên phạm vi toàn cụm.

4.5.3. Phát cảnh báo

Cảnh báo được gửi qua một *chuỗi xử lý* (AlertHandlerChain) cấu hình được: handler ghi tệp alerts.json (NDJSON) và handler ghi stderr. Mỗi cảnh báo mang đủ ngữ cảnh để SOC xử lý về sau: loại, nguồn, mức nghiêm trọng, node_name, namespace, nhãn và phần details dạng JSON (ví dụ tần suất ước lượng, ngưỡng, danh sách đích trong bán kính ảnh hưởng). Mã nguồn 4.10 trích đoạn phát cảnh báo port_scan.

```

1  if (freq > threshold && alerted_sources.find(evt.src_ip) ==
    alerted_sources.end()) {
2      alerted_sources.insert(evt.src_ip);
3      dsa::soc::Alert alert;
4      alert.type      = "port_scan";

```

```

5     alert.source      = ip_to_string(evt.src_ip);
6     alert.severity    = severity_for(alert.type, cfg.
        severity_rules);
7     alert.node_name   = node_name;           // tu Downward API (
        NODE_NAME)
8     alert.tags        = {"port_scan"};
9     chain.send(alert);                       // -> alerts.json ->
        alert-bridge -> NATS
10  }

```

Mã nguồn 4.10. Phát cảnh báo port_scan khi vượt ngưỡng (rút gọn từ engine.cpp).

Container sidecar alert-bridge (Go) theo dõi đuôi tệp alerts.json, mỗi dòng mới được nó publish lên chủ đề NATS zt.alerts.raw. Tách phần đầu NATS ra một tiến trình Go riêng giúp bộ máy C++ không phải liên kết libnats, đồng thời cho phép siết quyền sidecar tối đa (phân tích ở mục 5.5).

4.6. Trung tâm vận hành an ninh (SOC)

SOC nhận cảnh báo thô từ các node, tương quan thành sự cố, làm giàu ngữ cảnh Kubernetes, sinh chính sách cách ly và phục vụ người vận hành. Dòng chảy thông điệp: pipeline mỗi node $\xrightarrow{\text{zt.alerts.raw}}$ aggregator $\xrightarrow{\text{zt.alerts.enriched}}$ incident-service $\rightarrow$ PostgreSQL $\rightarrow$ web-console.

4.6.1. Bộ tổng hợp (aggregator)

Bộ tổng hợp (Hình 3.12, trái) đăng ký zt.alerts.raw theo *queue group* (để chạy nhiều bản sao mà không trùng lặp), đưa mỗi cảnh báo qua bộ *tương quan* (Correlator) rồi tái phát những cảnh báo đáng phát lên zt.alerts.enriched. Khóa tương quan là incident_key = type|source; hai cảnh báo cùng khóa trong cửa sổ thời gian được coi là cùng một sự cố. Logic cốt lõi (Mã nguồn 4.11): cảnh báo *đầu tiên* của một khóa được phát ngay; các cảnh báo lặp bị *nén*; nhưng đúng thời điểm cùng một nguồn xuất hiện trên node thứ hai (*cross-node*), mức nghiêm trọng được *nâng cấp* và cảnh báo được tái phát kèm nhãn cross_node_attacker. Đây chính là góc nhìn “toàn cục” của hệ thống – đạt được bằng tương quan cảnh báo, không phải bằng một đồ thị toàn cục.

```

1  key := a.Type + "|" + a.Source
2  st, exists := c.incidents[key]
3  if !exists || now-st.LastSeenNS > c.Window.Nanoseconds() {
4      st = &incidentState{Key: key, FirstSeenNS: now,
        LastSeenNS: now, Count: 1, ...}
5      c.incidents[key] = st
6      return a, true           // lan dau -> phat

```

```

    ngay
7  }
8  prevCrossNode := len(st.Nodes) >= 2
9  st.LastSeenNS = now; st.Count++; st.Nodes[a.NodeName] =
    struct{}{}
10 if !prevCrossNode && len(st.Nodes) >= 2 { // vua thanh
    cross-node
11     a.Severity = upgradeSeverity(a.Severity)
12     a.Tags = appendUnique(a.Tags, "cross_node_attacker")
13     return a, true // nang cap -> tai
        phat
14 }
15 return nil, false // trung lap ->
    nen

```

Mã nguồn 4.11. Tương quan và nâng cấp cross-node trong aggregator (rút gọn từ correlator.go).

4.6.2. Dịch vụ sự cố (incident-service)

Dịch vụ sự cố (Hình 3.13, trái) là thành phần phức tạp nhất của SOC, gồm năm gói nội bộ: natssub (tiêu thụ cảnh báo), store (PostgreSQL), response (sinh NetworkPolicy và thao tác pod), api (REST) và rbac/audit (phân quyền và kiểm toán).

Tiêu thụ cảnh báo và làm giàu. Bộ tiêu thụ (Mã nguồn 4.12) *upsert* mỗi cảnh báo enriched thành sự cố theo *incident_key*: nếu chưa có thì tạo mới, nếu có thì cập nhật *last_seen*/số lần/nodes/mức nghiêm trọng. Với sự cố mới, tôi cho dịch vụ thực hiện hai việc: (i) *làm giàu ngữ cảnh Kubernetes* – tra cứu pod nào đang giữ địa chỉ nguồn (LookupPodByIP duyệt *status.PodIP*) rồi lưu kèm tên pod/namespace, để người vận hành thấy ngay danh tính kẻ tấn công thay vì chỉ một địa chỉ IP; (ii) *tự sinh hành động* khi mức nghiêm trọng thuộc {high, critical} – tạo bản nháp NetworkPolicy ở trạng thái pending chờ duyệt.

```

1  inc, created, _ := st.UpsertIncident(ctx, &inAlert)
2  st.AppendAlert(ctx, inc.ID, msg.Data)
3  if created && resolver != nil { //
    lam giâu ngu canh K8s
4      if name, found, _ := resolver.LookupPodByIP(ctx, ns,
        inc.SourceIP); found {
5          st.SetIncidentPodContext(ctx, inc.ID, name, ns)
6      }

```

```

7 }
8 if _, ok := autoSet[inc.Severity]; ok { //
    high/critical -> sinh nhap chinh sach
9     yaml := response.GenerateQuarantine(response.
        QuarantineRequest{
10         IncidentID: inc.ID, SourceIP: inc.SourceIP,
            Severity: inc.Severity})
11     st.CreateAction(ctx, &store.Action{IncidentID: inc.ID,
        Type: "quarantine_pod", YAMLDraft: yaml})
12 }

```

Mã nguồn 4.12. Tiêu thụ cảnh báo: upsert sự cố, làm giàu pod, tự sinh hành động (rút gọn từ subscriber.go).

Lưu trữ. Trạng thái được giữ trong PostgreSQL qua bốn bảng – incidents, alerts, actions, audit_log – mà lược đồ chi tiết đã trình bày ở ERD Chương 3. Tôi gộp các lần xuất hiện lặp của cùng một sự cố bằng phép *upsert* INSERT ... ON CONFLICT (incident_key) DO UPDATE, dựa trên ràng buộc duy nhất trên incident_key (Mã nguồn 4.13); migration được nhúng vào nhị phân và áp tự động khi khởi động.

```

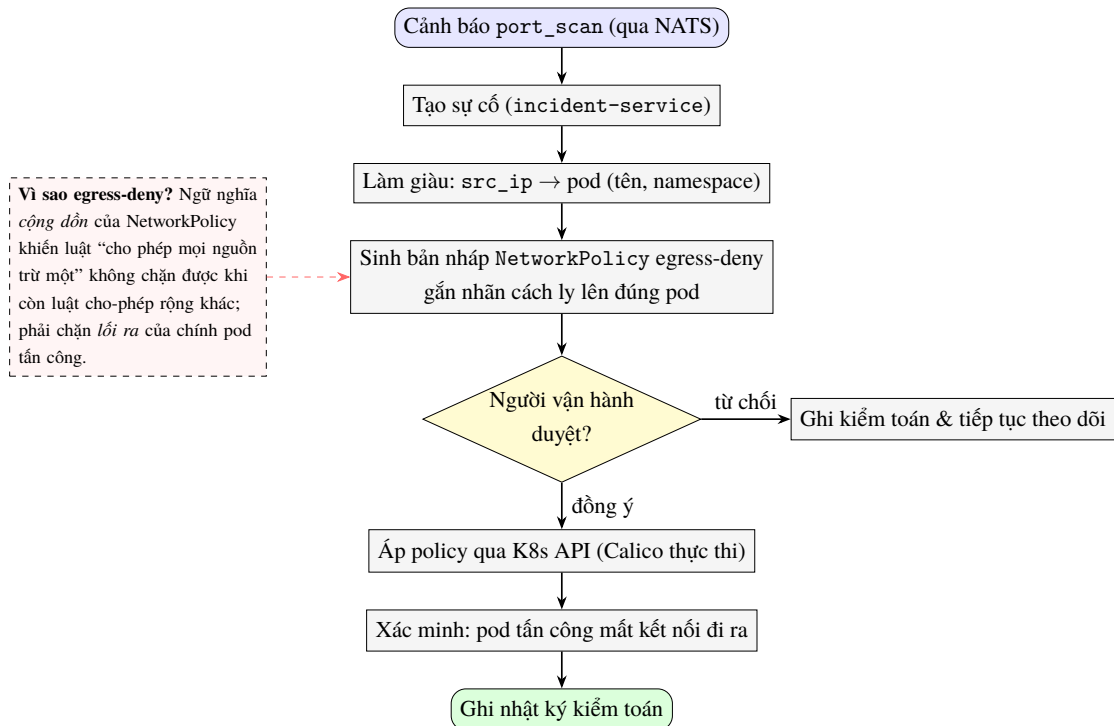
1 CREATE TABLE IF NOT EXISTS incidents (
2     id                BIGSERIAL PRIMARY KEY,
3     incident_key      TEXT NOT NULL,           -- type
    /source: khoa khu trung lap
4     type              TEXT NOT NULL,
5     source_ip         TEXT NOT NULL,
6     severity          TEXT NOT NULL DEFAULT 'medium',
7     status            TEXT NOT NULL DEFAULT 'new', -- new/
    ack/resolved/suppressed
8     nodes_observed    TEXT[] NOT NULL DEFAULT '{}',
9     occurrence_count  INT NOT NULL DEFAULT 1,
10    cross_node         BOOLEAN NOT NULL DEFAULT FALSE,
11    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
12    UNIQUE (incident_key)
13 );

```

Mã nguồn 4.13. Lược đồ bảng incidents và khóa duy nhất dùng cho upsert (001_init.sql).

Sinh và áp chính sách cách ly. Bộ sinh phản hồi (response) tạo NetworkPolicy dạng *chặn mọi lưu lượng đi ra* (egress-deny) gắn lên đúng pod tấn công qua một nhãn

theo sự cố. Khi hiện thực bước này tôi gặp một vấn đề kỹ thuật quan trọng: vì ngữ nghĩa NetworkPolicy là *cộng dồn* (một kết nối được phép nếu *bất kỳ* policy nào cho phép), nên kiểu “cho phép mọi nguồn trừ kẻ tấn công” sẽ bị các luật cho-phép rộng khác vô hiệu hóa; ngược lại, chặn *egress của chính pod tấn công* thì không policy ingress nào ghi đè được, và dừng được cả quét cổng lẫn dịch chuyển ngang tại nguồn. Cơ chế áp dụng theo *có người duyệt*: khi admin phê duyệt, hệ thống mới gắn nhãn cách ly lên pod (QuarantinePodByIP) rồi áp policy qua API Kubernetes. Toàn bộ luồng này được tóm tắt ở Hình 4.6, còn bản nháp sinh ra có dạng Mã nguồn 4.14.



Hình 4.6. Lưu đồ sinh và áp NetworkPolicy cách ly.

```

1  apiVersion: networking.k8s.io/v1
2  kind: NetworkPolicy
3  metadata:
4    name: zt-quarantine-incident-<id>
5    namespace: zt-targets
6  spec:
7    podSelector:
8      matchLabels:
9        zt-soc-quarantine-<id>: "true"    # pod tan cong, dan
                                         nhan boi enforcer
10   policyTypes: [Egress]
11   egress: []                             # chan TOAN BO luu
                                         luong di ra cua pod do
  
```

Mã nguồn 4.14. Bản nháp NetworkPolicy egress-deny sinh tự động cho pod tấn công.

API, phân quyền và kiểm toán. Dịch vụ phơi bày REST API (router chi) với phân quyền theo vai trò ba mức – viewer (chỉ đọc), editor (ghi nhận/đóng sự cố), admin (phê duyệt/bác bỏ phản hồi). Phân quyền hiện thực bằng middleware xác thực Bearer token và bộ chặn RequireRole theo bậc thang admin > editor > viewer (Mã nguồn 4.15). Mọi thao tác làm thay đổi trạng thái – tạo sự cố, đổi trạng thái, sinh/duyet/bác bỏ hành động – đều được ghi vào bảng audit_log kèm chủ thể, vai trò và phần diff, phục vụ truy vết và xuất báo cáo tuân thủ.

```
1 r.Use(rbac.Middleware(tokens))
// xac thuc Bearer token
2 r.Route("/api/v1/actions", func(r chi.Router) {
3     r.Get("/", s.listActions)
// viewer doc duoc
4     r.With(rbac.RequireRole(rbac.RoleAdmin)).Post("/{id}/
    approve", s.approveAction)
5     r.With(rbac.RequireRole(rbac.RoleAdmin)).Post("/{id}/
    reject", s.rejectAction)
6 })
7 // RequireRole: admin(3) > editor(2) > viewer(1)
8 func RequireRole(min Role) func(http.Handler) http.Handler
{ /* tu choi neu rank < min */ }
```

Mã nguồn 4.15. Định tuyến có phân quyền: phê duyệt hành động yêu cầu vai trò admin (rút gọn từ api.go, rbac.go).

4.6.3. Console web

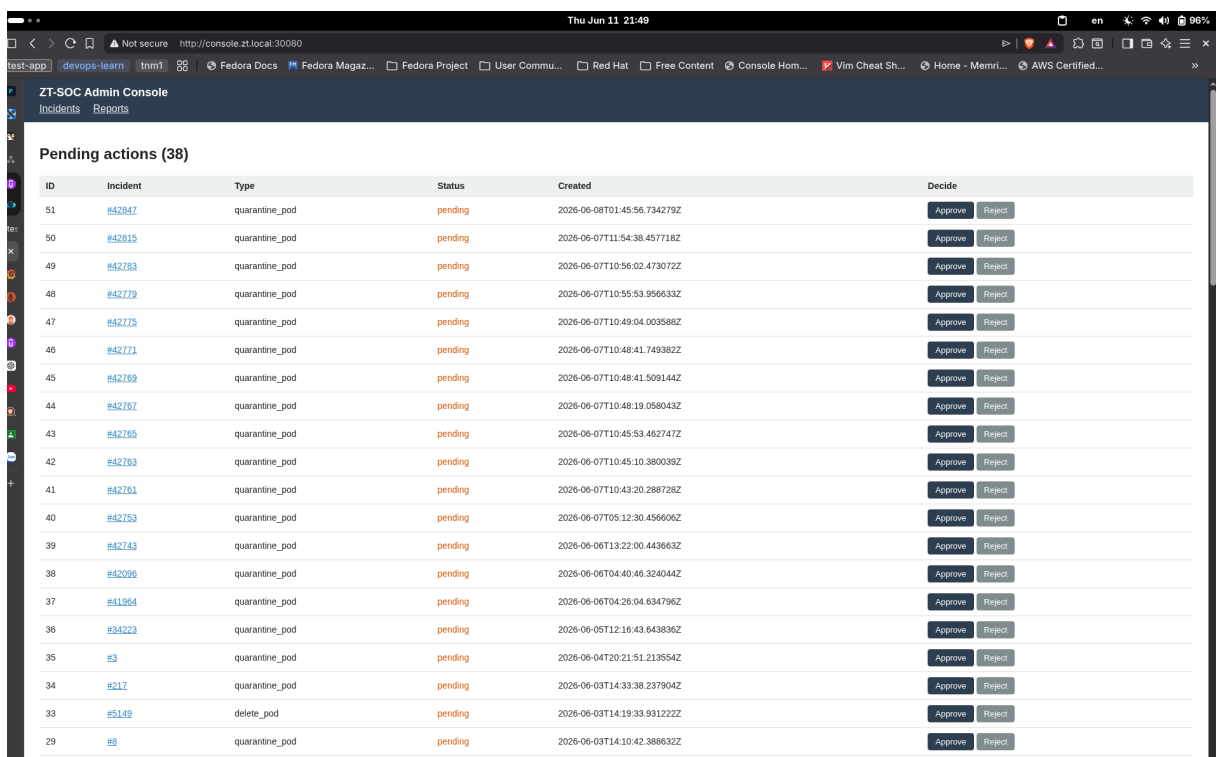
Console web (Hình 3.13, phải) là một giao diện mỏng Flask + HTMX đặt trước REST API của dịch vụ sự cố. Nó liệt kê sự cố và hành động đang chờ, cho phép admin phê duyệt/bác bỏ NetworkPolicy ngay tại dòng (HTMX thay từng hàng mà không tải lại trang), và xem báo cáo tuần. Về bảo mật, console xác thực người dùng bằng Basic Auth, còn khi gọi xuống API thì dùng *token admin phía máy chủ* (RBAC_ADMIN_TOKEN) – trình duyệt không bao giờ giữ token API. Đây là một proxy có chủ đích: tách phần xác thực người dùng khỏi phần phân quyền API.

4.7. Ứng dụng và giao diện vận hành (chạy thật)

Mục này trình bày ứng dụng *chạy thật* trên cụm, phân biệt với phần thiết kế giao diện (mockup) ở Chương 3. Console phục vụ trọn vòng xử lý sự cố: người vận hành thấy

danh sách sự cố có thể lọc/sắp xếp, mở chi tiết một sự cố để xem siêu dữ liệu đã làm giàu (gồm pod nguồn), các cảnh báo liên quan và bản nháp chính sách cách ly, rồi ghi nhận hoặc phê duyệt tùy vai trò. Song song, dashboard Grafana cung cấp góc nhìn kỹ thuật theo thời gian thực về số lượng cảnh báo, dòng thời gian sự cố và các nguồn hoạt động mạnh nhất.

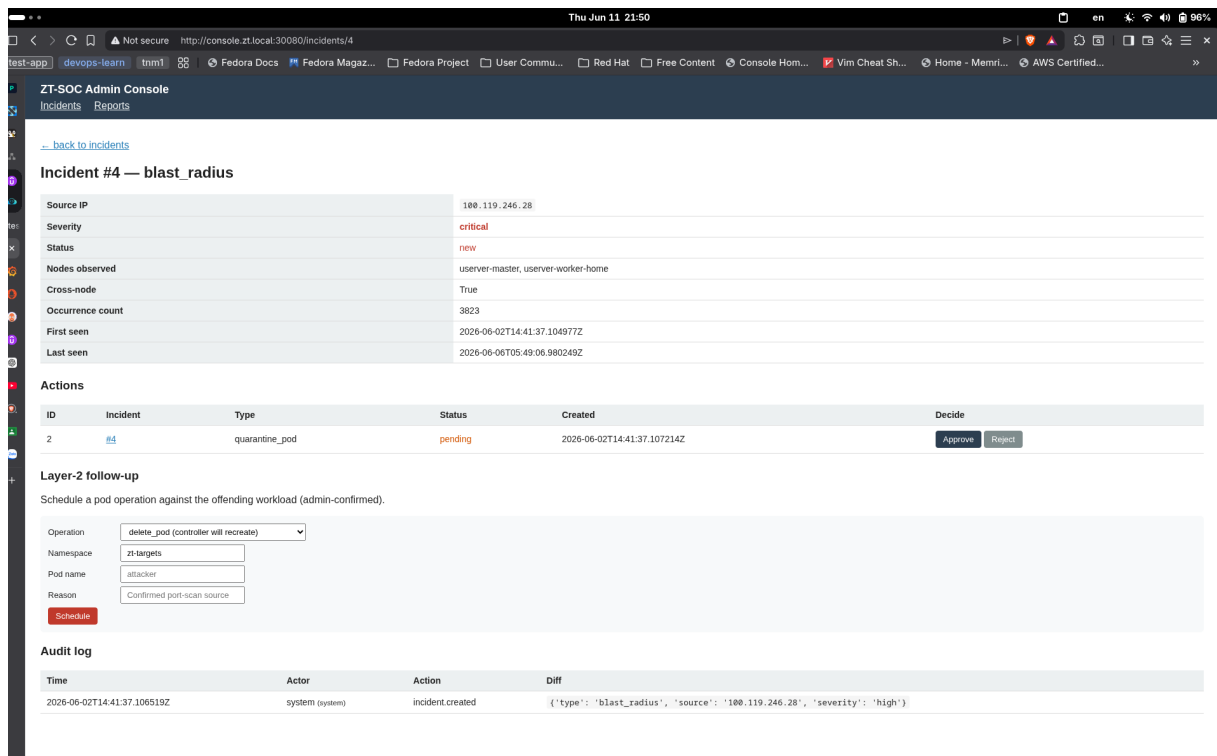
Console được phục vụ tại `console.zt.local` qua ingress của cụm. Màn hình *Pending actions* (Hình 4.7) là hàng đợi phê duyệt: mỗi dòng là một bản nháp hành động (chủ yếu `quarantine_pod`) do dịch vụ sự cố tự sinh từ cảnh báo mức cao, kèm hai nút *Approve/Reject* – hiện thực hóa điểm dừng *có người trong vòng lặp*. Tại thời điểm chụp, console liệt kê 38 hành động đang chờ, đúng bằng số sự cố mức `high/critical` đang lưu trong cơ sở dữ liệu.



ID	Incident	Type	Status	Created	Decide
51	#42847	quarantine_pod	pending	2026-06-08T01:45:56.734279Z	Approve Reject
50	#42815	quarantine_pod	pending	2026-06-07T11:54:38.457718Z	Approve Reject
49	#42783	quarantine_pod	pending	2026-06-07T10:56:02.473072Z	Approve Reject
48	#42779	quarantine_pod	pending	2026-06-07T10:55:53.956633Z	Approve Reject
47	#42775	quarantine_pod	pending	2026-06-07T10:49:04.003588Z	Approve Reject
46	#42771	quarantine_pod	pending	2026-06-07T10:48:41.749382Z	Approve Reject
45	#42769	quarantine_pod	pending	2026-06-07T10:48:41.509144Z	Approve Reject
44	#42767	quarantine_pod	pending	2026-06-07T10:48:19.058043Z	Approve Reject
43	#42765	quarantine_pod	pending	2026-06-07T10:45:53.462747Z	Approve Reject
42	#42763	quarantine_pod	pending	2026-06-07T10:45:10.380039Z	Approve Reject
41	#42761	quarantine_pod	pending	2026-06-07T10:43:20.288728Z	Approve Reject
40	#42753	quarantine_pod	pending	2026-06-07T05:12:30.456606Z	Approve Reject
39	#42743	quarantine_pod	pending	2026-06-06T13:22:00.443663Z	Approve Reject
38	#42096	quarantine_pod	pending	2026-06-06T04:40:46.324044Z	Approve Reject
37	#41964	quarantine_pod	pending	2026-06-06T04:26:04.634796Z	Approve Reject
36	#34223	quarantine_pod	pending	2026-06-05T12:16:43.643836Z	Approve Reject
35	#3	quarantine_pod	pending	2026-06-04T20:21:51.213554Z	Approve Reject
34	#217	quarantine_pod	pending	2026-06-03T14:33:38.237904Z	Approve Reject
33	#5149	delete_pod	pending	2026-06-03T14:19:33.931222Z	Approve Reject
29	#8	quarantine_pod	pending	2026-06-03T14:10:42.388632Z	Approve Reject

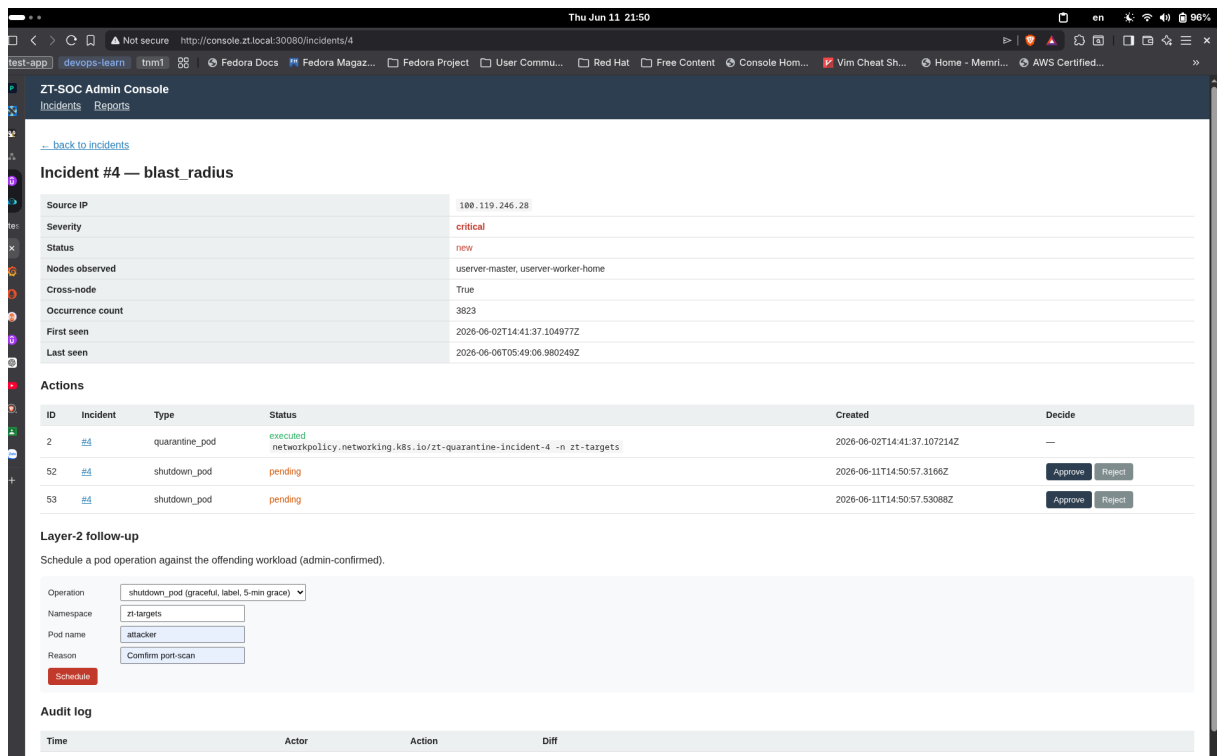
Hình 4.7. Hàng đợi phê duyệt hành động trên console.

Mở chi tiết một sự cố (Hình 4.8) cho thấy toàn bộ ngữ cảnh đã làm giàu: địa chỉ nguồn, mức nghiêm trọng, các node quan sát được, cờ *cross-node*, số lần tái phát, mốc thời gian đầu/cuối, danh sách hành động, biểu mẫu thao tác tầng 2 và *nhật ký kiểm toán*. Ví dụ sự cố `blast_radius` mức `critical` từ nguồn `100.119.246.28` được quan sát đồng thời trên hai node (`userver-master`, `userver-worker-home`) nên được nâng mức và đánh dấu *cross-node*.



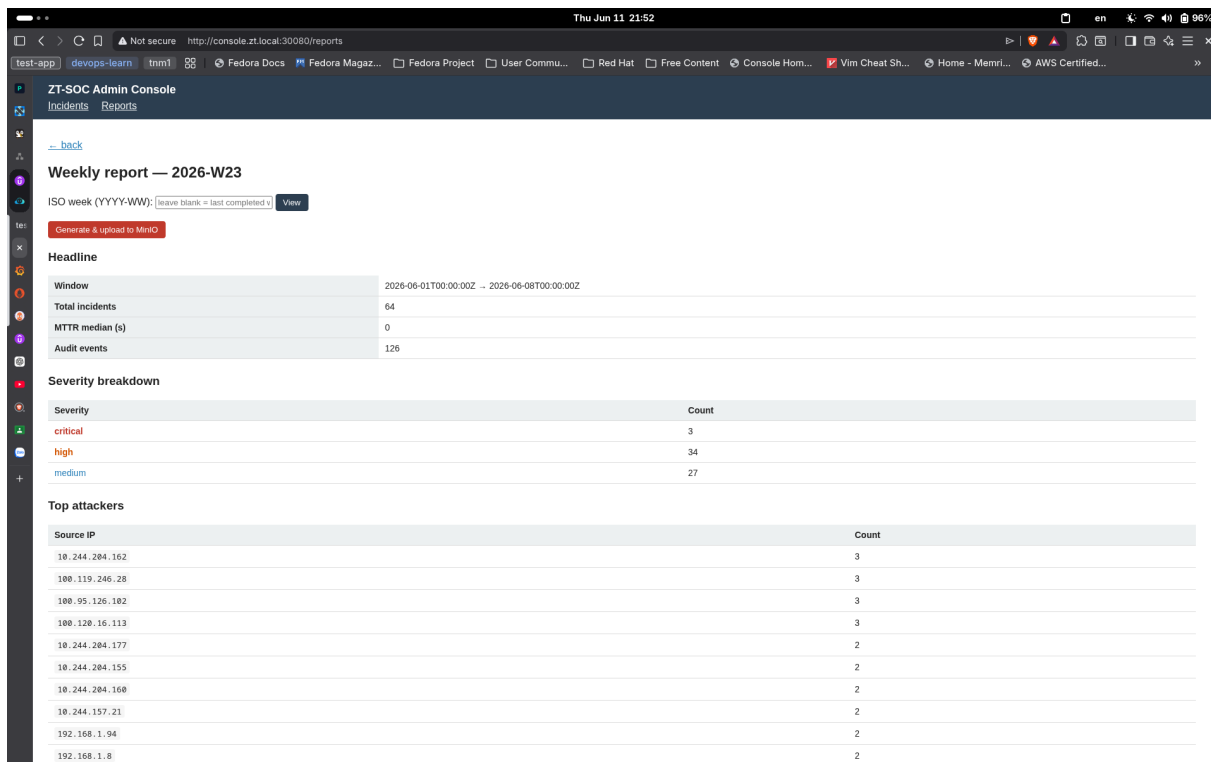
Hình 4.8. Chi tiết một sự cố với ngữ cảnh đã làm giàu và nhật ký kiểm toán.

Sau khi quản trị viên phê duyệt, hành động chuyển trạng thái executed và đối tượng NetworkPolicy tương ứng được áp vào cụm – Hình 4.9 ghi lại dòng networkpolicy.networking.k8s.io/zt-quarantine-incident-4 đã được áp trong không gian tên zt-targets, minh chứng vòng phản hồi đã khép kín tới tầng thực thi.



Hình 4.9. Hành động cách ly đã được áp (executed) sau phê duyệt.

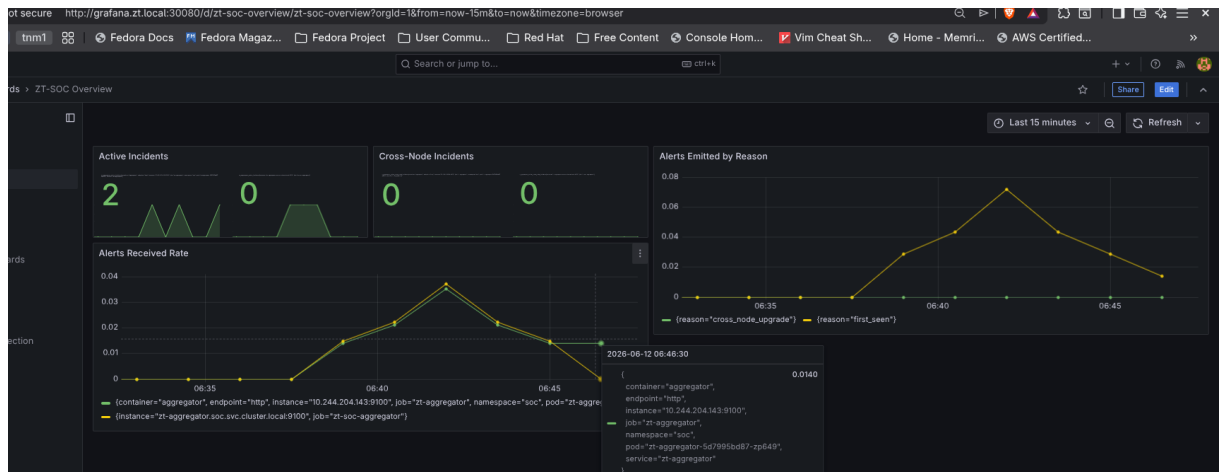
Trang *Reports* tổng hợp báo cáo tuần đọc trực tiếp từ PostgreSQL (Hình 4.10): tiêu đề kỳ, tổng số sự cố, phân bố theo mức nghiêm trọng và bảng nguồn tấn công nhiều nhất, kèm nút *Generate & upload to MinIO* để kết xuất và lưu trữ báo cáo. Đây cũng là nơi thể hiện rõ rằng số liệu hiển thị trên console/Reports là *số liệu thật* tích lũy trong cơ sở dữ liệu, không phải dữ liệu dựng sẵn.



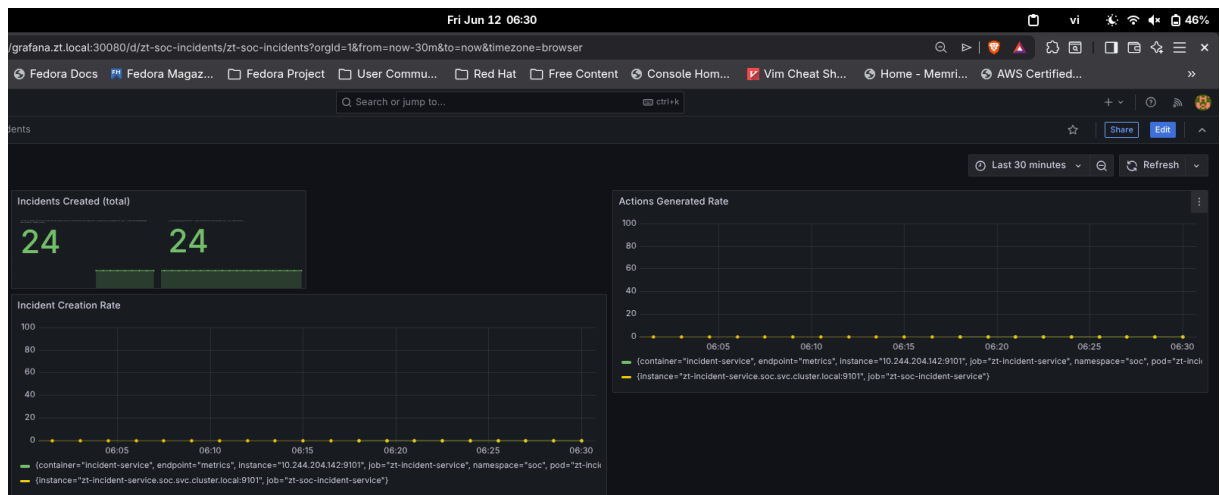
Hình 4.10. Trang báo cáo tuần kết xuất từ cơ sở dữ liệu.

Về đường dữ liệu của dashboard Grafana. Cần phân biệt hai nguồn dữ liệu khác nhau. Console/Reports đọc *trực tiếp* PostgreSQL – hệ thống lưu trữ bền vững tích lũy mọi sự cố từ trước tới nay (Hình 4.10). Trong khi đó các dashboard Grafana là *góc nhìn quan trắc theo cửa sổ thời gian*: bảng *Alerts Overview* truy vấn nhật ký trong Loki theo cửa sổ mặc định 30 phút, còn bảng *Incidents & Actions* truy vấn các bộ đếm Prometheus của dịch vụ SOC. Do đó dashboard chỉ phản ánh hoạt động *gần đây* và đòi hỏi (i) có lưu lượng tấn công trong cửa sổ đang xem, và (ii) các bộ đếm Prometheus phải được thu thập. Hệ quả của sự khác biệt nguồn dữ liệu này – và cấu hình thu thập tương ứng – được bàn kỹ hơn ở mục giới hạn của Chương 5.

Sau khi khai báo ServiceMonitor cho các dịch vụ SOC, Prometheus thu thập đúng hai mục tiêu `zt-aggregator:9100` và `zt-incident-service:9101`; hai dashboard *ZT-SOC Overview* và *ZT-SOC Incidents* (Hình 4.11 và 4.12) hiển thị các chỉ số như số sự cố đang hoạt động, sự cố cross-node, tốc độ nhận cảnh báo và tốc độ tạo sự cố. Do các panel dạng *rate* phản ánh *biến thiên* của bộ đếm, đường biểu diễn chỉ nhô lên khi có lưu lượng tấn công *ngay trong cửa sổ thời gian đang xem*; vì vậy ảnh chụp được lấy bằng cách thu hẹp cửa sổ (vài phút), bật tự làm mới và chạy kịch bản tấn công (Phụ lục F.3) ngay trước khi quan sát.



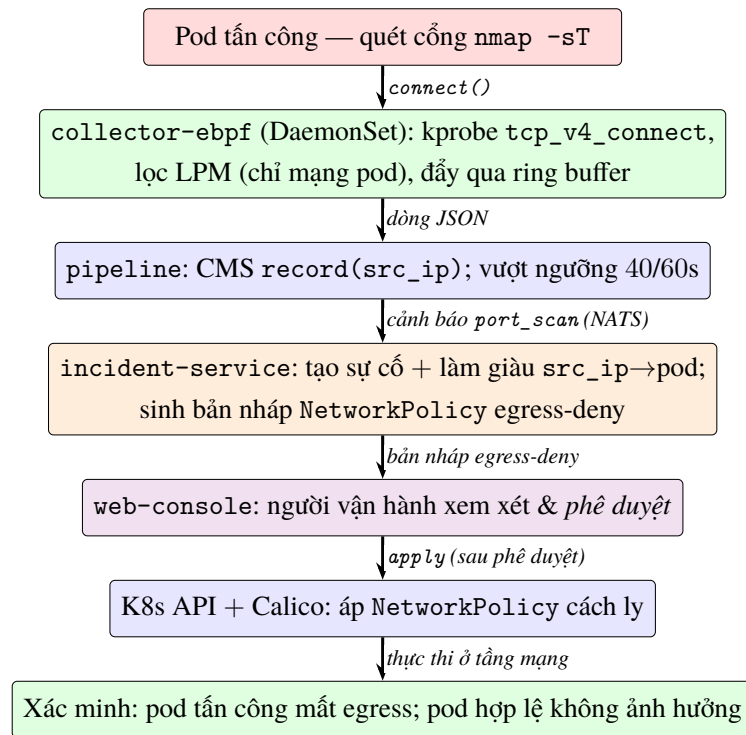
Hình 4.11. Dashboard *ZT-SOC Overview* trên Grafana.



Hình 4.12. Dashboard *ZT-SOC Incidents* trên Grafana.

4.8. Chạy thử nghiệm hệ thống end-to-end

Để kiểm chứng toàn bộ vòng phát hiện → phản hồi hoạt động liên mạch, hệ thống được chạy thử theo một kịch bản end-to-end trên cụm thật. Hình 4.13 tóm tắt kịch bản qua các thành phần đã triển khai, từ pod tấn công tới bước xác minh cách ly.



Hình 4.13. Lưu đồ kịch bản thử nghiệm end-to-end.

Một pod đóng vai kẻ tấn công được đặt trên một node worker và quét cổng bằng nmap ở chế độ connect (-sT) nhằm vào các pod dịch vụ. Bộ thu thập eBPF bắt các lời gọi connect() và đẩy sự kiện lên pipeline; khi số kết nối từ pod tấn công vượt ngưỡng trong cửa sổ thời gian, pipeline phát cảnh báo port_scan (và blast_radius khi pod chạm tới nhiều đích) lên NATS. Bộ tổng hợp khử trùng lặp và nâng cấp mức nghiêm trọng nếu nguồn xuất hiện trên nhiều node; dịch vụ sự cố tạo sự cố, làm giàu địa chỉ nguồn về đúng pod tấn công, rồi sinh bản nháp NetworkPolicy chặn mọi lưu lượng đi ra của pod đó. Người vận hành xem xét và phê duyệt qua console (điểm dừng có người trong vòng lặp); chỉ khi đó chính sách mới được áp vào cụm qua API Kubernetes. Bước cuối xác nhận hiệu lực: sau khi áp chính sách, pod tấn công không còn thiết lập được kết nối đi ra, trong khi lưu lượng hợp lệ của các pod khác không bị ảnh hưởng.

Mã nguồn 4.16 cho thấy chính xác hệ thống chạy ra như thế nào trong một lần thử: nhật ký của pipeline ghi cảnh báo port_scan/blast_radius, dịch vụ sự cố tạo sự cố và tự sinh hành động cách ly, và sau khi tôi phê duyệt trên console thì NetworkPolicy được áp khiến pod tấn công mất kết nối đi ra (mã trả về của lệnh kết nối đổi từ 0 sang khác 0).

```

1 # 1) Pipeline phát hiện quét cổng + blast radius (node
   master)
2 $ kubectl -n zt-mapper logs zt-pipeline-ebpf-qztgt -c
   pipeline | grep ALERT
3 [ALERT] Port scan detected: 100.119.246.28 (freq > tau=40)
4 [ALERT] Blast radius from 100.119.246.28: multi-node within

```

```

3 hops

5
6 # 2) Dịch vụ su co tạo su co cross-node + tự sinh ban nhập
   NetworkPolicy
7 $ kubectl -n soc logs deploy/zt-incident-service | grep -E
   'created|action'
8 [sub] incident.created id=4 type=blast_radius source
   =100.119.246.28 severity=critical
9 [sub] action.auto_generated incident_id=4 type=
   quarantine_pod

10
11 # 3) Sau khi admin phê duyệt trên console -> áp chỉnh sách
    cach ly
12 [apply] networkpolicy.networking.k8s.io/zt-quarantine-
    incident-4 created -n zt-targets

13
14 # 4) Xác minh: pod tan công mất kết nối đi ra (trước cach
    ly exit=0)
15 $ kubectl -n zt-targets exec attacker -- nc -w2
    10.244.204.161 80; echo "exit=$?"
16 exit=1

```

Mã nguồn 4.16. Nhật ký một lần chạy thử end-to-end (rút gọn từ sự cố #4 thật trên cụm).

Kịch bản này chứng minh tính *đúng đắn về mặt chức năng* của toàn hệ thống; các số liệu định lượng (tỷ lệ phát hiện, FPR, chi phí, độ trễ) được phân tích ở Chương 5. Mã nguồn đầy đủ của các thành phần cốt lõi được đưa vào phần Phụ lục: chương trình eBPF (Phụ lục C), console web (Phụ lục D) và lược đồ dữ liệu PostgreSQL (Phụ lục E).

4.9. Kết luận chương

Chương này đã trình bày trọn vẹn quá trình hiện thực và vận hành hệ thống trên một cụm Kubernetes thật, bao trùm cả ba lớp: hạ tầng nền tảng theo mô hình GitOps (giới thiệu vắn tắt), ba thành phần lõi được đào sâu kèm trích đoạn mã nguồn/cấu hình thật – bộ thu thập eBPF, pipeline phát hiện và các dịch vụ SOC – cùng ứng dụng giao diện cho người vận hành. Riêng *mô hình tin cậy* của bộ thu thập đặc quyền được phân tích cùng các giới hạn an ninh ở Chương 5. Việc chạy thử end-to-end cho thấy hệ thống không dừng ở mức nguyên mẫu rời rạc mà thực sự khép kín được vòng phát hiện → phản hồi với con người ở đúng điểm ra quyết định. Đây là tiền đề trực tiếp cho phần đánh giá định lượng ở chương tiếp theo, nơi từng mắt xích của vòng này được đo đạc và phân tích trên dữ liệu thật.

CHƯƠNG 5

ĐÁNH GIÁ VÀ KẾT LUẬN

Chương này trình bày phương pháp và kết quả đánh giá định lượng hệ thống trên một cụm Kubernetes thật, tập trung vào bốn câu hỏi: (i) hệ thống có phát hiện đúng các hành vi quét cổng và dịch chuyển ngang hay không; (ii) tỷ lệ báo động giả (*false positive rate*, FPR) trên lưu lượng hợp lệ ở mức nào; (iii) chi phí (*overhead*) mà tầng thu thập eBPF áp lên nhân Linux là bao nhiêu; và (iv) độ trễ của vòng phát hiện $\rightarrow$ phản hồi. Toàn bộ số liệu trong chương được đo bằng các kịch bản tự động (mục 5.1), không phải ước lượng lý thuyết.

5.1. Phương pháp đánh giá

5.1.1. Môi trường thực nghiệm

Thực nghiệm chạy trên một cụm Kubernetes thật gồm ba node (một node control-plane và hai node worker), hệ điều hành Ubuntu 24.04, nhân Linux phiên bản 6.8, runtime container containerd, mạng pod do CNI *Calico* đảm nhiệm (hỗ trợ NetworkPolicy). Tầng thu thập là một DaemonSet chạy chương trình eBPF *CO-RE* gắn kprobe vào hàm nhân `tcp_v4_connect`; mọi phép đo được thực hiện trên node worker-1, nơi đặt các pod mục tiêu (dịch vụ web cổng 80 và api cổng 8080).

5.1.2. Bộ công cụ đo đạc

Các kịch bản được đóng gói thành script có thể lắp lại (thư mục `scripts/eval/` trong kho `K8sSecDaP-soc`):

- `run-eval.sh` — sinh các pod nguồn mới, chạy kịch bản tấn công/hợp lệ, kiểm tra cảnh báo tương ứng trong nhật ký pipeline eBPF, xuất bảng kết quả (Markdown + CSV).
- `overhead-ebpf.sh` và `overhead-repeat.sh` — đo chi phí CPU nhân cho mỗi lời gọi `connect()` bằng cơ chế đếm thời gian chạy của chính nhân (`bpf_stats`); bản `-repeat` lặp lại nhiều chu kỳ để tính khoảng tin cậy (mục 5.3.1).
- `latency-report.sh` — truy vấn cơ sở dữ liệu sự cố để tính độ trễ vòng phát hiện $\rightarrow$ phản hồi (mục 5.3.2).

5.1.3. Đặc tả bộ phát hiện đo được

Qua khảo sát mã nguồn pipeline, luật phát hiện thực tế được xác định như sau: một địa chỉ nguồn bị gắn cờ `port_scan` khi *số lượng kết nối* của nó vượt ngưỡng $\tau = 40$ trong cửa sổ trượt kiểu *tumbling* dài 60 giây; cảnh báo `blast_radius` (dịch chuyển ngang) chỉ được tính cho nguồn đã vượt ngưỡng đó và đo số pod mà nguồn tiếp cận được. Như vậy

bộ phát hiện hoạt động dựa trên *tần suất kết nối theo nguồn*; đặc điểm này quyết định cách thiết kế kịch bản hợp lệ ở mục 5.2 và được phân tích kỹ ở mục 5.6.

5.1.4. Nguyên tắc tránh nhiễu thí nghiệm

Vì bộ phát hiện khóa theo địa chỉ IP nguồn và IP của pod bị tái sử dụng nhanh, một pod hợp lệ tạo ngay sau một pod quét có thể *thừa hưởng* bộ đếm kết nối của pod quét trong cùng cửa sổ 60 giây, gây dương tính giả *giả tạo*. Để loại bỏ hiện tượng này, harness (a) chạy các mẫu hợp lệ *trước* mọi kịch bản tấn công, trên một cửa sổ đã được “xả” (chờ 75 giây), và (b) cấp một pod nguồn riêng cho mỗi mẫu. Đây là một bài học phương pháp luận quan trọng khi đánh giá các bộ phát hiện khóa theo IP trong môi trường pod có vòng đời ngắn.

5.2. Kết quả phát hiện và tỷ lệ báo động giả

Bảng 5.1 tổng hợp kết quả. Bốn kịch bản tấn công đều được phát hiện (**độ phủ phát hiện 4/4**); sáu mẫu lưu lượng hợp lệ với số kết nối từ 15 đến 38 (dưới ngưỡng) đều không sinh cảnh báo (**FPR = 0/6**). Hai mốc biên xác nhận hiệu chỉnh ngưỡng: 38 kết nối không kích hoạt, 45 kết nối kích hoạt — khớp với ngưỡng $\tau = 40$. Kết quả nhất quán qua hai lần chạy độc lập.

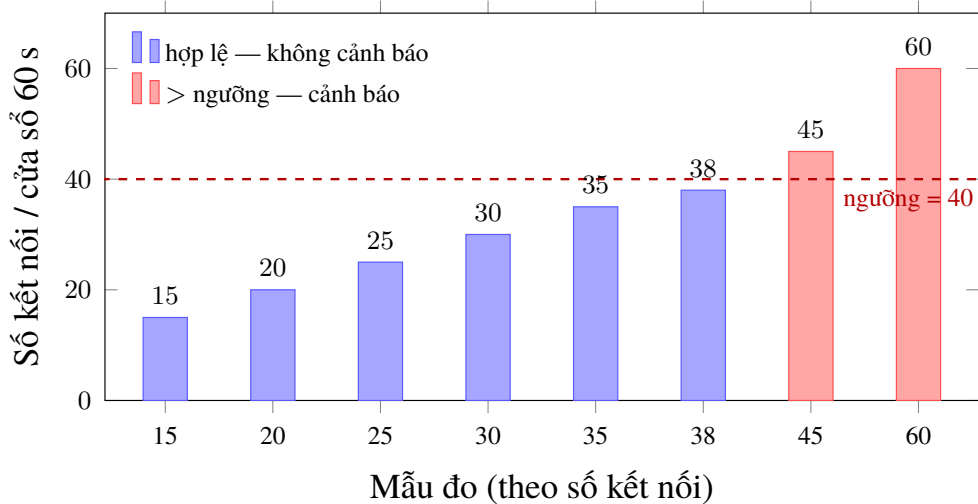
Bảng 5.1. Kết quả phát hiện trên cụm thật (node worker-1, nhân 6.8). Hai dòng cuối là minh họa giới hạn, không tính vào chỉ số chính.

Kịch bản	Loại	Cảnh báo kỳ vọng	Kết quả
Quét dọc (1 đích, 200 cổng)	tấn công	port_scan	Phát hiện
Quét ngang (6 pod, 30 cổng)	tấn công	blast_radius	Phát hiện
Quét đồn (1 đích, 1000 cổng)	tấn công	port_scan	Phát hiện
Biên trên (45 kết nối)	tấn công	port_scan	Phát hiện
Hợp lệ 15 kết nối	hợp lệ	(không)	Không cảnh báo
Hợp lệ 20 kết nối	hợp lệ	(không)	Không cảnh báo
Hợp lệ 25 kết nối	hợp lệ	(không)	Không cảnh báo
Hợp lệ 30 kết nối	hợp lệ	(không)	Không cảnh báo
Hợp lệ 35 kết nối	hợp lệ	(không)	Không cảnh báo
Biên dưới (38 kết nối)	hợp lệ	(không)	Không cảnh báo
<i>Quét chậm (50 cổng, 2s/cổng)</i>	<i>né tránh</i>	port_scan	<i>Bỏ sót (FN)</i>
<i>Hợp lệ “ồn” (60 kết nối)</i>	<i>giới hạn</i>	(không)	<i>Báo giả (FP)</i>

Độ phủ phát hiện: 4/4 kịch bản tấn công. **Tỷ lệ báo động giả:** 0/6 mẫu hợp lệ thực tế.

Hình 5.1 trực quan hóa quá trình hiệu chỉnh quanh ngưỡng: sáu mẫu hợp lệ (15–38 kết nối) đều nằm *dưới* ngưỡng 40 nên không sinh cảnh báo, trong khi mẫu tấn công ở biên

(45 kết nối) vượt ngưỡng và bị bắt. Cột 60 kết nối là trường hợp giới hạn “hợp lệ nhưng ồn” — một client hợp lệ tạo nhiều kết nối — vượt ngưỡng và do đó bị báo giả, minh họa đúng giới hạn của cách đếm theo *khối lượng* kết nối đã phân tích ở Chương 2 (bộ đếm khóa theo địa chỉ nguồn).



Hình 5.1. Hiệu chỉnh ngưỡng phát hiện (số liệu đo thật).

Đáng chú ý là *biên phân tách* quanh ngưỡng khá hẹp: mẫu hợp lệ cao nhất (38 kết nối) và mẫu tấn công biên (45 kết nối) chỉ cách nhau 7 kết nối, tức ngưỡng $\tau = 40$ nằm gần như chính giữa. Biên hẹp này là hệ quả trực tiếp của việc đếm theo *khối lượng* kết nối: một client hợp lệ “ồn” và một cuộc quét “nhẹ” có thể tạo cùng một lượng kết nối, nên không tồn tại một ngưỡng tĩnh nào tách sạch hai lớp. Quan sát này củng cố cho hướng cải tiến đổi khóa đếm sang *cặp (nguồn, cổng đích)* để đo số cổng phân biệt — một đại lượng tách hai lớp tốt hơn — được phân tích ở mục 5.6.

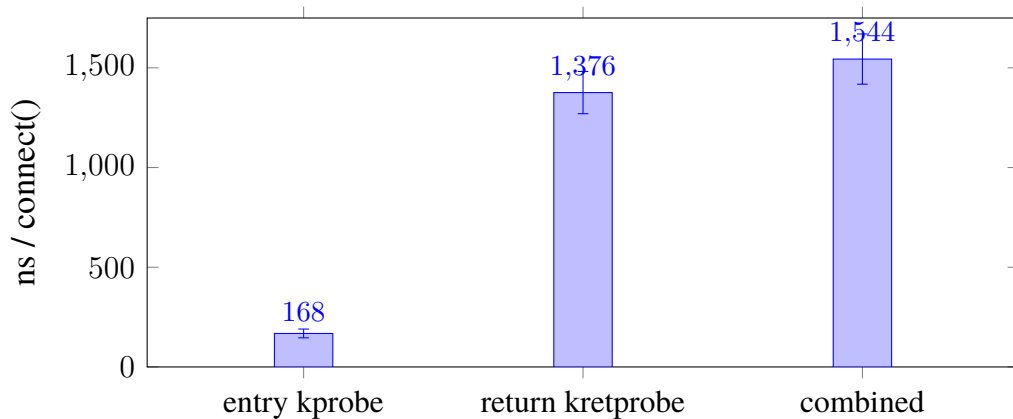
5.3. Đánh giá hiệu năng

5.3.1. Chi phí CPU nhân của tầng eBPF

Chi phí của hai chương trình eBPF (kprobe lúc vào và kretprobe lúc ra của `tcp_v4_connect`) được đo bằng cơ chế `bpf_stats` của nhân: nhân tự ghi tổng thời gian chạy và số lần chạy của từng chương trình. Phép đo lấy *hiệu* của cặp giá trị này trước và sau một đợt tải kết nối, nên loại được phần thời gian tích lũy trước khi bật bộ đếm; `bpftool` của host được gọi qua `nsenter` từ chính pod thu thập (đặc quyền, `hostPID`) nên *không* cần gỡ bỏ collector và *không* ảnh hưởng tới GitOps. Kết quả lấy trung bình trên $n = 10$ chu kỳ, kèm khoảng tin cậy 95% theo phân phối Student-*t* (Bảng 5.2, Hình 5.2).

Bảng 5.2. Chi phí CPU nhân cho mỗi `connect()` (trung bình $\pm$ CI 95%, $n = 10$ chu kỳ, ≈ 3000 kết nối/chu kỳ).

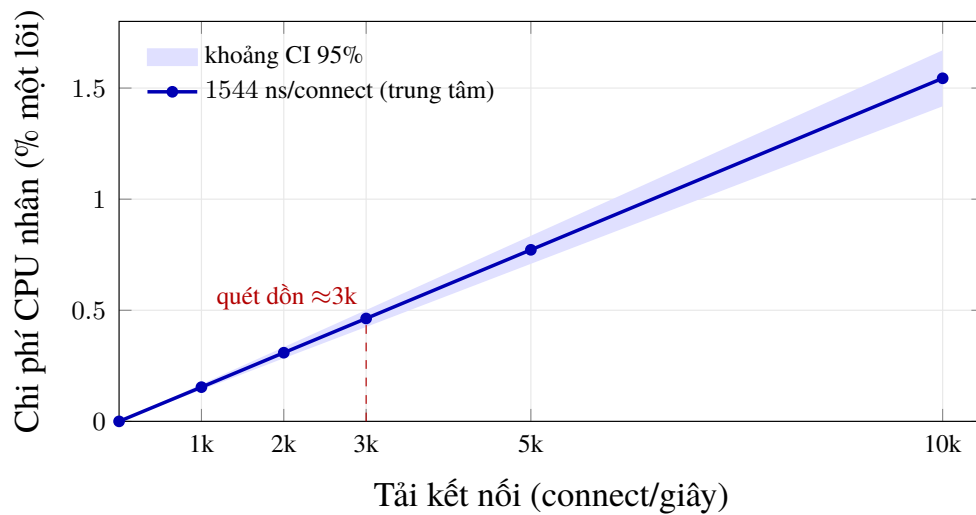
Chương trình eBPF	Chi phí / <code>connect()</code>
kprobe lúc vào (entry)	168 ± 22 ns
kretprobe lúc ra (return)	1376 ± 106 ns
Tổng cộng mỗi <code>connect()</code>	1544 ± 126 ns



Hình 5.2. Chi phí CPU nhân cho mỗi `connect()`.

Phần lớn chi phí nằm ở nhánh trả về (kretprobe) do đây là nơi chương trình dự trữ ô đệm vòng, tra cứu LPM_TRIE phân loại CIDR và đẩy sự kiện lên không gian người dùng. Quy đổi sang mức tải thực tế: ở 1000 kết nối/giây, tổng chi phí là $1544 \text{ ns} \times 1000 = 1,54 \text{ ms/giây}$, tức $\approx 0,15\%$ một lõi CPU; ngay cả ở mức 3000 kết nối/giây (đợt quét dồn) cũng chỉ $\approx 0,46\%$ một lõi. Về phía không gian người dùng, tiến trình collector chiếm ≈ 1 mCPU và 3 MiB bộ nhớ (mức sàn phân giải của `kubect1 top`), không đổi giữa lúc rảnh và lúc chịu tải. Như vậy tầng thu thập eBPF có chi phí gần như không đáng kể, phù hợp với đặc tính của eBPF được nêu trong tài liệu [8].

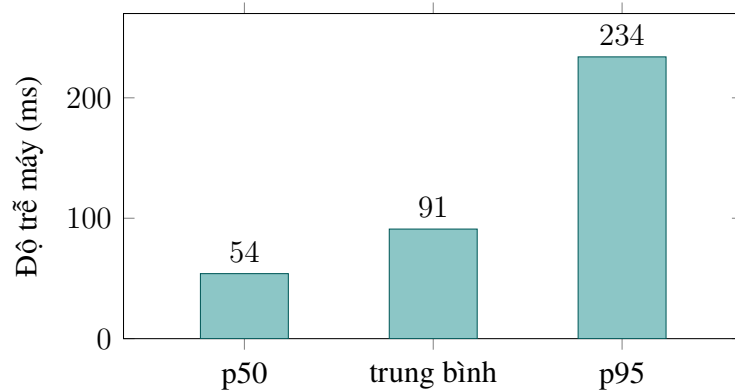
Từ chi phí đo được mỗi `connect()`, có thể *suy diễn giải tích* mức tải mà tầng eBPF chịu được trước khi chi phí trở nên đáng kể: Hình 5.3 chiếu tuyến tính con số 1544 ± 126 ns sang phần trăm một lõi CPU theo mức tải kết nối. Đây là đường cong *suy diễn* (không phải đo đa-mức-tải): vì mỗi `connect()` đi qua đúng một lần cặp kprobe/kretprobe, tổng chi phí tỉ lệ tuyến tính với số kết nối mỗi giây. Ngay ở mức 10 000 kết nối/giây – cao hơn nhiều so với đợt quét dồn ≈ 3000 quan sát được – chi phí cũng chỉ $\approx 1,5\%$ một lõi. Việc *đo thực nghiệm* đường cong này ở nhiều mức tải trên cụm được để ở hướng phát triển (mục 5.6).



Hình 5.3. Chi phí CPU nhân theo mức tải kết nối (suy diễn).

5.3.2. Độ trễ vòng phát hiện → phản hồi

Độ trễ “máy” của SOC — từ lúc nhận cảnh báo đến lúc áp NetworkPolicy cách ly, *không kể* thời gian con người duyệt — có trung bình ≈ 91 ms, trung vị (p50) 54 ms và phân vị 95 (p95) 234 ms. Con số này cho thấy một khi quyết định được duyệt, hệ thống chuyển từ cảnh báo sang thực thi chính sách gần như tức thời; độ trễ tổng thể đầu-cuối bị chi phối bởi bước con người trong vòng *human-in-the-loop*, đúng như thiết kế. Hình 5.4 trực quan hóa ba mốc thống kê này. Khoảng cách giữa trung vị (54 ms) và p95 (234 ms) phản ánh *đuôi phân phối* dài — điển hình cho các hệ phân tán trải qua tranh chấp lập lịch hoặc trễ mạng nội cụm ở một số ít sự cố — chứ không phải mức trễ thường gặp. Việc dựng *histogram/CDF* đầy đủ của phân phối độ trễ đòi hỏi thu nhiều mẫu sự cố hơn trên cụm và được để ở hướng phát triển; ở đây báo cáo chỉ trình bày các phân vị đã đo được, không nội suy phân phối.



Hình 5.4. Độ trễ máy của vòng phản hồi.

5.4. So sánh với các công cụ liên quan

Bảng 5.3 đối chiếu hệ thống của đề án với bốn công cụ an ninh runtime tiêu biểu cho Kubernetes. Điểm chung là đều khai thác eBPF/LSM để quan sát ở kernel-space; điểm khác biệt nằm ở trọng tâm: các công cụ công nghiệp thiên về quy tắc trên syscall (Falco [17]) hoặc thực thi tổng quát (Tetragon [10], KubeArmor [1], Cilium [4]), trong khi hệ thống của đề án tập trung hẹp vào bài toán quét cổng/dịch chuyển ngang và *lượng hoá* bằng cấu trúc dữ liệu luồng (Count-Min Sketch, Tarjan, BFS) — đây là đóng góp mang tính học thuật mà các công cụ trên không đặt làm trọng tâm.

Bảng 5.3. So sánh định tính với các công cụ an ninh runtime cho Kubernetes.

Công cụ	Cơ chế nhân	Trọng tâm / khác biệt so với đề án
Falco [17]	eBPF trên syscall	Phát hiện theo <i>luật</i> trên syscall; không có cấu trúc luồng lượng hoá quét cổng
Tetragon [10]	eBPF (tracepoint/LSM)	Mạnh về <i>thực thi</i> (kill/override) kernel-space; ít phân tích đồ thị dịch chuyển ngang
Cilium [4]	eBPF datapath (CNI)	Thực thi NetworkPolicy L3–L7 trong datapath; là tầng <i>mạng</i> , không phải bộ phát hiện chuyên biệt
KubeArmor [1]	BPF-LSM	Thực thi chính sách hành vi process/file/network; không nhắm bài toán quét cổng
Đề án này	eBPF kprobe	Phát hiện quét cổng/dịch chuyển ngang bằng <i>cấu trúc dữ liệu luồng</i> + phản hồi NetworkPolicy có người duyệt

5.5. Mô hình tin cậy và giới hạn an ninh của bộ thu thập

Vì bộ thu thập chạy dưới dạng DaemonSet đặc quyền và nạp mã eBPF vào nhân của mỗi node, tôi xem nó là thành phần có quyền cao hơn hẳn ứng dụng thường và đặt nó vào một ranh giới tin cậy riêng. Mục này phân tích mô hình tin cậy đó cùng các giới hạn an ninh kèm theo, vì chính việc can thiệp vào không gian nhân đặt ra câu hỏi: nếu một pod bị tấn công thì sao?

Phân tách quyền theo pod. Quyền đặc quyền nằm ở *pod thu thập*, không ở mọi pod ứng dụng. Nếu một pod ứng dụng bị chiếm, kẻ tấn công chỉ tạo được lưu lượng xấu (quét cổng, dịch chuyển ngang) – tức làm *tăng* khả năng bị hệ thống phát hiện – chứ không tự có quyền chạm vào không gian nhân: chương trình eBPF đã được tôi viết sẵn và gắn từ bộ thu thập, còn logic ở nhân chỉ *nhận* sự kiện theo luồng connect() (Mã nguồn 4.4, Chương 4), không nhận lệnh từ pod ứng dụng. Như vậy chiếm một pod ứng dụng làm tăng xác suất bị phát hiện, không biến thành quyền điều khiển cảm biến.

Ranh giới tin cậy thật sự. Rủi ro đáng kể hơn là khi kẻ tấn công chiếm được

chính pod thu thập hoặc thoát ra node; khi đó họ có thể tác động đến cảm biến ở mức nhân. Vì vậy tôi coi bộ thu thập là một *trust boundary* riêng, không cùng mức tin cậy với workload thường, và chủ trương làm cứng theo nguyên tắc đặc quyền tối thiểu: chạy trên node pool riêng tách khỏi workload nhạy cảm; dùng ảnh bất biến, ký số, chỉ cập nhật qua GitOps/CI; thay `privileged: true` bằng tập *capability* tối thiểu (BPF, PERFMON, NET_ADMIN), gán BTF chỉ đọc và bật `readOnlyRootFilesystem`; không expose service và không cấp token RBAC rộng cho pod thu thập, kèm profile seccomp/AppArmor.

Đối chứng trong cùng pod. Tôi đã áp nguyên tắc này ngay trong chính DaemonSet: trong khi collector buộc phải đặc quyền, container sidecar alert-bridge cạnh nó chạy theo chuẩn an toàn (`runAsNonRoot`, `readOnlyRootFilesystem`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`) vì nó chỉ đọc tệp cảnh báo và đẩy lên NATS. Đây là mẫu hình “đặc quyền tối thiểu theo từng container”: chỉ thành phần thật sự cần chạm nhân mới được cấp quyền cao.

Giới hạn an ninh chưa khắc phục. Tôi thừa nhận một điểm chưa hoàn thiện: bộ thu thập hiện vẫn chạy `privileged: true` thay vì tập *capability* tối thiểu nói trên – manifest đã ghi chú hướng siết nhưng tôi chưa hiện thực. Đây là việc cần làm trước khi đưa hệ thống vào môi trường sản xuất, và tôi xếp nó vào hướng phát triển.

5.6. Giới hạn và hướng phát triển

Tôi ghi nhận thẳng thắn các giới hạn sau của hệ thống, kèm hướng khắc phục:

- **Điểm mù cửa sổ tumbling (né tránh).** Một cuộc quét *chậm* giữ số kết nối dưới ngưỡng trong mỗi cửa sổ 60 giây sẽ không bị phát hiện (dòng “quét chậm” trong Bảng 5.1). Hướng khắc phục: thay cửa sổ tumbling bằng *sliding-window* CMS để bắt các luồng vắt biên cửa sổ.
- **Phát hiện theo lượng kết nối, không theo số cổng phân biệt (báo giả).** Vì bộ đếm khóa theo IP nguồn và cộng dồn *mọi* kết nối, một client hợp lệ nhưng “ồn” (vượt 40 kết nối/cửa sổ tới ít cổng) vẫn bị gán cờ (dòng “hợp lệ ồn”). Hướng khắc phục: đổi khóa CMS sang *cặp* (nguồn, cổng đích) để đếm *số cổng phân biệt* — đúng bản chất của hành vi quét cổng.
- **Phạm vi quan sát của kprobe.** kprobe trên `tcp_v4_connect` chỉ bắt lời gọi `connect()`, nên quét kiểu raw-SYN (nmap `-sS` với quyền root) đi vòng qua được; cần `-sT` (connect scan) để nằm trong tầm quan sát. Mở rộng sang XDP/tc sẽ bao phủ cả raw-SYN.
- **Phạm vi giao thức và quy mô.** Hiện chỉ hỗ trợ IPv4 (`tcp_v4_connect`); ngưỡng τ là hằng số tĩnh, chưa thích nghi theo đặc tính tải; số liệu đo trên một node, chưa hợp nhất sketch đa node.

5.6.1. Khó khăn triển khai và bài học rút ra

Bên cạnh các giới hạn về mặt thuật toán ở trên, quá trình triển khai trên cụm thật để lại một số bài học mà tôi ghi lại trung thực, vì chúng định hình các quyết định kỹ thuật của đồ án:

- **Đổi CNI từ Flannel sang Calico để thực thi được chính sách.** Ban đầu tôi chọn Flannel vì nhẹ và dễ cài, nhưng khi kiểm thử bước phản hồi thì phát hiện NetworkPolicy cách ly *không có tác dụng*: Flannel chỉ làm mạng overlay, không thực thi NetworkPolicy. Tôi phải chuyển CNI sang Calico (Tigera Operator) để có khả năng thực thi chính sách L3/L4 – đây là lý do toàn bộ đánh giá được thực hiện trên Calico. Bài học: với bài toán *ngăn chặn*, việc chọn CNI có thực thi chính sách là điều kiện tiên quyết, không phải lựa chọn tối ưu hóa.
- **Ngữ nghĩa cộng dồn của NetworkPolicy.** Bản nháp cách ly đầu tiên tôi sinh ra theo kiểu “cho phép mọi nguồn *trừ* kẻ tấn công” ở chiều vào; khi kiểm thử mới thấy nó bị một luật cho-phép rộng khác vô hiệu hóa, vì NetworkPolicy mang ngữ nghĩa *cộng dồn*. Tôi đổi sang *chặn toàn bộ chiều ra của chính pod tấn công* (egress-deny), cách này không policy nào ghi đè được (Chương 4).
- **Nhiều do vòng đời pod ngắn.** Vì bộ phát hiện khóa theo IP nguồn mà IP pod bị tái sử dụng nhanh, một pod hợp lệ tạo ngay sau một pod quét có thể *thừa hưởng* bộ đếm của pod quét trong cùng cửa sổ, gây dương tính giả giả tạo. Tôi khắc phục bằng cách chạy mẫu hợp lệ trên cửa sổ đã “xả” và cấp pod nguồn riêng cho mỗi mẫu (mục 5.1).
- **Khác miền đồng hồ khi đo độ trễ.** Sự kiện eBPF mang mốc thời gian *đồng hồ đơn điệu* của nhân (bpf_ktime_get_ns), trong khi SOC ghi nhận theo *đồng hồ tường*; hai miền không trừ trực tiếp được. Do đó tôi chỉ đo độ trễ “máy” phía SOC (mục 5.3.2) thay vì độ trễ đầu-cuối xuyên miền đồng hồ.
- **Đo chi phí nhân mà không phá vỡ GitOps.** Để đo chi phí eBPF, thay vì gỡ bộ thu thập (sẽ bị ArgoCD tự khôi phục), tôi bật bpf_stats của nhân và lấy *hiệu số* đếm trước/sau một đợt tải qua bpftool của host – đo tại chỗ, không can thiệp vào vòng đồng bộ (mục 5.3.1).
- **Hai đường dữ liệu quan trắc dễ gây hiểu nhầm.** Console/Reports đọc trực tiếp PostgreSQL nên luôn đầy đủ sự cố tích lũy, trong khi dashboard Grafana lại thường *trống*. Truy nguyên cho thấy đó là hai nguồn khác nhau: bảng *Alerts Overview* đọc nhật ký Loki theo cửa sổ mặc định 30 phút (không có tấn công gần đây thì trống), còn bảng *Incidents & Actions* đọc bộ đếm Prometheus vốn *không được thu thập* – vì cụm dùng Prometheus Operator khám phá mục tiêu qua ServiceMonitor chứ không theo chú thích prometheus.io/scrape, và các bộ đếm còn bị đặt lại 0 mỗi lần pod khởi động lại. Bài học: tách bạch rõ *hệ thống ghi nhận bên vũng* (cơ sở dữ liệu) với *lớp quan trắc theo cửa sổ* (Grafana); cách bổ sung ServiceMonitor và nói cửa sổ thời

gian để dashboard hiển thị đúng được nêu ở Phụ lục F.6.

5.7. Kết luận

Đồ án đã hiện thực và *kiểm chứng trên cụm Kubernetes* thật một hệ thống phát hiện và ngăn chặn hành vi quét cổng/dịch chuyển ngang dựa trên eBPF kết hợp cấu trúc dữ liệu luồng. Trên trục phát hiện, hệ thống đạt độ phủ 4/4 và FPR 0/6 trên các mẫu hợp lệ thực tế, với ngưỡng được hiệu chỉnh rõ ràng. Trên trục hiệu năng, tầng thu thập eBPF chỉ tốn ≈ 1544 ns CPU nhân mỗi `connect()` (dưới 0,5% một lõi ngay cả khi chịu tải quét), và vòng phản hồi máy đạt p95 234 ms. Các giới hạn được nhận diện và đo lường tường minh (né tránh cửa sổ, báo giả theo lượng kết nối) cùng hướng phát triển cụ thể (sliding-window CMS, đổi khóa đếm theo cổng phân biệt, mở rộng XDP/tc và IPv6) tạo nền cho các bước tiếp theo.

TÀI LIỆU THAM KHẢO

- [1] Accuknox / CNCF Sandbox. *KubeArmor: Runtime Security Enforcement System for Kubernetes*. <https://kubearmor.io>. Thực thi chính sách hành vi (process/file/network) ở kernel qua LSM (BPF-LSM/AppArmor/SELinux). 2024.
- [2] Austin Appleby. *MurmurHash3*. <https://github.com/aappleby/smhasher>. Non-cryptographic hash function used in Count-Min Sketch implementation. 2011.
- [3] Davide Berardi, Saverio Giallorenzo, Jacopo Mauro, Andrea Melis, Fabrizio Montesi, and Marco Prandini. “Microservice security: a systematic literature review”. In: *PeerJ Computer Science* 8 (2022). Systematic literature review of 290 publications, e779. DOI: [10.7717/peerj-cs.779](https://doi.org/10.7717/peerj-cs.779).
- [4] Cilium Authors / CNCF. *Cilium: eBPF-based Networking, Observability and Security*. <https://cilium.io>. CNI dùng eBPF; thực thi NetworkPolicy L3–L7 trực tiếp trong datapath kernel. 2024.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd. MIT Press, 2009. ISBN: 978-0-262-03384-8.
- [6] Graham Cormode and S. Muthukrishnan. “An Improved Data Stream Summary: The Count-Min Sketch and its Applications”. In: *Journal of Algorithms* 55.1 (2005), pp. 58–75. DOI: [10.1016/j.jalgor.2003.12.001](https://doi.org/10.1016/j.jalgor.2003.12.001).
- [7] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Strategy Pattern (Chapter 5.9). Addison-Wesley, 1995. ISBN: 978-0-201-63361-0.
- [8] Brendan Gregg. *BPF Performance Tools: Linux System and Application Observability*. 2019.
- [9] Nurman Rasyid Panusunan Hutasuhut, Mochamad Gani Amri, and Rizal Fathoni Aji. “Security Gap in Microservices: A Systematic Literature Review”. In: *International Journal of Advanced Computer Science and Applications (IJACSA)* 15.12 (2024). SLR of 487 papers (87 selected via snowball), focusing on post-2020 vulnerabilities. DOI: [10.14569/IJACSA.2024.0151218](https://doi.org/10.14569/IJACSA.2024.0151218).
- [10] Isovalent / Cilium Authors. *Tetragon: eBPF-based Security Observability and Runtime Enforcement*. <https://tetragon.io>. Quan sát và thực thi (kill/override) ở kernel-space bằng eBPF, gắn vào tracepoint/LSM hook. 2024.
- [11] Vern Paxson. “Bro: A System for Detecting Network Intruders in Real-Time”. In: *Computer Networks* 31.23–24 (1999), pp. 2435–2463. DOI: [10.1016/S1389-1286\(99\)00112-7](https://doi.org/10.1016/S1389-1286(99)00112-7).

- [12] Anelis Pereira-Vale, Eduardo B. Fernández, Raúl Monge, Hernán Astudillo, and Gastón Márquez. “Security in microservice-based systems: A Multivocal literature review”. In: *Computers & Security* 103 (2021). Multivocal review combining 36 academic + 34 grey-literature sources, p. 102200. DOI: [10.1016/j.cose.2021.102200](https://doi.org/10.1016/j.cose.2021.102200).
- [13] Red Hat, Inc. *The State of Kubernetes Security Report, 2024 Edition*. Tech. rep. Industry survey of 600 DevOps, engineering, and security professionals. Red Hat, 2024. URL: <https://www.redhat.com/en/engage/state-kubernetes-security-report-2024>.
- [14] Martin Roesch. “Snort – Lightweight Intrusion Detection for Networks”. In: *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*. Seattle, Washington, USA: USENIX Association, 1999, pp. 229–238.
- [15] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. *Zero Trust Architecture*. Tech. rep. SP 800-207. National Institute of Standards and Technology (NIST), 2020. DOI: [10.6028/NIST.SP.800-207](https://doi.org/10.6028/NIST.SP.800-207).
- [16] Robert Tarjan. “Depth-First Search and Linear Graph Algorithms”. In: *SIAM Journal on Computing* 1.2 (1972), pp. 146–160. DOI: [10.1137/0201010](https://doi.org/10.1137/0201010).
- [17] The Falco Authors / CNCF. *Falco: Cloud Native Runtime Security*. <https://falco.org>. Dự án tốt nghiệp CNCF; phát hiện bất thường runtime bằng quy tắc trên syscall qua eBPF/kernel module. 2024.
- [18] Wiz Research. *Kubernetes Security Report 2025*. Tech. rep. Industry telemetry on cloud-native attack patterns observed across managed Kubernetes environments. Wiz, 2025. URL: <https://www.wiz.io/reports/kubernetes-security-report-2025>.

PHỤ LỤC

CHƯƠNG A

MÃ NGUỒN LỖI: COUNT-MIN SKETCH

Count-Min Sketch là cấu trúc đếm tần suất xấp xỉ dùng trong pipeline để phát hiện quét cổng (kho K8sSecDaP-pipeline, libdsa/src/frequency/count_min_sketch.cpp). Cấu trúc gồm một bảng $d \times w$ ô đếm và d hàm băm độc lập; record tăng một ô trên mỗi hàng, còn estimate trả về giá trị nhỏ nhất trong d ô tương ứng – đảm bảo ước lượng không bao giờ thấp hơn tần suất thật. Cơ sở lý thuyết về sai số ϵ - δ đã trình bày ở Chương 2.

```
1  // table_ : d hang x w cot (uint64_t); seeds_ : d hat giống
   bam doc lap.
2  void CountMinSketch::record(const void* key, size_t key_len
   ) {
3      for (size_t i = 0; i < depth_; ++i) {
4          uint32_t col = hash(key, key_len, seeds_[i]) %
               width_;
5          table_[i][col]++;                // tang 1 o
               tren moi hang
6      }
7  }
8
9  uint64_t CountMinSketch::estimate(const void* key, size_t
   key_len) const {
10     uint64_t min_val = std::numeric_limits<uint64_t>::max()
        ;
11     for (size_t i = 0; i < depth_; ++i) {
12         uint32_t col = hash(key, key_len, seeds_[i]) %
             width_;
13         min_val = std::min(min_val, table_[i][col]); //
               lay MIN -> chan tren nhieu
14     }
15     return min_val;
16 }
17
18 void CountMinSketch::reset() {                // dat lai
   khi het cua so tumbling
19     for (auto& row : table_) std::fill(row.begin(), row.end
        (), 0);
20 }
21
```

```

22 size_t CountMinSketch::memory_usage() const {    // bộ nhớ
    không đổi theo số khóa
23     return depth_ * width_ * sizeof(uint64_t) + seeds_.size
        () * sizeof(uint32_t);
24 }

```

Mã nguồn A.1. Lỗi record/estimate/reset của Count-Min Sketch.

Với cấu hình của đồ án ($d = 5$, $w = 2048$), bộ nhớ là $5 \times 2048 \times 8 \approx 80$ KB và *không đổi* dù số địa chỉ nguồn phân biệt tăng lên – đây chính là ưu thế của sketch so với đếm chính xác bằng HashMap (bộ nhớ tuyến tính theo số khóa).

CHƯƠNG B

MÃ NGUỒN LỖI: TARJAN SCC VÀ BFS BÁN KÍNH ẢNH HƯỞNG

Hai giải thuật đồ thị của pipeline (kho K8sSecDaP-pipeline, libdsa/src/graph/) phục vụ phát hiện dịch chuyển ngang: Tarjan tìm thành phần liên thông mạnh (chu trình giao tiếp bất thường), còn BFS đo bán kính ảnh hưởng (số pod một nguồn tiếp cận được trong k bước). Cơ sở lý thuyết đã trình bày ở Chương 2; phần này trích phần lõi của hiện thực.

B.1. Tarjan SCC (hiện thực lặp)

Để tránh tràn ngăn xếp lời gọi trên đồ thị lớn, Tarjan được hiện thực bằng DFS *lặp* (dùng ngăn xếp tường minh) thay vì đệ quy; mỗi đỉnh giữ chỉ số khám phá `disc` và liên kết thấp `low`, một SCC được kết xuất khi `low[u] == disc[u]`.

```
1 // Khi quay lui từ đỉnh con v về cha u:
2 low[cur.u] = std::min(low[cur.u], low[v]);
3 // Khi gap đỉnh v đã tham va con tren stack (canh ngược):
4 if (on_stack[v]) low[cur.u] = std::min(low[cur.u], disc[v])
5 ;
6 // u la goc của một SCC:
7 if (low[cur.u] == disc[cur.u]) {
8     Component scc;
9     uint32_t w;
10    do { w = stk.top(); stk.pop(); on_stack[w] = false; scc
11        .push_back(w); }
12    while (w != cur.u);
13    components.push_back(std::move(scc));
14 }
```

Mã nguồn B.1. Điểm mấu chốt của Tarjan: cập nhật low-link và kết xuất SCC (tarjan.cpp).

B.2. BFS bán kính ảnh hưởng (k-hop)

BFS duyệt theo lớp từ địa chỉ nguồn, dừng ở độ sâu `max_hops`; tập đỉnh tới được chính là *bán kính ảnh hưởng* (blast radius) của nguồn – thước đo mức độ lan rộng nếu nguồn đó tiếp tục dịch chuyển ngang.

```
1 ReachabilityResult BFSReachability::reachable_within(
2     const DirectedGraph& g, uint32_t source, int max_hops)
3     {
4         std::queue<uint32_t> q; q.push(source);
```

```

4      std::unordered_map<uint32_t,int> dist; dist[source] =
        0;
5      while (!q.empty()) {
6          uint32_t u = q.front(); q.pop();
7          int d = dist[u];
8          if (d >= max_hops) continue;           // không mo
            rong qua k buoc
9          for (uint32_t v : g.neighbors(u))
10             if (dist.find(v) == dist.end()) { dist[v] = d +
                1; q.push(v); }
11     }
12     return dist;    // tap dinh toi duoc + so buoc -> ban
        kinh anh huong
13 }

```

Mã nguồn B.2. Duyệt BFS theo lớp, dừng ở k bước (bfs.cpp).

CHƯƠNG C

MÃ NGUỒN LỖI: CHƯƠNG TRÌNH EBPF NHÂN VÀ BỘ ĐỌC TẦNG NGƯỜI DÙNG

Phụ lục này trích dẫn mã nguồn cốt lõi của bộ thu thập (kho K8sSecDaP-collector). Phần thân của kretprobe, bảng nạp CIDR và phần khai báo DaemonSet đã được trình bày trong Chương 4; ở đây bổ sung phần *khai báo ba BPF map* và kprobe lúc vào, cùng trình tự *nạp/gắn* ở tầng người dùng.

C.1. Khai báo BPF map và kprobe lúc vào (tcp_connect.bpf.c)

```
1  struct network_event_t {                                // phải trung khớp
    common.h o tang user
2      __u32 src_ip; __u32 dst_ip; __u16 dst_port; __u32 pid;
        __u64 timestamp_ns;
3  };
4  struct lpm_key { __u32 prefixlen; __u32 addr; };
5
6  // LPM Trie: tien to CIDR -> nhan (1=pod, 2=svc, 3=node, 0=
    external).
7  struct {
8      __uint(type, BPF_MAP_TYPE_LPM_TRIE);
9      __type(key, struct lpm_key); __type(value, __u32);
10     __uint(max_entries, 1024);
11     __uint(map_flags, BPF_F_NO_PREALLOC);    // BAT BUOC cho
        LPM_TRIE
12 } cidr_map SEC(".maps");
13
14 // Ring buffer truyền su kien len user space (256 KiB, page
    -aligned).
15 struct {
16     __uint(type, BPF_MAP_TYPE_RINGBUF);
17     __uint(max_entries, 256 * 1024);
18 } events SEC(".maps");
19
20 // Lưu con tro sock giữa kprobe (entry) và kretprobe (
    return), khoa theo pid_tgid.
21 struct {
22     __uint(type, BPF_MAP_TYPE_HASH);
23     __type(key, __u64); __type(value, struct sock *);
```

```

24     __uint(max_entries, 4096);
25 } sock_store SEC(".maps");
26
27 SEC("kprobe/tcp_v4_connect")
28 int BPF_KPROBE(tcp_v4_connect_entry, struct sock *sk)
29 {
30     __u64 pid_tgid = bpf_get_current_pid_tgid();
31     bpf_map_update_elem(&sock_store, &pid_tgid, &sk,
32                         BPF_ANY);
33     return 0;
34 }
35 char LICENSE[] SEC("license") = "GPL";

```

Mã nguồn C.1. Ba BPF map và kprobe lúc vào (tcp_v4_connect).

C.2. Trình tự nạp và gắn ở tầng người dùng (collector.cpp)

```

1  struct bpf_object *obj = bpf_object__open_file("tcp_connect
    .bpf.o", nullptr);
2  bpf_object__load(obj); //
    nạp BPF vào kernel
3  auto *kp = bpf_object__find_program_by_name(obj, "
    tcp_v4_connect_entry");
4  auto *krp = bpf_object__find_program_by_name(obj, "
    tcp_v4_connect_return");
5  bpf_program__attach(kp); //
    gắn kprobe
6  bpf_program__attach(krp); //
    gắn kretprobe
7  populate_cidr_map(bpf_map__fd(bpf_object__find_map_by_name(
    obj, "cidr_map")));
8  auto *rb = ring_buffer__new(bpf_map__fd(events_map),
    handle_event, nullptr, nullptr);
9  while (!exiting) //
    vòng sự kiện chính
10     ring_buffer__poll(rb, 100 /* timeout_ms */);

```

Mã nguồn C.2. Nạp tệp đối tượng BPF, gắn probe, vào vòng đọc ring buffer.

Hàm callback `handle_event` đổi địa chỉ nhị phân sang chuỗi chấm-thập phân và in một dòng JSON cho mỗi sự kiện; bộ đệm vòng (BPF_MAP_TYPE_RINGBUF) được chọn thay cho perf buffer vì cho thông lượng cao hơn, không trùng lặp và giữ đúng thứ tự sự

kiện giữa các CPU.

CHƯƠNG D

MÃ NGUỒN LỖI: CONSOLE WEB (FLASK + HTMX)

Console web (kho K8sSecDaP-soc, dịch vụ web-console) là một lớp giao diện mỏng đặt trước REST API của dịch vụ sự cố. Mọi lời gọi xuống API đều kèm token admin phía máy chủ (RBAC_ADMIN_TOKEN); trình duyệt chỉ xác thực bằng Basic Auth và không bao giờ giữ token API.

```
1 def _api(method, path, **kwargs):
2     """Goi REST API của incident-service voi tu cach SOC
        admin token."""
3     headers = kwargs.pop("headers", {})
4     headers["Authorization"] = f"Bearer {ADMIN_TOKEN}"
5     return requests.request(method, INCIDENT_API + path,
6                             headers=headers,
7                             timeout=REQUEST_TIMEOUT, **
8                             kwargs)
9
10 @app.get("/")
11 @_require_basic_auth
12 def index():
13     incidents = _api("GET", "/api/v1/incidents?limit=50").
14         json() or []
15     actions = _api("GET", "/api/v1/actions?status=pending
16         &limit=50").json() or []
17     return render_template("incidents.html", incidents=
18         incidents, actions=actions)
19
20 @app.get("/incidents/<int:inc_id>")
21 @_require_basic_auth
22 def incident_detail(inc_id):
23     inc = _api("GET", f"/api/v1/incidents/{inc_id}").
24         json()
25     audit = _api("GET", f"/api/v1/incidents/{inc_id}/
26         audit?limit=100").json() or []
27     actions = _api("GET", "/api/v1/actions?limit=200").json
28         () or []
29     related = [a for a in actions if a.get("incident_id")
30         == inc_id]
31     return render_template("incident_detail.html", incident
32         =inc,
```

```

23             actions=related, audit=audit)
24
25 @app.post("/actions/<int:action_id>/approve")    # HTMX:
           thay 1 hàng, không tải lại trang
26 @_require_basic_auth
27 def approve_action(action_id):
28     resp = _api("POST", f"/api/v1/actions/{action_id}/
           approve")
29     return _htmx_action_row(resp.json())

```

Mã nguồn D.1. Các route nghiệp vụ chính của console (app.py).

Việc phê duyệt/bác bỏ dùng HTMX để thay *một hàng* hành động trong bảng mà không tải lại toàn trang (`_action_row.html`), giúp thao tác duyệt phản hồi nhanh và liền mạch cho người vận hành.

CHƯƠNG E

DDL VÀ TRUY VẤN MẪU CỦA POSTGRESQL

Lược đồ dữ liệu của dịch vụ sự cố (kho K8sSecDaP-soc) được khai báo trong các tệp migration 001_init.sql ...004_incident_pod_context.sql, nhúng vào nhện phân và áp tự động (idempotent) khi khởi động. Phần dưới trích DDL khởi tạo bốn bảng cột lỗi và một số chỉ mục; sơ đồ quan hệ thực thể (ERD) tương ứng đã trình bày ở Chương 3.

```
1 CREATE TABLE IF NOT EXISTS incidents (
2     id                BIGSERIAL PRIMARY KEY,
3     incident_key       TEXT NOT NULL,           -- type
4     /source: khoa khu trung lap
5     type              TEXT NOT NULL,
6     source_ip         TEXT NOT NULL,
7     severity          TEXT NOT NULL DEFAULT 'medium',
8     status            TEXT NOT NULL DEFAULT 'new', -- new/
9     /ack/resolved/suppressed
10    nodes_observed     TEXT[] NOT NULL DEFAULT '{}',
11    first_seen_ns      BIGINT NOT NULL,
12    last_seen_ns       BIGINT NOT NULL,
13    occurrence_count   INT NOT NULL DEFAULT 1,
14    blast_radius       JSONB,
15    tags               TEXT[] NOT NULL DEFAULT '{}',
16    cross_node         BOOLEAN NOT NULL DEFAULT FALSE,
17    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
18    updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
19    UNIQUE (incident_key)
20 );
21
22 CREATE INDEX IF NOT EXISTS idx_incidents_status ON
23 incidents (status);
24
25 CREATE INDEX IF NOT EXISTS idx_incidents_last_seen ON
26 incidents (last_seen_ns DESC);
27
28 CREATE TABLE IF NOT EXISTS alerts (
29     id                BIGSERIAL PRIMARY KEY,
30     incident_id       BIGINT REFERENCES incidents(id) ON DELETE
31     CASCADE,
32     alert_json        JSONB NOT NULL,
33     received_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
34 );
```

```

29 CREATE TABLE IF NOT EXISTS actions (
30     id          BIGSERIAL PRIMARY KEY,
31     incident_id BIGINT NOT NULL REFERENCES incidents(id) ON
        DELETE CASCADE,
32     type        TEXT NOT NULL,                -- vd:
        quarantine_pod
33     status      TEXT NOT NULL DEFAULT 'pending', --
        pending/approved/rejected/executed/...
34     yaml_draft  TEXT NOT NULL,
35     approved_by TEXT, approved_at TIMESTAMPTZ,
36     executed_at TIMESTAMPTZ, error TEXT,
37     expires_at  TIMESTAMPTZ NOT NULL DEFAULT (NOW() +
        INTERVAL '24 hours'),
38     created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
39 );
40
41 CREATE TABLE IF NOT EXISTS audit_log (
42     id          BIGSERIAL PRIMARY KEY,
43     actor       TEXT NOT NULL, actor_role TEXT,
44     action      TEXT NOT NULL,                -- ack/
        resolve/approve/reject/...
45     target_type TEXT NOT NULL, target_id BIGINT NOT NULL,
46     diff        JSONB,
47     occurred_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
48 );

```

Mã nguồn E.1. DDL khởi tạo bốn bảng cốt lõi (001_init.sql, rút gọn).

Phép gộp các lần xuất hiện lặp của cùng một sự cố dựa trên ràng buộc duy nhất UNIQUE (incident_key) kết hợp câu lệnh INSERT ... ON CONFLICT (incident_key) DO UPDATE (mô tả ở mục 4.6.2). Hàm phụ trợ severityRank() trong cùng migration sắp thứ tự mức nghiêm trọng để phép ON CONFLICT chỉ nâng (không hạ) mức của sự cố đã có.

CHƯƠNG F

KỊCH BẢN DEMO VÀ MINH CHỨNG VẬN HÀNH

Phụ lục này mô tả quy trình *demo có thể tái lập* dùng để minh chứng hệ thống chạy thật trên cụm: từ việc xác nhận eBPF can thiệp ở tầng nhân, mô phỏng tấn công, tới vòng phát hiện → phản hồi → cách ly → xác minh, cùng cách truy cập kho đối tượng MinIO và cách làm cho dashboard quan trắc hiển thị dữ liệu. Mọi lệnh đều chạy trên cụm thật (3 node: userver-master, userver-worker-1, userver-worker-home).

F.1. Chuẩn bị và xác nhận hiện trạng cụm

```
1 $ kubectl get nodes
2 $ kubectl -n zt-mapper get pods -o wide # DaemonSet thu
    thap/pipeline tren moi node
3 $ kubectl -n soc get pods # NATS, postgres,
    aggregator, incident, console, minio
```

Mã nguồn F.1. Xác nhận node và các pod thành phần đang chạy.

DaemonSet zt-pipeline-ebpf hiện diện trên cả ba node (mỗi pod gồm ba container collector/pipeline/alert-bridge); các dịch vụ SOC nằm trong không gian tên soc.

F.2. Minh chứng eBPF can thiệp ở tầng nhân

Đây là phần cốt lõi cần thể hiện khi bảo vệ. Mục tiêu: chứng minh chương trình eBPF đã *nạp vào nhân, qua trình kiểm chứng, được JIT, gắn móc vào tcp_v4_connect*, và đang *bắt sự kiện* theo thời gian thực. Vì DaemonSet thu thập chạy hostNetwork và đặc quyền, chương trình eBPF nằm trong nhân của *host*; do đó bpftool được chạy *trên chính node* (đăng nhập node qua SSH, hoặc dùng container gõ lỗi mức node):

```
1 # Dang nhap node worker-1 (noi dat pod tan cong) roi chay:
2
3 # (1) Chuong trinh da nap: kieu kprobe, gan vao
    tcp_v4_connect, da qua verifier + JIT
4 $ sudo bpftool prog show | grep -A4 -i 'kprobe\|
    tcp_v4_connect '
5
6 # (2) Ba BPF map: cidr_map (lpm_trie), events (ringbuf),
    sock_store (hash)
7 $ sudo bpftool map show | grep -i 'cidr_map\|events\|
    sock_store '
8
```

```

9 # (3) Doi chieu: cac CIDR pod-network da nap vao bo loc LPM
10 $ sudo bpftool map dump name cidr_map | head
11
12 # (4) Tracing thoi gian thuc tu trong container (khong can
    bpftool):
13 #     moi connect() sinh mot su kien JSON
14 $ POD=$(kubectl -n zt-mapper get pod -l app.kubernetes.io/
    name=zt-pipeline-ebpf \
15     --field-selector spec.nodeName=userver-worker-1 \
16     -o jsonpath='{.items[0].metadata.name}')
17 $ kubectl -n zt-mapper logs $POD -c collector -f

```

Mã nguồn F.2. Bốn minh chứng eBPF ở tầng nhân (chạy trên node, chụp màn hình từng bước).

[Chèn ảnh chụp đầu ra của bốn lệnh trên – đây là minh chứng mạnh nhất cho phần eBPF.] Điểm cần chỉ ra khi trình bày: dòng kprobe ... tcp_v4_connect (điểm móc), các trường xlated/jited (đã qua verifier và JIT), và ba map đúng kiểu lpm_trie/ringbuf/hash. So với một agent ở không gian người dùng, đây là bằng chứng hệ thống quan sát *tại nguồn* trong nhân với chi phí cố định trên mỗi lời gọi.

F.3. Mô phỏng tấn công: quét cổng và lan truyền ngang

Một pod đóng vai kẻ tấn công trong không gian tên zt-targets thực hiện quét cổng connect (nmap -sT) tới nhiều pod dịch vụ, vượt ngưỡng $\tau = 40$ kết nối trong cửa sổ thời gian để kích hoạt cảnh báo, đồng thời chạm tới nhiều đích để kích hoạt cảnh báo *blast radius*.

```

1 # Quet cong connect-scan toi dai pod dich vu (vuot nguong
    -> port_scan)
2 $ kubectl -n zt-targets exec attacker -- \
3     nmap -sT -p 1-200 --max-rate 200 10.244.204.0/24
4
5 # Chạm toi nhieu dich de kích hoạt blast_radius (lan truyền
    ngang)
6 $ kubectl -n zt-targets exec attacker -- \
7     sh -c 'for ip in 10.244.204.{150..180}; do nc -w1 -z
    $ip 80; done'

```

Mã nguồn F.3. Kích bản tấn công từ pod attacker.

F.4. Vòng phát hiện – phản hồi – cách ly – xác minh

```

1 # (1) Pipeline phát cảnh báo (port_scan / blast_radius)
2 $ kubectl -n zt-mapper logs $POD -c pipeline | grep ALERT
3
4 # (2) Dịch vụ xử lý tạo sự cố + tự sinh bản nhập
    NetworkPolicy
5 $ kubectl -n soc logs deploy/zt-incident-service | grep -E
    'incident.created|action'
6
7 # (3) Phê duyệt trên console (console.zt.local) -> hành
    động chuyển 'executed'
8 #     -> NetworkPolicy cách ly được áp
9 $ kubectl -n zt-targets get networkpolicy | grep quarantine
10
11 # (4) Xác minh hiệu lực: pod tan công mất kết nối đi ra
12 $ kubectl -n zt-targets exec attacker -- nc -w2
    10.244.204.161 80; echo "exit=$?"
13 # exit=1 (trước khi cách ly: exit=0)

```

Mã nguồn F.4. Quan sát vòng end-to-end từ phát hiện tới cách ly.

[Chèn ảnh chụp console: hàng đợi phê duyệt, chi tiết sự cố, và trạng thái `executed` của hành động cách ly – xem các Hình ở mục 4.7.]

F.5. Truy cập kho đối tượng MinIO

MinIO (không gian tên soc) là kho đối tượng lưu báo cáo tuần, bản sao lưu cơ sở dữ liệu và bản nháp NetworkPolicy. MinIO *không* mở ra ngoài qua ingress; truy cập an toàn bằng cách chuyển tiếp cổng tới giao diện console (:9001):

```

1 # Chuyển tiếp cổng console MinIO ra máy cục bộ
2 $ kubectl -n soc port-forward svc/minio 9001:9001
3 # Mo http://localhost:9001 (user: zt-soc-admin)
4
5 # Hoặc dùng mc qua port-forward API (:9000)
6 $ kubectl -n soc port-forward svc/minio 9000:9000 &
7 $ mc alias set zt http://localhost:9000 zt-soc-admin '<
    minio-password>'
8 $ mc ls zt # zt-postgres-backups, zt-reports, zt-pcap-
    dumps, zt-netpol-archives

```

Mã nguồn F.5. Truy cập MinIO console và liệt kê bucket.

Trên trang *Reports* của console, nút *Generate & upload to MinIO* kết xuất báo cáo

tuần và đẩy lên bucket `zt-reports`; có thể kiểm chứng bằng `mc ls zt/zt-reports`.

F.6. Làm cho dashboard quan trắc hiển thị dữ liệu

Console/Reports đọc trực tiếp PostgreSQL nên luôn hiển thị đầy đủ sự cố tích lũy. Trái lại, dashboard Grafana là góc nhìn theo cửa sổ thời gian và phụ thuộc nguồn thu thập, nên có thể trống dù cơ sở dữ liệu đầy. Để dashboard hiển thị dữ liệu cần:

1. **Bảng *Alerts Overview* (nguồn Loki).** Nới cửa sổ thời gian (mặc định 30 phút) lên phạm vi bao trùm đợt tấn công, hoặc chạy lại kịch bản tấn công ở mục F.3 ngay trước khi xem để có nhật ký cảnh báo trong cửa sổ.
2. **Bảng *Incidents & Actions* (nguồn Prometheus).** Các bộ đếm (`zt_incidents_created_total`, `zt_aggregator_active_incidents` ...) chỉ hiển thị khi Prometheus thật sự thu thập điểm `/metrics` của dịch vụ SOC. Cụm dùng Prometheus Operator (`kube-prometheus-stack`), vốn khám phá mục tiêu qua ServiceMonitor chứ không qua chú thích `prometheus.io/scrape`; vì vậy cần khai báo một ServiceMonitor cho các dịch vụ SOC (kèm nhãn `release` mà Operator chọn). Ngoài ra các bộ đếm là biến đếm trong tiến trình – bị đặt lại 0 mỗi khi pod khởi động lại – nên chúng phản ánh hoạt động kể từ lần khởi động gần nhất, không phải lịch sử.

Đây cũng là một giới hạn quan trắc được bàn ở Chương 5: cơ sở dữ liệu là nguồn dữ liệu bền vững và là hệ thống ghi nhận chính thức của các số liệu sự cố; dashboard là lớp quan trắc thời gian thực bổ trợ.